GeLB User Tutorial

Release 1.0

Dragos-Bogdan Chirila

Nov 01, 2026

Contents

A Orientation **2**

1 Why GeLB? **3**
1.1 What makes GeLB different . . . 5
1.2 Errors at the domain level, not the generated-code level . . . 5
1.3 Restrictive on the surface, expressive underneath . . . 7
1.4 Who this tutorial is for . . . 8
1.5 Where to next . . . 8

2 Getting Started **9**
2.1 Installation . . . 9
2.2 The three CLI tools . . . 11
2.3 Your first simulation . . . 12
2.4 Troubleshooting . . . 14
2.5 Working in VS Code . . . 14
2.6 Where to next . . . 16

3 GeLB in 10 Minutes **18**
3.1 The anatomy of a GeLB program . . . 18
3.2 Lattices . . . 19
3.3 Reserved implicit symbols . . . 20
3.4 Reserved words . . . 21
3.5 The `f.in` / `f.out` contract . . . 21
3.6 Built-in intrinsics . . . 22
3.7 User-defined functions . . . 23
3.8 Node classes and geometry . . . 23
3.9 Types . . . 24
3.10 Expressions and control flow . . . 25
3.11 Command-line constants . . . 26
3.12 Two-minute “what just happened” . . . 26

B A guided tour **27**

4 Hello World **28**
4.1 The program . . . 28
4.2 Running it . . . 31
4.3 What the compiler actually emitted . . . 32

4.4 What's next . . . 33

5 1D Diffusion **34**
5.1 The physics . . . 34
5.2 The program . . . 35
5.3 Running it . . . 39
5.4 The compiler catches a common mistake . . . 39
5.5 Peeking at the stencil . . . 40
5.6 What's next . . . 41

6 Lid-Driven Cavity **42**
6.1 The physics . . . 42
6.2 D2Q9 and the macroscopic moments . . . 43
6.3 The program . . . 43
6.4 Geometry: painting the walls . . . 47
6.5 Running it . . . 47
6.6 A note on the Mach number . . . 48
6.7 Corners and the no-slip discontinuity . . . 49
6.8 Quantitative validation against Ghia, Ghia & Shin (1982) . . . 50
6.9 What's next . . . 51

7 Poiseuille with Body Force **52**
7.1 The physics . . . 52
7.2 Why MRT, and why now? . . . 53
7.3 Body forcing with Strang splitting . . . 53
7.4 Bounce-back — what, and why . . . 54
7.5 The program . . . 55
7.6 Running it . . . 57
7.7 Summary . . . 58
7.8 What's next . . . 58

8 Rayleigh-Bénard Convection **60**
8.1 The physics . . . 60
8.2 The LBM strategy . . . 61
8.3 Declaring two lattices . . . 61
8.4 Equilibrium functions and the coupled MRT dynamic block . . . 62
8.5 The geometry and wall boundary conditions . . . 63
8.6 Running it . . . 65
8.7 Validation: the snapshot regression test . . . 65
8.8 What's next . . . 66

9 Kármán Vortex Street **67**
9.1 The physics . . . 67
9.2 The GeLB strategy . . . 68
9.3 Conditional paint and the `InCircle` predicate . . . 68
9.4 `border()` — automatic wall-layer inference . . . 68
9.5 The `masked` modifier and empty node classes . . . 69
9.6 The program . . . 69
9.7 Running it . . . 71
9.8 Summary . . . 72
9.9 What's next . . . 73

10 Shan-Chen Multiphase **74**
10.1 The physics . . . 74
10.2 How Shan–Chen force injection works . . . 75

10.3 Non-local reads in GeLB . . . 75
10.4 The rest of the dynamic block . . . 76
10.5 The initializer: random initial density . . . 77
10.6 Running it . . . 77
10.7 Summary . . . 79
10.8 What's next . . . 79

11 Beyond LBM: Conway's Game of Life **80**
11.1 The rules . . . 80
11.2 The GeLB program . . . 81
11.3 The initializer: a glider at (1, 1) . . . 81
11.4 The dynamic: Conway's rules in four lines . . . 81
11.5 The gauge and geometry . . . 82
11.6 Running it . . . 82
11.7 Why this matters . . . 84
11.8 What's next . . . 84

12 Reduced-Gravity Ocean Circulation **85**
12.1 The physics . . . 85
12.2 The GeLB program . . . 86
12.3 The geometry — 9 node classes . . . 88
12.4 Running it . . . 89
12.5 What this demonstrates . . . 89

C Reference material 92

13 Boundary Conditions Cookbook **93**
13.1 Static Dirichlet (fixed value) . . . 93
13.2 Neumann flux (time-dependent) . . . 93
13.3 Moving-wall bounce-back . . . 94
13.4 Bounce-back written out per link . . . 94
13.5 Yoshida anti-bounce-back (fixed-T thermal wall) . . . 94
13.6 Periodic boundaries . . . 95
13.7 Zou-He velocity / pressure . . . 95
13.8 Masked node classes (obstacle interiors) . . . 95

14 Source Visualisation: `gelb-view source` **96**
14.1 Quick start . . . 96
14.2 Single-view subcommands . . . 97
14.3 Reading the combined view . . . 97
14.4 Lint warnings (`--lint`) . . . 97
14.5 VSCode integration . . . 97
14.6 When to use it . . . 98

15 Performance and Backends **99**
15.1 The three backends . . . 99
15.2 When to use which . . . 99
15.3 Performance numbers . . . 100
15.4 Practical notes . . . 101
15.5 Hand-tuning hot kernels . . . 101

16 Validation and Convergence **104**
16.1 The validation script . . . 104
16.2 Poiseuille convergence — the reference result . . . 104

16.3 Taylor-Green vortex decay . . . 105
16.4 Numerical conformance . . . 105
16.5 Known limitations . . . 105
16.6 Adding your own validation . . . 106

17 Numerical Semantics **107**
17.1 Build modes . . . 107
17.2 Types and promotion . . . 108
17.3 Special values . . . 110
17.4 What the optimizer promises . . . 110
17.5 Library functions . . . 111
17.6 Reproducibility . . . 111
17.7 What must agree . . . 112
17.8 Random numbers . . . 113
17.9 Constant folding . . . 114
17.10 Not in this version . . . 115

18 Further Reading **116**
18.1 Textbook cross-reference . . . 116
18.2 Bibliography . . . 117

Bibliography **118**

A guided tour through the Lattice Boltzmann Method using GeLB.

This manual teaches you how to write, run, and visualise LBM simulations in GeLB. It assumes some prior exposure to the Lattice Boltzmann Method — if BGK collision and streaming are unfamiliar, work through chapters 1–3 of Krüger, Kusumaatmaja, Kuzmin, Shardt, Silva & Viggen, *The Lattice Boltzmann Method: Principles and Practice* (Springer, 2017) before starting here. Everything else you need to know about GeLB is in these pages.

The tutorial is organised in three parts:

Part A — Orientation. Why GeLB exists, how to install it, and a two-page language cheat-sheet ("GeLB in 10 minutes") that you can return to as a reference.

Part B — A guided tour through LBM with GeLB. Nine worked examples, each anchored by one `.gelb` file from the repository. Chapters build on each other: every new GeLB feature is introduced exactly once. The physics goes from 1D diffusion through 2D Navier-Stokes, MRT collision, multi-lattice Rayleigh-Bénard convection, obstacle flows and Shan-Chen multiphase to a reduced-gravity ocean model. One chapter steps outside LBM: Conway's Game of Life, written in GeLB, shows that GeLB compiles stencil updates in general, not only lattice Boltzmann models.

Part C — Reference material. Boundary condition cookbook, stencil visualisation, performance and backends, validation and convergence, numerical semantics, and a cross-reference table mapping each tutorial chapter to the relevant sections of Krüger et al. and Mohamad (2nd ed.).

> **Note**
>
> All tutorial figures are reproducible from scratch by running `python scripts/gen_tutorial_figures.py` from the repository root, and the VS Code screenshots by `node scripts/gen_vscode_screenshots.mjs` (Linux, under `xvfb`). Committed figures under `docs/user/figures/` come with a `MANIFEST.txt` recording the git commit they were rendered at.

Acknowledgements. GeLB grew out of research supervised by Gerrit Lohmann (Alfred Wegener Institute, Bremerhaven), who secured its funding and whose questions and feedback shaped both the language and this tutorial.

Part A

Orientation

CHAPTER 1

Why GeLB?

The Lattice Boltzmann Method is one of the most widely used mesoscopic fluid solvers in computational science. It is also, in practice, one of the most error-prone to implement from scratch. A typical research code is a few thousand lines of C++ or Fortran mixing a physics kernel with boundary conditions, forcing terms, collision operators, checkpoint I/O, and parallelisation directives; a single off-by-one in a streaming step can silently break the mass balance or quietly break second-order accuracy at walls. Established frameworks (OpenLB, Palabos, waLBerla, lbmpy) cut that cost, but they are *libraries*: you still write C++ or Python, you still assemble the framework's building blocks yourself, and the gap between "what the textbook says" and "what compiles" is wide enough to hide bugs.

GeLB takes a different stance. **GeLB is a language, not a library.** You describe a lattice, an initialiser, a dynamic, a gauge, and a geometry in a concise domain-specific syntax; the compiler decides how to lay out memory, which pull/push scheme to use, how to parallelise the loops over the cells, and how to handle ping-pong buffers. The same source compiles to OpenMP-parallel portable-SIMD C++ via Google Highway (`cpu_highway`, the default), to CUDA C++ (`gpu_cuda_cpp`), or to single-threaded Fortran (`cpu_fortran_serial`, the educational reference) with no source changes — you pick the backend when you run `gelb-run -b`. Compile-time validation catches lattice-shape mismatches, missing distribution-function initialisation, and out-of-bounds stencil reads *before* a single line of C++, CUDA or Fortran is generated. The result is a research workflow where the code you write looks like the paper you read.

A concrete number: measured across the 67 examples in the repository, the Fortran reference backend writes **5.6× more code** than the GeLB source on a size-weighted SLOC basis — a GeLB program is **82% shorter** than the generated Fortran, and the ratio stays in that range whether you look at 1D diffusion (3.2×), 2D LBM benchmarks (4.6×), or 3D flows (4.4×). And the code you don't write runs in parallel: the default `cpu_highway` backend uses all cores and SIMD lanes, and the CUDA backend uses the GPU — same `.gelb` source, no code changes, no parallel directives written by hand, no race conditions to debug. Numbers are in *Performance and Backends*.

How short is short? Here is the complete 1D heat-diffusion program from *1D Diffusion*; what the compiler generates from it is described below.

GeLB source — 65 lines, one file, one kernel:

```
simulation_description("1D heat diffusion with Dirichlet BCs (D1Q3)");

global_constants {
    const int GRID = 32;
```

(continues on next page)

(continued from previous page)

```
    const array[real, 3] W  = {4.0/6.0, 1.0/6.0, 1.0/6.0};
    const array[real, 3] EX = {0.0, 1.0, -1.0};
    const real tau   = 0.75;
    const real omega = 1.0 / tau;
    const real T_left  = 1.0;
    const real T_right = 0.0;
}

lattices {
    lattice f(1, 3, real, GRID, false);
}

function DiffusionEq(real T) -> (array[real, 3] feq) {
    feq = W * T;
}

initializer InitBulk  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitLeft  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitRight { f.out[x | :] = DiffusionEq(T_right); }

dynamic CollideAndStream {
    array[real, 3] fIn = upstream_dfs(f, x, EX);
    real T = sum(fIn);
    array[real, 3] feq = DiffusionEq(T);
    f.out[x | :] = (1.0 - omega) * fIn + omega * feq;
}

gauge Temperature -> (real temperature) {
    temperature = sum(f.in[x | :]);
}

node_classes {
    node_class BULK(InitBulk, CollideAndStream, Temperature);
    node_class LEFT_WALL(InitLeft, Temperature);
    node_class RIGHT_WALL(InitRight, Temperature);
}

geometry {
    nodes[:]        = BULK;
    nodes[0]        = LEFT_WALL;
    nodes[GRID - 1] = RIGHT_WALL;
}
```

What the default backend generates from it is one C++ file, `heat_diffusion_1d_dirichlet.cpp`, of **285 lines**: the lattice storage and the constants, one loop per node class for the initializers, the gauge and its NetCDF output at the initial step, then the time loop with the collision-and-streaming loop and the gauge writes, and a timing report at the end. You usually don't have to read it, but `gelb-run -k` keeps it (and the binary) in the current directory if you want to, and it is the file to attach to a bug report. The ratio of what-you-write to what-actually-runs is something like one page of GeLB to four pages of generated code — for a problem where every line of the generated code needs to be correct for the physics to be correct.

1.1 What makes GeLB different

A real language, not a library.
OpenLB, Palabos, waLBerla, and lbmpy are libraries. You still write C++ or Python, and you assemble the framework's building blocks yourself, in the way the framework expects. GeLB's building blocks are the notions of the method itself — lattice, initializer, dynamic, gauge, node class, geometry — written in a language with its own grammar, type system, and validation passes. The user writes physics; the compiler decides backend strategy. That is a different product, with a different workflow.

Backend portability that actually works.
The same `.gelb` source compiles to OpenMP-parallel portable-SIMD C++ via Google Highway (`cpu_highway`, the default), to CUDA C++ (`gpu_cuda_cpp`), or to single-threaded Fortran (`cpu_fortran_serial`, the educational reference) with no source changes. The `f.in` / `f.out` ping-pong illusion is documented as a *language-level guarantee*, so backends are free to drop the second buffer without breaking user code. See *Performance and Backends*.

A type system that catches LBM bugs early.
Implicit symbol scoping (`x`/`y`/`z` are reserved in simulation blocks, but usable as function parameters), lattice-access context validation (`f.out` is writable in `initializer` / `dynamic`; `f.in` is readable in `dynamic` / `gauge`; anything else is a compile-time error), function-recursion detection, `masked` keyword enforcement, stencil extraction driving boundary-access checks. These are not generic compiler features — they target the mistakes LBM users actually make.

End-to-end tooling.
`gelbc` (compiler), `gelb-run` (compile-and-run wrapper), `gelb-view data` (interactive NetCDF viewer with vector fields, streamlines, and particle animation), `gelb-view source` (geometry heatmap + stencil diagrams + geometry-lint checks), a VSCode extension (`vscode-gelb`) with live preview, Problems-panel diagnostics, and an interactive 3D geometry explorer, NetCDF output, validation against analytical solutions (`scripts/validate_analytical.py`), MLUPS measurement.

Algorithmic clarity as a headline feature.
The cascaded MRT collision operator in GeLB reads like the paper. The `border()` + `nodes[where InCircle(x, y, ...)] = SOLID` pattern reads like the textbook. The Zou-He velocity BC reads like the derivation. Part of the value proposition is the reproducibility of boundary-condition, forcing and collision combinations; GeLB tries to offer that to its users.

1.2 Errors at the domain level, not the generated-code level

When you make a mistake in a hand-written LBM code, the C++ or Fortran compiler catches *some* of it — typos, undeclared variables, type mismatches. Everything else — wrong DF index, unwritten distribution function, bounce-back on the wrong direction, a boundary region whose index range silently clips at the grid edge — shows up either as a `NaN` in the NetCDF output several hours into the run, or (worse) as a plausible-looking but physically wrong result.

GeLB rejects most of those mistakes before it emits a single line of generated code. The compiler understands the domain — distribution functions, node classes, stencils, geometry — and its error messages are phrased in the domain. A few real examples from deliberately broken programs:

Mistake: writing to `f.in` in a dynamic block (`f.in` is read-only by language contract):

```
      ...
   6  }
   7  dynamic Collide {
   8->    f.in[x | 0] = 1.0;
            ^^
[Problem] Your GeLB program contains a syntax error.
```

(continues on next page)

(continued from previous page)

```
[Hint] mismatched input 'in' expecting 'out'
   9  }
  10  gauge G -> (real v) { v = sum(f.in[x | :]); }
      ...
```

This one is caught by the parser itself, hence the generic message; the hint names `out` as the required write target.

Mistake: an initializer that only writes some of the distribution functions:

```
      ...
   2  lattices { lattice f(1, 3, real, 16, false); }
   3
   4->initializer Init {
                  ^^
[Problem] Initializer 'Init' (for node class 'BULK') does not write all DFs␣
↪of lattice 'f': missing DFs [1, 2]
[Hint] Add write statements for the missing distribution functions, or use␣
↪the full-range syntax f.out[...|:] to write all DFs at once.
   5      f.out[x | 0] = 0.5;
   6  }
      ...
```

Mistake: a geometry paint range that extends past the grid (this one bites beginners from Python backgrounds, because GeLB's paint ranges are inclusive on both ends):

```
      ...
  11  geometry {
  12      nodes[:, :] = BULK;
  13->    nodes[:, 0:N] = WALL;
                          ^^
[Problem] Geometry paint to 'WALL' is out of bounds: y-range [0, 16] extends␣
↪outside [0, 15]
[Hint] GeLB paint ranges are INCLUSIVE on both ends.
  14  }
```

Mistake: referencing a node class that wasn't declared (here a copy-paste typo):

```
      ...
  10  gauge G -> (real v) { v = sum(f.in[x | :]); }
  11  node_classes { node_class BULK(Init, Collide, G); }
  12->geometry { nodes[:] = WALL_TYPO; }
                            ^^
[Problem] You mention WALL_TYPO. However, you did not define it.
[Hint] Define WALL_TYPO.
```

Every error quoted above is what `gelbc` prints for the program in question, with two lines of context on each side. The pattern is the same for every compile-time check GeLB performs: the offending token is underlined, the problem is described in domain terms, and a hint tells you how to fix it. The goal is that a wrong program fails quickly and informatively, so the expensive feedback loop of "compile → run simulation → stare at NaN → diff against analytical solution → guess what went wrong" is replaced by a sub-second round-trip with a helpful error message.

This is the second main value proposition, alongside compactness. The compiler is not just a translator — it is a reviewer that checks your LBM program in the language of LBM.

1.3 Restrictive on the surface, expressive underneath

GeLB looks like a smaller language than it is. On first contact the list of "missing" features is conspicuous:

- **No explicit loops** — no `for`, no `while`. The per-cell loop around every `initializer` / `dynamic` / `gauge` block is implicit; you never write it.
- **No data-dependent branches** — no `if`, no `else`, no `switch`. Branching on spatial position is expressed by painting different node classes onto different regions of the geometry; branching on a field value is expressed via the `min` / `max` / `abs` intrinsics or the multiplicative `real(cond)` cast, the same on every backend.
- **No mutable state outside lattices** — the only objects that persist across time steps are lattices (and the outer loop counter `t`). Function-local variables exist but are scoped to a single cell and die at the end of the block.
- **No side effects in user functions** — they cannot read or write lattices, cannot call `random()`, cannot recurse.
- **No ``print`` / ``write``** — every output goes through a gauge block and lands in NetCDF.

Each of these restrictions is there for a single reason: **so that the compiler can parallelise every kernel across every cell with no synchronisation, no race conditions, and no hand-tuned directives.** The per-cell kernel has no control-flow branches that depend on other cells' state, which means the same source compiles unchanged to the multi-threaded SIMD C++ backend, to CUDA, to the single-threaded Fortran reference, and to any future MPI backend. You get the parallel speed reported in *Performance and Backends* for free, precisely because you cannot write the kind of code that would break it.

It would be fair to worry that the restrictions are too strict. In practice they aren't. LBM is already a data-parallel method by construction — every cell's collision and streaming step is a pure function of its own and its neighbours' previous-timestep state — and every common LBM idiom can be written as a closed-form expression of that state. A few concrete examples of "how do I do X without `if`?":

Obstacles and solid boundaries. Paint a `SOLID` node class onto the obstacle region, and use a different dynamic block (bounce-back, or no dynamic at all) for that class. The `border()` geometry statement and conditional paints (`nodes[where InCircle(x, y, cx, cy, r)] = SOLID`) cover anything from a single point to an arbitrary pixel-mask geometry. See *Kármán Vortex Street* for flow past a circular obstacle.

Multiple collision operators in one simulation. Use BGK in the bulk and MRT near walls, or LES in high-shear regions and plain BGK elsewhere. Each is its own node class with its own `dynamic` block; geometry paints decide which cells run which. The LES example in `examples/lbm/2d/lid_driven_cavity_2d_bgk_smagorinsky.gelb` recomputes τ per cell per time step from the local non-equilibrium stress tensor — all in one branch-free dynamic block.

Branch on a scalar value. You cannot write `if (rho < 0.0) { abort(); }`. GeLB does not stop on NaNs or prove physical constraints such as a positive density; inspect the simulation's output. For domain-boundary smoothing, use `max(0.0, rho - rho_min)` or an equivalent branch-free expression. For a choice between two finite values, convert each condition with `real(...)` and use it as a 0/1 weight: `real(v < 4.0) * a + real(v >= 4.0) * b` (both branches are evaluated).

Multi-lattice coupling. Two lattices (fluid + temperature, or fluid + species), read each other's `f.in` in a dynamic block, coupled via a source term that uses both. No synchronisation primitives, no parallel constructs: the language presents every block as if it ran sequentially over the cells, reading the previous time step and writing the next (currently backed by a pair of ping-pong buffers). See *Rayleigh-Bénard Convection* for Boussinesq-coupled thermal convection.

Multiphase flow. Shan-Chen pseudopotentials, free-energy models, phase-field: all expressible via a single non-local force term in the dynamic block that reads `f.in` at eight neighbours and computes a local force. See *Shan-Chen Multiphase*.

Non-LBM kernels. GeLB's per-cell model is stencil-kernel- general, not LBM-specific. The chapter *Beyond LBM: Conway's Game of Life* implements Conway's Game of Life — where the "DF" is a single integer, the "equilibrium" is the neighbour-count rule, and the "streaming" is an 8-neighbour Moore stencil (the cell's eight surrounding cells,

the neighbourhood of cellular-automata theory) — in 37 lines of GeLB, using exactly the same language constructs as every LBM example in the tutorial.

The pattern across all of these is the same: describe the per-cell kernel as a closed-form expression in the previous time step's data plus compile-time constants, and let the compiler materialise the implicit loop (readers who know functional programming will recognise the style). When you come from a C or C++ background the missing control-flow features feel restrictive at first. Two or three chapters into this tutorial, you will hopefully stop noticing — they turn out not to have been needed.

1.4 Who this tutorial is for

If you know LBM already — you have read Krüger et al. or Mohamad or Sukop, and you have implemented a D2Q9 kernel by hand at least once — this tutorial will get you productive in an afternoon. Parts A and B take you through every language feature via worked examples from the repository; Part C is reference material you can come back to when you need to look up a boundary condition idiom or a performance tuning hint.

If you do not know LBM yet, this tutorial will teach you GeLB but it will not teach you the physics. Read Krüger *et al.*, *The Lattice Boltzmann Method: Principles and Practice* (Springer, 2017), chapters 1–5, first. Chapters 1–2 cover the hydrodynamic and kinetic-theory background, chapter 3 introduces the Lattice Boltzmann equation proper, chapter 4 does the Chapman-Enskog analysis, and chapter 5 covers boundary conditions — after which the *Hello World* chapter of *this* tutorial will feel natural. Mohamad's *Lattice Boltzmann Method: Fundamentals and Engineering Applications with Computer Codes* (2nd ed., Springer 2019) is a gentler, more applied alternative.

1.5 Where to next

- *Getting Started* — install GeLB, compile and run your first simulation.
- *GeLB in 10 Minutes* — the language cheat-sheet; bookmark it.
- *Hello World* — the simplest possible GeLB program, annotated line by line.

See also

The LOC numbers above are generated by `scripts/measure_loc_savings.py`, which compiles every `.gelb` file under `examples/` with the `cpu_fortran_serial` backend and counts source and generated lines (raw `wc -l` and SLOC, size-weighted and per-example). Run it yourself to see the breakdown by example category:

```
uv run python scripts/measure_loc_savings.py --backend cpu
```

CHAPTER 2

Getting Started

This chapter walks you from a clean checkout to a successful first run in about ten minutes. By the end of it you will have installed GeLB, compiled a small example, run the resulting binary, and viewed its output, first from the command line and then from Visual Studio Code, where you will probably do most of your work.

2.1 Installation

GeLB is developed on Linux and macOS; Windows is supported via PowerShell scripts but less exercised. All dependencies are installed into a self-contained virtual environment by a single bootstrap script. The supported install is a source checkout (`git clone`, then the bootstrap); packaged installers for macOS and Ubuntu, which install the same three commands without a checkout, are planned for a later release.

Hard requirements (the bootstrap script refuses to run without these):

- Python ≥ 3.11
- `git`
- `curl` (used to fetch `uv`, which manages the Python environment)
- An internet connection for the initial dependency download

Soft requirements (not checked by the bootstrap; `gelb-run` needs them for the backends you use):

Backend	Compiler	Output library
`cpu_highway` (default)	A C++17 compiler with OpenMP: `g++` on Linux (part of GCC); on macOS `brew install gcc`, or `brew install libomp` for Apple's `clang++`. The Google Highway sources come with the checkout as a git submodule, which the bootstrap script initialises.	NetCDF-C: `libnetcdf-dev` on Debian/Ubuntu, `brew install netcdf` on macOS; detected via `nc-config`.
`gpu_cuda_cpp`	The CUDA toolkit (`nvcc`); see *Performance and Backends*.	NetCDF-C, as above.
`cpu_fortran_seri` (educational reference)	`gfortran` (part of GCC on Linux; `brew install gcc` on macOS).	NetCDF-Fortran: `libnetcdff-dev` on Debian/Ubuntu, `brew install netcdf-fortran` on macOS; detected via `nf-config`.

Clone the repository and run the bootstrap script:

```
git clone https://github.com/dchirila/gelb.git
cd gelb
./scripts/gelb_bootstrap.sh        # Linux / macOS
# .\scripts\gelb_bootstrap.ps1     # Windows
```

The script installs `uv` if it is not already on your system, creates a `.venv/` directory, and pins dependencies from `pyproject.toml`. It takes about a minute on a warm cache.

The three GeLB commands, `gelbc`, `gelb-run` and `gelb-view`, are installed in `.venv/bin`. To make them plain commands, the bootstrap ends by adding one PATH line to your shell's startup file (`~/.bashrc` for bash on Linux, `~/.bash_profile` on macOS, `~/.zshenv` or `~/.zprofile` for zsh, `~/.tcshrc` for tcsh, `~/.config/fish/config.fish` for fish; it prints which file it changed, and running it again never adds a second line). Open a new terminal, or paste the one-liner the bootstrap prints into the current one, and the commands work from any directory: every command in this tutorial is written that way, and a packaged installation uses the same words. If you would rather keep your startup files as they are, delete that line and put `uv run` in front of each command instead (`uv run gelb-run ...`, and the same for `gelbc` and `gelb-view`); `uv run` must then be typed from inside the checkout. The same line can be added for another shell with `scripts/gelb-shell-setup "$PWD/.venv/bin" zsh`.

Optional components:

- **Java** (any JDK ≥ 11) is needed *only* if you modify the grammar in `gelb/antlr_part/grammars/GeLB.g4` and need to regenerate the ANTLR4 parser via `./scripts/regenerate_antlr.sh`. Day-to-day use does **not** need Java: the generated parser files already come with the source checkout and with the installers.
- **CUDA toolkit (``nvcc``)** is needed for the GPU CUDA backend. See *Performance and Backends* for setup notes.

Verify the install by running the GeLB compiler's `--help`:

```
gelbc --help
```

You should see a long `Usage: gelbc [OPTIONS] INPUT_FILE` listing. If you do, the Python side of GeLB is up and ready. (`command not found` means the PATH line is not in effect yet: open a new terminal, or put `uv run` in front of the command, from inside the checkout.)

If you use Visual Studio Code, install the GeLB extension as well (*Working in VS Code* below): editing and running programs there is the most comfortable way to follow this tutorial. The command-line tools come first in this chapter because the extension runs them for you.

2.2 The three CLI tools

GeLB ships three command-line entry points. You will use all of them if you follow this tutorial.

2.2.1 `gelbc` — the compiler

`gelbc` reads a `.gelb` file and emits a single source file in the language of the chosen backend (C++ by default). It does no compilation or linking — just the source-to-source translation. A typical invocation:

```
gelbc examples/basic/1d/ex0_int_vec_setto_one.gelb \
    -o /tmp/hello.cpp \
    --num-timesteps 10
```

Useful flags:

- `-o FILE` — where to write the generated source.
- `--num-timesteps N` — number of simulation time steps to run. Default 1; you almost always want to set this.
- `--write-every N` — write gauge output every `N` steps (default: every step). Reduce this for long runs to save disk.
- `--backend` — `cpu_highway` (default; portable-SIMD C++ via Google Highway, OpenMP-parallel), `gpu_cuda_cpp` (CUDA C++), or `cpu_fortran_serial` (single-threaded educational reference).

The remaining flags are collected under *Advanced options* below.

2.2.2 `gelb-run` — compile, link, run

`gelbc` gives you C++, CUDA or Fortran source; `gelb-run` gives you an executable and runs it. It wraps `gelbc`, the native compiler, execution and cleanup into one command, analogous to `go run` or `cargo run`.

```
gelb-run -n 100 \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
```

That compiles the example, links it against NetCDF, runs it for 100 time steps, and writes the resulting `ex0_int_vec_setto_one.nc` into the current directory. Useful flags mirror those of `gelbc`, with a few additions:

- `-b BACKEND` — `cpu_highway` (default), `gpu_cuda_cpp`, or `cpu_fortran_serial`.
- `-n N` / `-w N` — `--num-timesteps` / `--write-every`.
- `-m stdout` — print gauge output to the terminal with `-b cpu_fortran_serial` instead of writing NetCDF (useful for tiny examples and debugging).
- `-V` — launch `gelb-view` automatically after the run.
- `-k` — keep the generated source and binary instead of cleaning them up.
- `-v` — print each compilation step.

The common options have the same short and long forms in both tools: `-b`/`--backend`, `-n`/`--num-timesteps`, `-w`/`--write-every`, `-m`/`--output-mode`, `-s`/`--seed`.

`gelb-run` also detects the NetCDF libraries and shows a progress bar during long runs; it inherits your shell's resource limits.

2.2.3 Advanced options

You will not need these for the tutorial; they are listed here for later reference. Each is accepted by `gelbc` and by `gelb-run` unless stated otherwise.

- `-D NAME=VALUE` — override a global constant at compile time (e.g. `-D N=128` to raise a resolution); repeatable. See *GeLB in 10 Minutes*.
- `--seed N` — fixed seed for reproducible runs of programs that use the `random()` intrinsic.
- `--stencils` (`gelbc` only) — print the stencil analysis report and exit; see *Source Visualisation: gelb-view source*.
- `--no-geometry-lint` — suppress the non-fatal geometry-lint warnings. By default the compiler warns about suspicious disconnected regions and isolated fluid pockets.
- `--poison-uninit` — debugging aid: fill lattice storage with NaN at startup instead of zeros, so a read of a lattice cell that no initializer or update ever wrote surfaces as NaN in the gauge output instead of hiding behind a benign zero. A correct program produces output bitwise identical to a normal run (masked cells are unaffected — they hold zero DFs by language guarantee).
- `OMP_NUM_THREADS=1` in the environment — a single-threaded `cpu_highway` run, for example when comparing against `cpu_fortran_serial`.

2.2.4 `gelb-view` — the visualiser

`gelb-view` is a PySide6 + matplotlib viewer. Subcommands:

```
# View simulation output (NetCDF)
gelb-view data out.nc
gelb-view data out.nc --no-gui -o plot.png

# View geometry and/or stencil diagrams from a .gelb source file
gelb-view source program.gelb
gelb-view source program.gelb --show=both --no-gui -o preview.png

# Bare filename works too — .gelb routes to `source`, .nc routes to `data`
gelb-view program.gelb
gelb-view out.nc
```

The `data` subcommand opens an interactive window with a source/ variable/plot tree pane, dockable plot panels, cross-section line plots, and multi-variable composite plots (vector fields, streamlines, vorticity, particle animation). Pass `--no-gui` or `-o` to run it headless; that mode is used throughout this tutorial to generate the committed figures under `docs/user/figures/`.

The `source` subcommand visualises both the geometry (per-cell node-class heatmap) and the per-block read/write stencils from a `.gelb` source file. Four render modes (`geo`, `algo`, `both`, `combined`), plus flags for geometry-lint warnings (`--lint`), stats (`--stats`), ASCII grid (`--text`), and a JSON export used by the VSCode extension (`--export-json`). See *Source Visualisation: gelb-view source* for the full flag reference.

PySide6 is optional (but highly recommended): `pip install 'gelb[gui]'` adds it if you want the interactive GUI. Without PySide6, only the `--no-gui` matplotlib path is available.

2.3 Your first simulation

Let's compile and run the smallest example in the repository — `examples/basic/1d/ex0_int_vec_setto_one.gelb`. The whole program is 30 lines:

```
simulation_description("Pretty much the hello world of GeLB");

lattices {
  lattice f(1,        // d
            1,        // q
            int,      // df_type
            32,       // grid_size
            false     // periodicity
  );
}

initializer SetDFsToZero {
  f.out[x | 0] = 0;
}

dynamic SetDFsToOne {
  f.out[x | 0] = 1;
}

gauge CurrentState -> (int state) {
  state = f.in[x | 0];
}

node_classes {
  node_class BULK(SetDFsToZero, SetDFsToOne, CurrentState);
}

geometry {
  nodes[:] = BULK;
}
```

It declares a 1D lattice of 32 integer cells, an initialiser that sets every cell to 0, a dynamic that overwrites every cell with 1 at every time step, and a gauge that reads the current value back out. The complete program in 30 lines. We will dissect this program properly in *Hello World*; for now, just run it.

Compile and run in one step:

```
gelb-run -b cpu_fortran_serial -n 10 -m stdout \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
```

The `-m stdout` flag tells the Fortran reference backend to print gauge values to the terminal instead of writing NetCDF — convenient for a tiny example. After the compiler's own messages it prints one gauge value per line: the 32 zeros of the initial state (`t=0`) first, then 32 ones for each of the ten steps (from `t=1` onwards the dynamic overwrites everything to `1`), 352 values in all, followed by a short timing report.

Now try the same thing without `-m stdout` — the default NetCDF mode:

```
gelb-run -n 100 \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
```

That writes an `ex0_int_vec_setto_one.nc` file to your current directory. Open it in `gelb-view`:

```
gelb-view data ex0_int_vec_setto_one.nc
```

The GUI opens with a space-time heatmap of the `state` variable — `0` in the first row, `1` in the 100 rows after it, across

all 32 cells (right-click a variable in the file tree for other plot types). Boring, as hello-world programs should be. We will start doing real physics in *1D Diffusion*.

If you want to see the generated code, pass `-k` to keep it:

```
gelb-run -k -n 10 \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
# prints the path of the generated file, kept next to the binary
```

`gelb-run` leaves `ex0_int_vec_setto_one.cpp` and the binary in the current directory. The file is 122 lines of C++: the lattice storage, the initializer loop, the gauge output, and the time loop that runs the dynamic. You don't need to read it. Throughout the rest of the tutorial we will mostly pretend the generated layer does not exist.

2.4 Troubleshooting

"gfortran not found" (`-b cpu_fortran_serial` only) — install GCC. On Ubuntu/Debian: `sudo apt install gfortran`. On macOS: `brew install gcc`.

"nf-config not found" (`-b cpu_fortran_serial` only) — install NetCDF-Fortran. On Ubuntu/Debian: `sudo apt install libnetcdff-dev`. On macOS: `brew install netcdf netcdf-fortran`. The default backend needs NetCDF-C (`nc-config`) instead.

"The ANTLR4 Java tool failed to compile the GeLB.g4 grammar" — you have picked up a fresh grammar change but do not have Java installed. Install a JDK (`brew install openjdk` on macOS, `sudo apt install default-jdk` on Ubuntu/Debian), then rerun `./scripts/regenerate_antlr.sh`. Day-to-day use does not need Java; this only fires when `GeLB.g4` changes and the generated parser is out of date.

"warning: VIRTUAL_ENV=… does not match the project environment path" — this is a harmless warning from `uv` when your shell has a different virtualenv active and you run the commands through `uv run`. You can silence it by running `deactivate` first; `gelb-run` itself clears the outer `VIRTUAL_ENV` before it invokes `uv`.

2.5 Working in VS Code

The GeLB extension for Visual Studio Code highlights the language, reports errors when you save, draws the geometry of a program, and runs it with `gelb-run` from a sidebar, opening `gelb-view` on the result. It does what the previous sections did by hand: the commands stay what stands behind it, and every "Running it" section of the tutorial gives both.

Installing it. From the repository root (the build needs Node.js and `npm`):

```
./scripts/install-vscode-ext.sh
```

The script builds the extension and copies it into `~/.vscode/extensions`. Reload VS Code afterwards (*Developer: Reload Window* in the Command Palette, `Ctrl+Shift+P`, or `Cmd+Shift+P` on macOS). The macOS installer offers to install the extension too. Then open the repository folder (*File* → *Open Folder…*): the extension finds `gelbc`, `gelb-view` and `gelb-run` in the checkout by itself, so there is nothing to configure.

Editing. Open a program, for example `examples/lbm/2d/lid_driven_cavity_2d_bgk.gelb`, the lid-driven cavity of *Lid-Driven Cavity*. The editor colours keywords, names and numbers, and the extension adds two buttons to the editor's title bar and a *Run* link above `simulation_description` (figure below). The GeLB icon in the activity bar on the left opens the GeLB sidebar: *Program Structure* lists the program's blocks and constants, and a click jumps to one.

Errors. Each time you save, the extension has `gelbc` check the file and lists what it finds in the *Problems* panel (*View* → *Problems*), with the compiler's message and hint; the name or number it points at is underlined in the editor (figure below). A click on the entry jumps to the line. The check runs on save, not while you type.

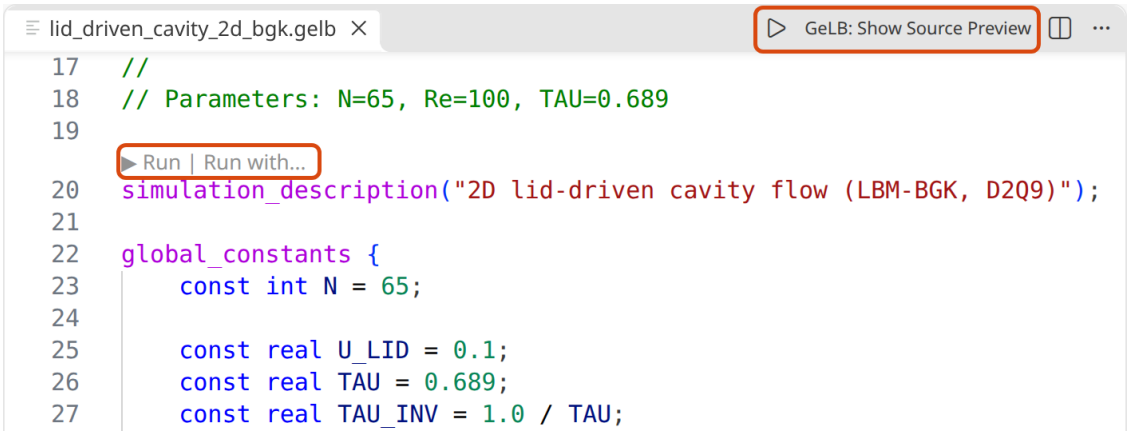


Fig. 1: A program open in VS Code. Outlined: the *Run* and *Run with…* links above `simulation_description`, and in the title bar the Run button (the triangle) and *GeLB: Show Source Preview*.

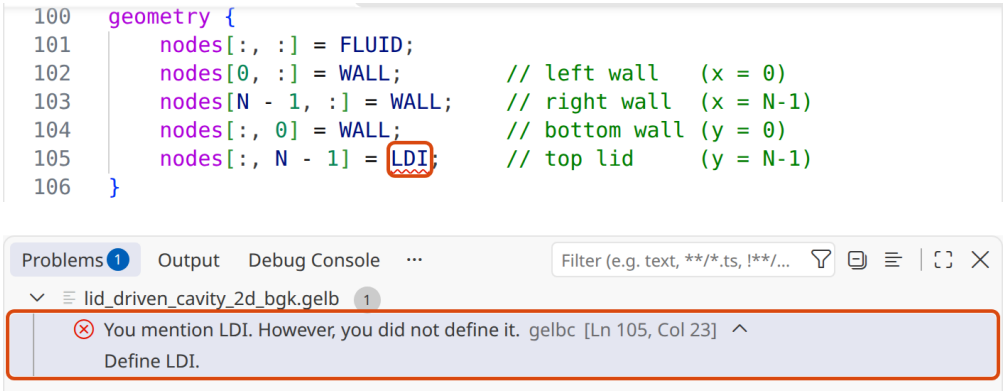


Fig. 2: `LID` misspelt as `LDI` in the `geometry` block, after saving: the name is underlined, and the *Problems* panel shows the compiler's message and hint.

The geometry. *GeLB: Show Source Preview* opens a preview beside the editor: the lattice with every node coloured by its node class, and a legend that counts them (figure below). It is the quickest check that the `geometry` block paints what you meant. Hover over a node to see its coordinates and class; click one to list the blocks of its node class and draw the cells the chosen block reads and writes, which `Space` then animates. The preview is redrawn each time you save, and refuses to open while the file has errors.

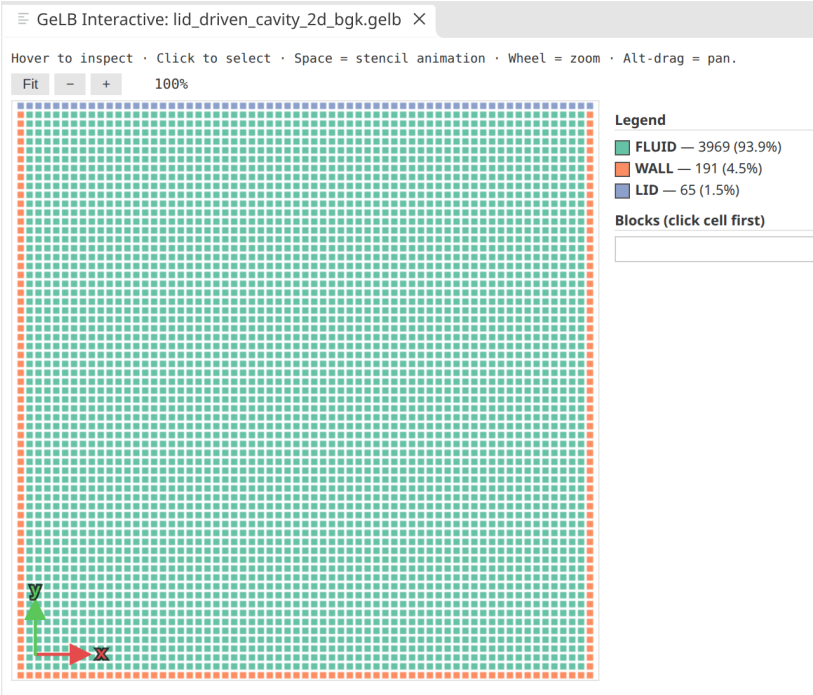


Fig. 3: The preview of the lid-driven cavity: 3969 fluid nodes, the walls on three sides and the moving lid on top.

Running. The sidebar's *Run* view (figure below) holds the parameters `gelb-run` would get on the command line, as rows you click to edit: the number of time steps (`-n`), how often the gauges are written (`-w`), constant overrides (`-D`), the backend (`-b`), whether to open the viewer at the end (`--view`), and the output folder, where the `.nc` file lands (next to the source file unless you choose another). They are remembered per file. *Run*, or the Run button of the editor, saves the file and starts `gelb-run` with them: its output, from the compiler messages to the progress bar and the MLUPS report, appears in a terminal of the panel named after the file, and the *Status* row follows the run. *Stop* ends it, together with everything it started. The view's *Show Run Log* button (in its title bar) shows, for each run, the command line it amounts to; you can paste that line into a terminal to add the less common `gelb-run` flags of *Advanced options*.

With *Open viewer* on, `gelb-view` opens on the output when the simulation ends (figure below). For a 2D program it shows a heatmap of the first variable at the last record; the slider below it steps through the records, and a right-click on a variable in the *Sources* tree offers the other plots. The run counts as going until you close the viewer; *Open last output in viewer* opens it again later.

2.6 Where to next

- *GeLB in 10 Minutes* — the language cheat-sheet. Bookmark it and come back to it whenever you need to look up a keyword.
- *Hello World* — a line-by-line annotation of the hello-world program you just ran.
- `vscode-gelb/README.md` — everything the VS Code extension does, and its settings.

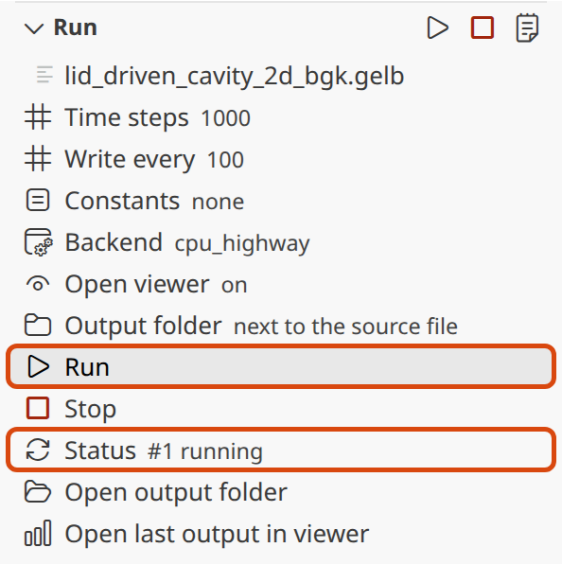


Fig. 4: The Run view while a run is going: its parameters as rows, the *Run* row and the *Status* row (outlined).

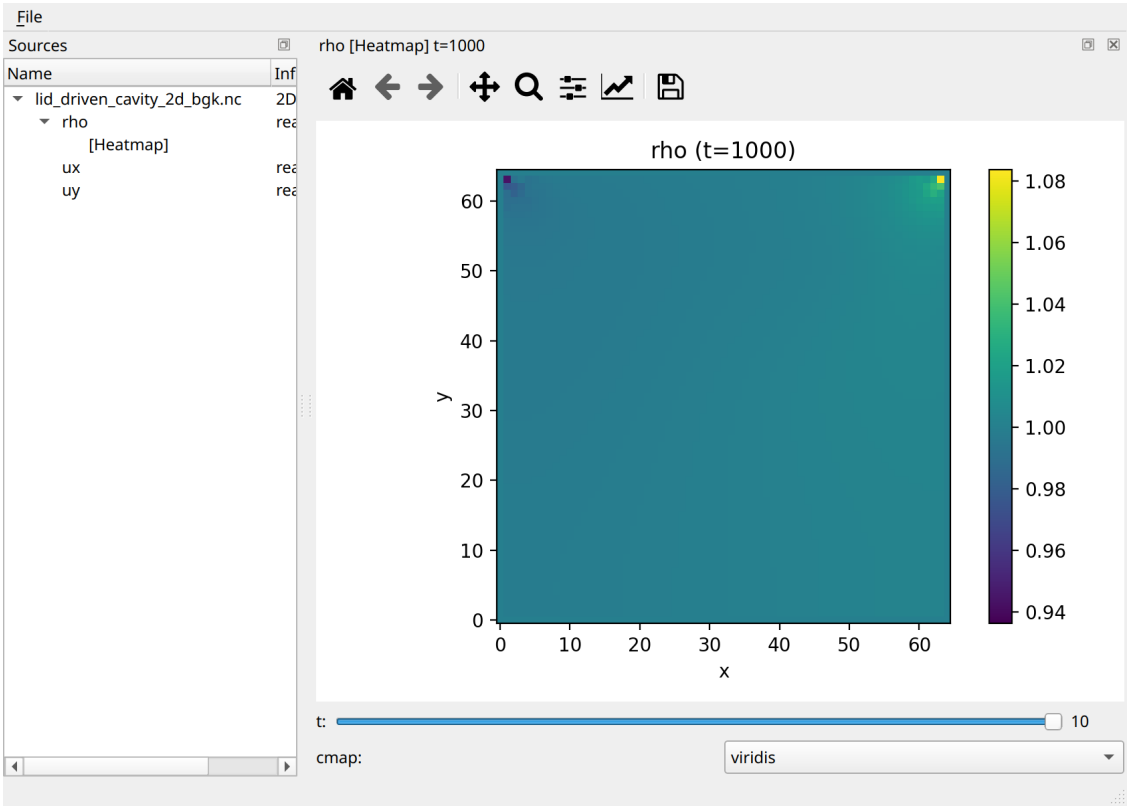


Fig. 5: `gelb-view` as the run opens it: the file's variables on the left, a heatmap of the density `rho` at the last record on the right.

CHAPTER 3

GeLB in 10 Minutes

This chapter is a language reference card. It is deliberately short: no physics, no examples, no prose — just the structural skeleton of a GeLB program, the reserved symbols, and the built-in intrinsics. Read it once straight through, then come back to it whenever you need to look something up.

> **Note**
>
> Every GeLB feature in this chapter is used in at least one worked example in Part B, so you don't need to memorise any of it — just know where to find it.

3.1 The anatomy of a GeLB program

A GeLB source file is a sequence of top-level blocks, each introduced by a keyword. There are eight block kinds; every program uses all of them except `global_constants` (optional). They must appear in roughly this order because some blocks reference declarations from earlier ones:

```
simulation_description("One-line description of the simulation");

global_constants {                // optional
    const int N = 64;
    const real tau = 0.6;
    const array[real, 9] W = {4.0/9.0, 1.0/9.0, ... };
}

lattices {
    lattice f(2, 9, real, {N, N}, {true, true});
}

function ComputeRho(array[real, 9] dfs) -> (real rho) {
    rho = sum(dfs);
}
```

(continues on next page)

(continued from previous page)

```
initializer Init { f.out[x, y | :] = W; }

dynamic Collide {
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    real rho = ComputeRho(fIn);
    f.out[x, y | :] = (1.0 - 1.0/tau) * fIn + (1.0/tau) * W * rho;
}

gauge Density -> (real rho) {
    rho = sum(f.in[x, y | :]);
}

node_classes {
    node_class BULK(Init, Collide, Density);
}

geometry {
    nodes[:, :] = BULK;
}
```

The eight block kinds, from top to bottom:

Block	Purpose
`simulation_description`	One-line string shown in tool output; required.
`global_constants`	Compile-time constants available throughout the program (optional); each can be overridden with the `-D` flag of `gelb-run`.
`lattices`	Declare one or more `lattice(d, q, dtype, size, periodicity)`.
`function`	User-defined pure, side-effect-free functions (optional).
`initializer`	Write initial values into `f.out`.
`dynamic`	Update `f.out` from `f.in` each timestep.
`gauge`	Read values from `f.in` for output.
`node_classes`	Bind (initializer, dynamic, gauge) triples into named classes.
`geometry`	Assign node classes to spatial regions of the grid.

The first line of every program is a `simulation_description(...)` call — not a block, just a one-line metadata string. It shows up in tool output and NetCDF attributes.

3.2 Lattices

A lattice declaration has five positional arguments:

```
lattice <name>(
    <dimensionality>,     // 1, 2, or 3
    <Q>,                  // number of distribution functions (e.g. 3, 5, 9,␣
↪19)
    <dtype>,              // int / real / int32 / real32 / real64 / bool
    <grid_size>,          // scalar for 1D, {Nx, Ny} for 2D, {Nx, Ny, Nz} for␣
↪3D
```

(continues on next page)

(continued from previous page)

```
    <periodicity>          // bool for 1D, {px, py} for 2D, ...
);
```

All dimensions, Q values, dtype choices, grid sizes, and periodicity flags must be compile-time constants (or arithmetic on compile-time constants). Common velocity sets:

Name	d	Q	Typical use
D1Q2	1	2	Advection
D1Q3	1	3	Diffusion, advection-diffusion
D2Q5	2	5	Diffusion, advection-diffusion
D2Q9	2	9	Incompressible Navier-Stokes
D3Q7	3	7	3D diffusion
D3Q19	3	19	Incompressible 3D Navier-Stokes
D3Q27	3	27	3D flows requiring full isotropy

GeLB itself does not encode these names; you pass the number of DFs as the `q` parameter and supply your own velocity and weight arrays as global constants (`EX`, `EY`, `EZ`, `W`).

Multi-lattice programs declare several lattices in the same block:

```
lattices {
    lattice f(2, 9, real, {N, N}, {false, false});   // fluid
    lattice g(2, 5, real, {N, N}, {false, false});   // temperature
}
```

See *Rayleigh-Bénard Convection* for a working multi-lattice example.

3.3 Reserved implicit symbols

Seven identifiers are reserved inside simulation blocks. You **cannot** use them for your own variables:

Symbol	Meaning
`x`	Current cell x-coordinate (integer, 0..Nx-1).
`y`	Current cell y-coordinate (2D/3D only).
`z`	Current cell z-coordinate (3D only).
`Nx`	Grid size in x (compile-time constant int).
`Ny`	Grid size in y (2D/3D).
`Nz`	Grid size in z (3D).
`t`	Time-step counter: `0` in the initial-condition snapshot, `1..num_timesteps` in the time loop.

These names are *only* reserved inside `initializer` / `dynamic` / `gauge` blocks. You are free to use `x`, `y`, `z`, `t` etc. as parameter names inside user-defined `function` blocks — functions see their caller's arguments, not the implicit coordinate symbols. Inside the `geometry` block, `x`, `y` and `z` are also available, but only within the condition of a `where` paint (see *the geometry paragraph* below); `t` is not, because the geometry is fixed for the whole run.

3.4 Reserved words

The block keywords (`simulation_description`, `global_constants`, `lattices`, `lattice`, `initializer`, `dynamic`, `gauge`, `function`, `node_classes`, `node_class`, `masked`, `geometry`, `nodes`, `where`, `border`), the type names, the intrinsic function names and `in` / `out` / `const` cannot be used as identifiers. Eight further words are reserved and cannot be used as identifiers: `enter`, `leave`, `reduce`, `reduction`, `update`, `global_variables`, `transition` and `zone`.

3.5 The `f.in` / `f.out` contract

Inside every `dynamic` block, GeLB guarantees two distinct logical snapshots of every lattice:

- `f.in[...]` reads from the **previous** timestep's state (the "old" data, frozen for the whole step).
- `f.out[...]` writes into the **next** timestep's state (the "new" data, not yet visible to anyone).

A read from `f.in` cannot observe a write from `f.out` made earlier in the same timestep, anywhere on the grid, by any worker, in any order. This is a language-level contract, not an implementation detail. It makes pull-streaming dynamics safe by construction, lets the compiler parallelise freely across cells, and allows future backends to drop the second buffer without breaking user code.

The `|` separator in `f.out[x, y | 3]` is phase-space notation: spatial coordinates on the left, DF index on the right.

```
f.out[x, y | :]        // all DFs at site (x, y)
f.in[x, y | 0]         // DF index 0 at site (x, y)
f.out[x, y | 1:4]      // DF slice, indices 1 to 4 (both bounds included)
f.in[x - 1, y | 3]     // DF 3 at the western neighbour
f.in[x + 1, y + 1 | :] // all DFs at the north-east neighbour
```

Each spatial index is relative to the current cell: its own coordinate plus or minus a constant integer (`x`, `x + 2`, or an expression that simplifies to that form), with the coordinates in the order `x, y, z`. The compiler rejects any other index (`2 * x`, `N - 1 - x`, `y` in the `x` position) with a message pointing at it: a stencil is a fixed displacement from the executing cell, and those expressions would make the displacement depend on the cell. Rectangle paints in the `geometry` block take constant bounds instead; coordinate-dependent painting is written as `nodes[where condition]`.

A write may also go to a neighbour: `f.out[x + 1, y | 1] = ...` executed for cell `(x, y)` sets DF 1 of the cell to its east ("push" streaming). On a periodic axis the destination wraps around; on a wall axis it must stay inside the grid, or the compiler rejects the block. A push lands in every cell the offset reaches, so it applies to same-class neighbours too, and three rules follow:

- **one writer per cell and DF.** In one time step (and, separately, in the initialization) at most one source cell may write a given DF of a given cell. Several statements of one block that hit the same cell are fine, the later wins; two *different* source cells writing the same cell would race across threads, so the compiler rejects the program and names both writers and a cell. In particular a block that pushes DF `d` must not also write DF `d` at its own cell.
- **no pushes into static or empty masked cells.** If a cell belongs to a static node class (explained below) its values are those its initializer set, and an empty masked class holds zero DFs by guarantee.
- **coverage is checked per cell.** Every DF a block reads must be written at that cell, in the right phase (see below), by the cell's own blocks or by a push landing there. In essence, this rules out ever reading uninitialised data.

The DF selection of a write must be known at compile time: a literal or constant index, a constant slice such as `0:2`, or `:`.

Prefer local writes where you have the choice. Reading from the neighbours and writing the cell's own values ("pull" streaming) is what the backends vectorize and lay out best: local stores are aligned and can stream past the cache, and a

pull program does not depend on what its neighbours are. Pushes are supported, but they force the plain storage layout and unaligned stores.

Every DF a block reads must have a defined value, both at `t = 0` (written by the node class's initializer) and after every buffer swap (written every step, by the cell's own dynamic or by a push from a neighbour; a static node class keeps its initializer's values). A DF the initializer writes but the dynamic does not is undefined after the first step, and the compiler says so. Storage of a `masked` node class starts at zero: a DF its initializer does not write is zero at `t = 0` whatever its dynamic writes later, and a DF nothing ever writes stays zero.

In `initializer` blocks, only `f.out[...]` is writable — there is no previous timestep yet. In `gauge` blocks, only `f.in[...]` is readable — gauges observe, they don't modify. Each gauge output is one scalar per cell, one field in the output file; an array-valued output (`-> (array[real, 2] u)`) is not supported, so declare one scalar per component (`-> (real ux, real uy)`).

3.6 Built-in intrinsics

Math intrinsics (scalar arguments only in this version):

```
sin, cos, tan, tanh          // trig
sqrt, exp, log               // elementary
abs, min, max, mod, atan2    // utility
```

Reductions and linear algebra:

`sum(v)`	Reduce an array to a scalar via addition.
`dot_prod(a, b)`	Dot product of two arrays. Maps to Fortran `dot_product`.
`matmul(A, B)`	Matrix product. Constant matrices are preferred by the backend.
`transpose(A)`	Matrix transpose.
`inverse(A)`	Compile-time-only — the argument must be a compile-time constant. Used for precomputed MRT collision matrices.
`determinant(A)`	Compile-time-only, like `inverse`: the determinant of a constant square matrix.

Random numbers:

`random()`	Returns a `real` in `[0, 1)`. Use `--seed N` at compile time (`gelbc --seed 42`) for reproducible runs. See *Shan-Chen Multiphase* for an example of random initial conditions.

Streaming intrinsics — not required, but they make the usual LBM dynamic block much more compact:

`upstream_dfs(f, x, y, {EX, EY})`
: Pull-streaming read. Returns an array of `Q` values, one per DF, where the `i`-th entry is `f.in[x - EX[i], y - EY[i] | i]`. Wraps around periodic dimensions. On a non-periodic dimension a read outside the grid is rejected at compile time, so a node class painted on the grid edge cannot use it. This is what makes LBM streaming a one-liner.

`bounce_back_dfs(f, x, y, {EX, EY})`
: Bounce-back wall. Returns an array of `Q` values where the `i`-th entry is `f.in[x + EX[i], y + EY[i] | opp[i]]`: the population of the neighbour in direction `i` that points back at this cell, where `opp[i]` is the DF with the opposite velocity, determined at compile time from the velocity set. The fluid neighbour pulls it on

the next step, so every population that reaches the wall goes back along its link (full-way bounce-back, the wall midway between the nodes). Allowed on the grid edge: a link pointing outside a non-periodic grid gives the cell's own `f.in[x, y | opp[i]]`, which nothing streams.

In 1D the velocity-set argument is a single 1D array: `upstream_dfs(f, x, EX)`. In 3D it is `upstream_dfs(f, x, y, z, {EX, EY, EZ})`. `bounce_back_dfs` takes the same arguments.

3.7 User-defined functions

GeLB functions are pure, side-effect-free, and cannot read or write lattices (they take their inputs as arguments): the result depends on the arguments alone, as in functional programming (Hughes, *Why Functional Programming Matters*, The Computer Journal 32, 1989, is a short, accessible introduction). They come in two flavours depending on output count:

```
// Single-output: called as an expression
function ComputeRho(array[real, 9] dfs) -> (real rho) {
    rho = sum(dfs);
}

// Later: inline in an expression
real rho = ComputeRho(fIn);

// Multi-output: called via destructuring assignment
function ComputeMacros(array[real, 9] dfs)
        -> (real rho, array[real, 2] u) {
    rho = sum(dfs);
    u = {dot_prod(dfs, EX) / rho, dot_prod(dfs, EY) / rho};
}

// Later: destructuring inside a dynamic block.  Declare the
// targets first; a destructuring target carries no type.
real rho;
array[real, 2] u;
{rho, u} = ComputeMacros(fIn);
```

Functions can call other functions, but **not recursively** (the compiler detects direct and indirect recursion and rejects it). A function is called from `initializer`, `dynamic`, `gauge` and `function` blocks; a call in `global_constants` is not in this version, and the compiler says so at the call.

3.8 Node classes and geometry

A `node_class` binds up to three blocks into a named class:

```
node_classes {
    node_class BULK(Init, Collide, Density);        // full triple
    node_class WALL(InitWall, Density);             // static — no dynamic
    node_class GHOST() masked;                      // masked — no gauge
}
```

The three positional arguments are `(initializer, dynamic, gauge)` in that order. You can omit the dynamic (a "static" node class: for a cell which belongs to it, the values are those its initializer set — useful for Dirichlet walls) or use the `masked` keyword to hide a node class from the gauge output (its cells are NaN-filled and it may not carry a gauge; an initializer and a dynamic still run if present). An empty masked class, `node_class SOLID() masked;`,

holds defined zero DFs — useful for solid obstacles you don't want in your plots. When node classes carry different gauges, the output file holds every field any of them writes; at a cell whose class does not write a field, a real field holds NaN and an integer field holds 0, as at masked cells.

Geometry paints node classes onto spatial regions via array-slice notation:

```
geometry {
    nodes[:, :] = BULK;               // default fill
    nodes[:, 0]       = BOT_WALL;     // horizontal bottom row
    nodes[:, N - 1]   = TOP_WALL;     // horizontal top row
    nodes[0, :]       = LEFT_WALL;    // vertical left column
    nodes[N - 1, :]   = RIGHT_WALL;   // vertical right column
    nodes[0, 0]       = SW_CORNER;    // single point

    // Predicate painting: every cell where the condition holds
    nodes[where InCircle(x, y, N/2, N/2, N/4)] = SOLID;

    // Auto-infer boundary layer around a solid region
    border(SOLID) = SOLID_WALL;
}
```

Paint statements come in two forms. `nodes[ranges] = CLASS;` paints a rectangle (or a single cell) given by constant, inclusive bounds: `nodes[1:3, :]` paints columns 1, 2 and 3. `nodes[where condition] = CLASS;` is a *predicate paint*: the condition is a `bool` expression in which `x`, `y` and `z` (as many as the lattice has dimensions) stand for the cell being considered; it may use constants, the scalar math intrinsics and pure single-output functions (not recursive ones), and it is evaluated once per cell at compile time, with the numeric widths the program declares: `real32(...)` rounds and `int32(...)` wraps there as they do at run time. The coordinates are available only inside a `where` condition; a rectangle paint takes constant bounds.

Ordinary and predicate paints apply in their source order, and a later paint overrides an earlier one whatever its kind (CSS-like semantics). Every cell must be painted by at least one statement or the compiler rejects the program with "Geometry does not cover node `(x, y)`".

`border(SOURCE) = TARGET;` repaints the outermost layer of a region: every cell of `SOURCE` with a neighbour of another class becomes `TARGET`. The neighbours are all the cells one step away along an axis or a diagonal, 8 in 2D and 26 in 3D, so the layer also covers the cells reached only along a diagonal link; a periodic axis wraps around its seam, and a non-periodic edge has no neighbour beyond it. The compiler applies the border rules after all paints, wherever they are written, one after the other in their source order: a rule reads one snapshot of the map and applies its replacements at once, and the next rule sees its result. This is how you get a bounce-back wall around an arbitrary obstacle without painting it by hand. See *Kármán Vortex Street* for a working example.

The geometry is fixed for the whole run: it is resolved once, at compile time, and a paint may not depend on the time step `t` or on a lattice value.

3.9 Types

GeLB has a fixed set of scalar types:

Type	Description
`int`	64-bit signed integer (alias for `int64`).
`int8`	8-bit signed integer.
`int16`	16-bit signed integer.
`int32`	32-bit signed integer.
`int64`	64-bit signed integer.
`real`	Double precision (alias for `real64`).
`real16`	Reserved keyword (16-bit half precision); rejected with a diagnostic.
`real32`	Single precision.
`real64`	Double precision.
`bool`	Boolean (Fortran `logical`, 1 byte).

`real` is double precision in every backend, whatever the lattices' type. In a program whose lattices are `real32`, a literal such as `3.0` is still `real`, so `3.0 * f` is computed in double precision and rounded when it is stored in a `real32` lattice or variable. To keep an expression in single precision, write `real32(3.0)` or use a `real32` constant (`const real32 THREE = 3.0;` in `global_constants`). The full promotion table, including what the intrinsics return, is in *Numerical Semantics*.

Array types couple element type and shape into a single type:

```
array[real, 9]            // 9-element 1D array of real
array[real, {3, 3}]       // 3x3 matrix
array[int, 4]             // 4-element int vector
```

Each type name doubles as an explicit cast intrinsic: `int(3.7)` yields `3`, `real(n)` promotes an int to real, `int8(255)` truncates to an 8-bit representation.

3.10 Expressions and control flow

GeLB expressions support the usual arithmetic (`+ - * /`, and `^` for powers), comparison (`< <= == != >= >`), and logical (`&& || !`) operators, with C-style precedence. `^` is right-associative and binds tighter than unary minus, so `-x^2` is `-(x^2)` and `2^3^2` is `2^9`. Array arithmetic is element-wise by default; `matmul` and `dot_prod` are the explicit tensor operations.

What GeLB does not have:

- `if` / `else` statements. Use node classes and geometry painting to branch on spatial position. For a choice that depends on data, write arithmetic that encodes it: `max(0.0, rho - rho_min)` clamps a value, `min(1.0, real(t) / T_RAMP)` ramps a forcing up and then holds it, and multiplying a term by `real(v < 4.0)` switches it on (`1.0`) or off (`0.0`) depending on the comparison (write the comparison inline, as here).
- `while` / `for` loops. Array operations and the implicit per-cell loop around every `initializer` / `dynamic` / `gauge` block cover most situations where these would be needed (the Hughes paper cited above shows the general pattern).
- `print` / `write`. All output goes through `gauge` blocks and lands in NetCDF.

These are deliberate restrictions: they guarantee that every GeLB program is easily parallelisable across cells, since there are no data-dependent control-flow branches inside the per-cell kernel.

3.11 Command-line constants

Any `const` in `global_constants` can be overridden at compile time with `-D NAME=VALUE`:

```
gelbc program.gelb -b cpu_fortran_serial -D N=128 -D tau=0.75 -o program.f90
gelb-run -D N=256 -n 1000 program.gelb
```

This is how you do parameter sweeps without editing the source.

3.12 Two-minute "what just happened"

Every GeLB program boils down to three things:

1. A **lattice** — where data lives. Declared once in `lattices`, with a fixed shape, dimensionality, and DF count.
2. A **per-cell kernel** — how data changes. Expressed as one `dynamic` block per node class. The compiler wraps it in an implicit loop over every cell in that node class and guarantees the `f.in` / `f.out` ping-pong contract.
3. A **geometry** — which cells run which kernel. Node classes bind `(initializer, dynamic, gauge)` triples, and `geometry { ... }` paints them onto the grid with CSS-like override semantics.

Everything else — functions, intrinsics, global constants, multi- lattice coupling — is syntactic sugar around those three ideas.

When you are stuck, re-read this page. When you are really stuck, read *Hello World* — it walks through every construct in this chapter in the context of a concrete 30-line program.

Part B

A guided tour

CHAPTER 4

Hello World

This chapter walks line-by-line through the smallest meaningful program in the GeLB repository: `examples/basic/1d/ex0_int_vec_setto_one.gelb`. It does not do any physics — it is a "hello world" program that declares a 1D integer lattice, sets every cell to 0, then overwrites every cell with 1 at every time step. By the end of this chapter you will know what every block in a GeLB program is for, how the compiler turns it into a per-cell loop, and what the NetCDF output file actually contains.

> **Note**
>
> If *GeLB in 10 Minutes* felt abstract, this chapter is the concrete counterpart. Every construct from the cheat-sheet is used here exactly once, in its simplest possible form.

4.1 The program

The entire program is 30 lines. Read it top to bottom:

```
simulation_description("Pretty much the hello world of GeLB");

lattices {
  lattice f(1,        // d         — one spatial dimension
            1,        // q         — one distribution function per cell
            int,      // df_type   — 64-bit integer
            32,       // grid_size — 32 cells
            false     // periodic  — no (x=0 and x=31 are distinct walls)
  );
}

initializer SetDFsToZero {
  f.out[x | 0] = 0;
}
```

(continues on next page)

(continued from previous page)

```
dynamic SetDFsToOne {
  f.out[x | 0] = 1;
}

gauge CurrentState -> (int state) {
  state = f.in[x | 0];
}

node_classes {
  node_class BULK(SetDFsToZero, SetDFsToOne, CurrentState);
}

geometry {
  nodes[:] = BULK;
}
```

Let's walk through it.

4.1.1 `simulation_description` — one-line metadata

```
simulation_description("Pretty much the hello world of GeLB");
```

Every GeLB program begins with this one-line call. The string is stored as a NetCDF global attribute; `gelbc` prints it after parsing, as `Simulation: "..."`, and `gelb-view` shows it in the *Info* column of its file tree. Its only purpose is human orientation — strip it out and the program still compiles and runs identically, but future-you will thank present-you for writing a one-sentence summary.

4.1.2 `lattices` — where the data lives

```
lattices {
  lattice f(1, 1, int, 32, false);
}
```

This block declares a single lattice called `f`. Its five arguments, in order:

- **Dimensionality** `1` — a one-dimensional lattice (a line of cells).
- **Q** `1` — one distribution function per cell. Real LBM programs use `q=3` for D1Q3 diffusion, `q=9` for D2Q9 flow, `q=19` for D3Q19. Here we use `q=1` because we are not doing LBM yet; we just want one integer per cell.
- **DF type** `int` — 64-bit signed integers. Most real LBM programs use `real` (= `real64`, double precision); we use `int` here because integers make space-time plots trivially readable.
- **Grid size** `32` — 32 cells along the single spatial dimension.
- **Periodicity** `false` — the two endpoints `x = 0` and `x = 31` are distinct non-periodic boundaries. Since the program never references neighbours (`f.in[x-1, ...]` or `f.in[x+1, ...]`), this choice is irrelevant here — but it would matter the moment we introduced streaming.

After this declaration, `f` behaves like a 1D array of 32 integers that the rest of the program can read and write at each time step.

4.1.3 `initializer` — what the lattice starts as

```
initializer SetDFsToZero {
  f.out[x | 0] = 0;
}
```

An `initializer` block describes how the lattice is populated *before* the main time loop starts. Inside the block, every reference to `x` is implicitly bound to the current cell's coordinate (`0`, `1`, …, `31`), and the compiler wraps the block body in an implicit loop over every cell in the node class. You never write the loop yourself.

The left-hand side `f.out[x | 0]` reads as "DF index 0 at site `x`, write channel". The `|` separator is phase-space notation (the creator of GeLB is a physicist, and it looks nice) — spatial coordinates on the left, DF index on the right — so `f.out[x | 0]` means "write to the 0th distribution function at cell `x`". Because the lattice has only one DF, `| 0` is the only valid choice.

The right-hand side is a plain integer literal `0`. After the implicit loop runs, every cell of `f` holds `0`.

In an `initializer` block only `f.out` is writable. There is no "previous" time step yet, so `f.in` is not in scope.

4.1.4 `dynamic` — how the lattice evolves

```
dynamic SetDFsToOne {
  f.out[x | 0] = 1;
}
```

A `dynamic` block describes one time-step update. It has access to both `f.in` (the previous time step's state, read-only) and `f.out` (the next time step's state, write-only). Here the update is trivial: unconditionally set every cell to 1.

The "previous / next" split is a *language-level contract* (see *GeLB in 10 Minutes* — the `f.in` / `f.out` section). Inside any single dynamic block, reads from `f.in` cannot observe writes from `f.out` made earlier in the same timestep, anywhere on the grid, by any worker. The compiler guarantees this illusion, even across OpenMP threads and CUDA kernels, by maintaining a ping-pong double buffer under the hood. In this particular program the contract is not exercised (we never read `f.in` at all), but understanding it matters as soon as you write your first real LBM kernel — see *1D Diffusion*.

4.1.5 `gauge` — what the program outputs

```
gauge CurrentState -> (int state) {
  state = f.in[x | 0];
}
```

A `gauge` block is how GeLB does I/O — by *declaring* the output variables it writes and filling them once per cell per time step. GeLB does not have `print`, `write`, or any other statement with observable side effects; every number that ever leaves a running simulation passes through a gauge block.

This gauge declares one output variable, `int state`, and fills it with the single DF at each cell. The compiler's NetCDF writer collects all the gauge outputs across all cells and time steps into a NetCDF file with dimensions `(time, x)` and a variable named `state` of type `int64`. (In 2D programs the variable becomes `state(time, y, x)`; in 3D, `state(time, z, y, x)`.)

Gauges run once before the main time loop, capturing the initial condition as the `t=0` record, and then once at the end of every time step. For a run of 10 time steps the NetCDF file therefore has 11 records.

4.1.6 `node_classes` — binding the three blocks together

```
node_classes {
  node_class BULK(SetDFsToZero, SetDFsToOne, CurrentState);
}
```

A node class bundles exactly three blocks — an `initializer`, a `dynamic`, and a `gauge` — into one named unit. The order matters: `(initializer, dynamic, gauge)`. Here we name the class `BULK` and bind our three blocks to it.

Real GeLB programs typically declare several node classes to distinguish interior cells from wall cells, inflow/outflow boundaries, solid obstacles, and so on. Each class can have its own dynamic — that is the GeLB idiom for "different physics at different points of the geometry", replacing the `if (cell_type == WALL) { ... } else { ... }` branches you would write in a hand-coded LBM solver.

4.1.7 `geometry` — which cells run which node class

```
geometry {
  nodes[:] = BULK;
}
```

Finally, `geometry` paints node classes onto the grid. The slice `nodes[:]` means "every cell", so every one of our 32 cells is painted as BULK. Every cell must be painted by at least one statement, or the compiler rejects the program:

```
[Problem] Geometry does not cover node (5)
```

Paint statements apply in source order, top to bottom, with CSS-like override semantics — a later statement can overwrite cells painted by an earlier one. This is how you write "everything is BULK except the top and bottom rows, which are WALL":

```
nodes[:, :]     = BULK;
nodes[:, 0]     = BOT_WALL;     // overrides row 0
nodes[:, N - 1] = TOP_WALL;     // overrides row N-1
```

(You'll see exactly that pattern in *1D Diffusion* and every chapter after.)

4.2 Running it

From the repository root:

```
gelb-run -n 10 \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* to 10 and *Write every* to 1 in the Run view and press **Run**: it runs this command with `--view` added, so the viewer opens on the result, and the `.nc` file lands next to the source file. The commands shown here are what stands behind it.

The compiler prints a warning that the dynamic block writes `f.out` but never reads `f.in`, with a hint about Dirichlet-style boundary conditions. That is expected for this program — the dynamic really does ignore the previous state — and you can ignore it here.

That produces an `ex0_int_vec_setto_one.nc` file in the current directory containing 11 records (one at `t=0`, then one per time step). Open it in the viewer:

```
gelb-view data ex0_int_vec_setto_one.nc
```

The viewer opens with a space-time heatmap of `state`; right-click a variable in the file tree for other plot types. You can also render a headless snapshot to a PNG (the same heatmap):

```
gelb-view data ex0_int_vec_setto_one.nc \
    -o hello.png --no-gui
```

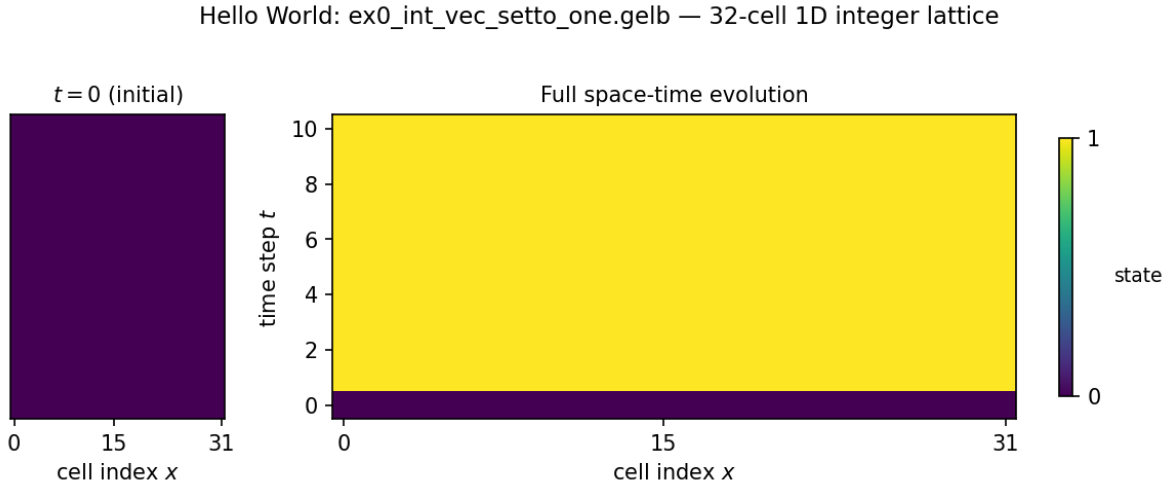


Fig. 1: The integer lattice over time. **Left:** the `t=0` pre-loop snapshot captured by the gauge block — all cells are `0` because `SetDFsToZero` has run but `SetDFsToOne` has not. **Right:** the full space-time evolution over 10 time steps. From `t=1` onwards every cell is `1`, because the dynamic block overwrites every cell unconditionally. The two-colour transition at `t=0 → t=1` is the visual signature of the initial-condition snapshot that every GeLB gauge block produces.

The figure reads as follows: the horizontal axis is space (cell index `x` from 0 to 31), the vertical axis is time (`t` from 0 to 10), and the colour is the value of `state`. Row 0 is the initial condition; row 1 is the first post-dynamic snapshot; row 10 is the final state.

4.3 What the compiler actually emitted

Pass `-k` to `gelb-run` and it will tell you where the generated code was saved:

```
gelb-run -k -n 10 \
    examples/basic/1d/ex0_int_vec_setto_one.gelb
```

You don't need to read the generated code — GeLB's whole point is that you shouldn't have to — but opening it once is worth it. The thing you wrote is 30 lines of GeLB; the thing GeLB generated, with the default `cpu_highway` backend, is 122 lines of C++ in one `main` function. You'll see the grid constants and the lattice storage (two copies of the lattice, for the previous and the next time step), the NetCDF setup, the initializer loop, a gauge pass before the time loop, a `for` loop over the 10 time steps containing the dynamic body and the gauge writes, and a timing report at the end. All of that is boilerplate the compiler writes for you.

You can also look at what the compiler thinks the program's read/write pattern is:

```
gelbc examples/basic/1d/ex0_int_vec_setto_one.gelb \
    --stencils
```

That prints a stencil report for every dynamic block — which DFs it reads, which it writes, and the spatial offsets. For this program the report is trivial ("writes DF 0 at offset (0)"), but for real LBM programs it is how you verify that your collision and streaming steps are touching the right cells. See *Source Visualisation: gelb-view source* for the full story.

4.4 What's next

The hello world program is as simple as a GeLB program gets. The next chapter, *1D Diffusion*, does the minimum to turn this into real LBM: a D1Q3 lattice (so we have three DFs, one per velocity direction), a BGK collision step, actual streaming, and Dirichlet walls at both ends. It is still a 1D program, still in under a hundred lines, but it produces a meaningful physics result you can compare against the analytical steady-state solution.

See also

Krüger et al. (2017) §3.2 (*The Lattice Boltzmann Equation in a Nutshell*) for the method in two pages, and §3.3.1–3.3.2 (*Initialisation*, *Time Step Algorithm*) for the sequence a GeLB program declares as lattice, initializer, dynamic and gauge.

CHAPTER 5

1D Diffusion

This chapter turns the hello-world program into a real LBM simulation. We solve 1D heat diffusion on a D1Q3 lattice with fixed-temperature walls at both ends, and watch the temperature profile relax from a hot initial condition toward the analytical linear steady state. Everything the chapter uses is already in *GeLB in 10 Minutes*; this is the first time you see it deployed on a real problem.

> **Note**
>
> The example walked here is `examples/lbm/1d/heat_diffusion_1d_dirichlet.gelb`. Open it in another pane as you read.

5.1 The physics

Picture a thin metal bar in vacuum, its left end held in a hot reservoir at temperature T_{left} and its right end in a cold one at T_{right}. Nothing enters or leaves through the sides, so heat flows only along the bar. Whatever the bar's temperature is at the start, it ends up varying linearly from the hot end to the cold end; how fast it gets there is set by the material's diffusivity α.

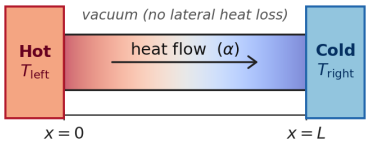


Fig. 1: A bar of length L in vacuum between two constant-temperature reservoirs T_{left} and T_{right}.

We solve the 1D heat equation

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}$$

on the interval $x \in [0, L]$ with Dirichlet boundary conditions $T(0) = T_{\text{left}}$ and $T(L) = T_{\text{right}}$. The analytical steady state is linear,

$$T_\infty(x) = T_{\text{left}} + (T_{\text{right}} - T_{\text{left}}) \cdot x/L \,,$$

and any initial condition relaxes toward it at a rate set by the diffusivity α. The transient is a superposition of sinusoidal eigenmodes $\sin(k\pi x/L)$ each decaying as $\exp(-\alpha k^2 \pi^2 t/L^2)$.

LBM on D1Q3 represents this with three distribution functions per cell — one for each discrete velocity in the set $c = \{0, +1, -1\}$ — plus corresponding weights $w = \{4/6, 1/6, 1/6\}$. The equilibrium distribution for a passive scalar is $f_i^{eq} = w_i T$, and the BGK collision operator relaxes the DFs toward equilibrium:

$$f_i^*(x,t) = (1-\omega) f_i(x,t) + \omega\, f_i^{eq}(T(x,t)) \,.$$

After collision, the populations stream to their neighbours — in our "pull" formulation, every cell reads its upstream DF from the previous time step:

$$f_i(x, t+1) = f_i^*(x - c_i, t) \,.$$

The effective diffusivity is $\alpha = c_s^2(\tau - 1/2)$ with $c_s^2 = 1/3$ for D1Q3 and $\tau = 1/\omega$. For $\tau = 0.75$ we get $\alpha = (0.75 - 0.5)/3 = 1/12 \approx 0.0833$ in lattice units. See Mohamad (2019) §5.3 and §5.5 or Krüger et al. (2017) §8.1 and §8.3.4 for the Chapman–Enskog derivation of α.

5.2 The program

```
simulation_description("1D heat diffusion with Dirichlet BCs (D1Q3)");

global_constants {
    const int GRID = 32;
    const array[real, 3] W  = {4.0/6.0, 1.0/6.0, 1.0/6.0};
    const array[real, 3] EX = {0.0, 1.0, -1.0};
    const real tau   = 0.75;
    const real omega = 1.0 / tau;
    const real T_left  = 1.0;
    const real T_right = 0.0;
}

lattices {
    lattice f(1, 3, real, GRID, false);
}

function DiffusionEq(real T) -> (array[real, 3] feq) {
    feq = W * T;
}

initializer InitBulk  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitLeft  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitRight { f.out[x | :] = DiffusionEq(T_right); }

dynamic CollideAndStream {
    array[real, 3] fIn = upstream_dfs(f, x, EX);
    real T = sum(fIn);
    array[real, 3] feq = DiffusionEq(T);
```

(continues on next page)

(continued from previous page)

```
    f.out[x | :] = (1.0 - omega) * fIn + omega * feq;
}

gauge Temperature -> (real temperature) {
    temperature = sum(f.in[x | :]);
}

node_classes {
    node_class BULK(InitBulk, CollideAndStream, Temperature);
    node_class LEFT_WALL(InitLeft,  Temperature);
    node_class RIGHT_WALL(InitRight, Temperature);
}

geometry {
    nodes[:]        = BULK;
    nodes[0]        = LEFT_WALL;
    nodes[GRID - 1] = RIGHT_WALL;
}
```

Forty-six lines of GeLB expressing the complete physics. If you notice a little repetition, you are right: `InitBulk` and `InitLeft` have the same effect, and two initializers would do; the example keeps one per node class for clarity. Let's walk through the new constructs.

5.2.1 Global constants and the velocity set

```
global_constants {
    const int GRID = 32;
    const array[real, 3] W  = {4.0/6.0, 1.0/6.0, 1.0/6.0};
    const array[real, 3] EX = {0.0, 1.0, -1.0};
    const real tau   = 0.75;
    const real omega = 1.0 / tau;
    const real T_left  = 1.0;
    const real T_right = 0.0;
}
```

The `global_constants` block is how you declare compile-time constants — grid size, relaxation rates, wall temperatures, and in particular *the D1Q3 velocity set itself*. GeLB does not have a built-in "D1Q3" keyword; you provide the weights `W` and the velocity components `EX` as two global arrays. This is a deliberate choice: instead of baking velocity sets into the language, GeLB lets you write BGK (D1Q3, D1Q5, D2Q9, D2Q17, …) in exactly the same way, and also lets you use non-standard sets for research purposes. We're using the standard D1Q3 numbering here — DF 0 is the rest particle, DF 1 moves in the `+x` direction, DF 2 moves in the `-x` direction — but you'd write the code identically for any other choice.

Notice the types: `W` and `EX` are `array[real, 3]` — 3-element 1D arrays of `real` (= `real64`). Array types couple element type and shape into a single type; see *GeLB in 10 Minutes*.

The relaxation rate `omega = 1.0 / tau` is a compile-time constant computed by the compiler (`tau` is a literal, `/` is a constant-foldable operator, the result is a scalar literal). You can use `omega` anywhere you'd use a literal.

5.2.2 User-defined functions

```
function DiffusionEq(real T) -> (array[real, 3] feq) {
    feq = W * T;
}
```

This is our first user-defined function. The `function` block starts with a comma-separated parameter list, then `->` introduces the parenthesized list of named results. Here `T` is the input and `feq` is the single, array-type result. The body computes the result by assigning to its declared name.

`DiffusionEq` computes the equilibrium distribution of a D1Q3 passive scalar from the scalar value T. The expression `W * T` multiplies the 3-element array `W` by the scalar `T`, returning a 3-element array (array operations are element-wise by default). The compiler inlines this into the call site, so there is no runtime function call in the generated code.

Functions in GeLB are **pure** — they have no side effects, cannot read or write lattices, and cannot recurse (the compiler rejects recursion at compile time). The one exception to purity is `random()`, which a function may call; runs stay reproducible with a fixed `--seed` for the same backend and execution configuration, thread count included (see *Numerical Semantics*).

Because `DiffusionEq` has a single output, it is called as an expression: `DiffusionEq(T_left)` in the initializer, or `DiffusionEq(T)` later in the dynamic block. Multi-output functions (which we'll see in *Lid-Driven Cavity*) are called via destructuring assignment (similar to Python's tuple unpacking): `{rho, u} = ComputeMacros(fIn);`.

5.2.3 Three initializers, three node classes

```
initializer InitBulk  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitLeft  { f.out[x | :] = DiffusionEq(T_left);  }
initializer InitRight { f.out[x | :] = DiffusionEq(T_right); }

node_classes {
    node_class BULK(InitBulk, CollideAndStream, Temperature);
    node_class LEFT_WALL(InitLeft,  Temperature);
    node_class RIGHT_WALL(InitRight, Temperature);
}
```

Three initializers, each writing the *full DF array* at a cell (`f.out[x | :]` — the `:` slice over the DF dimension). The right-hand side is a 3-element array literal from `DiffusionEq(...)`, and because GeLB checks that the shapes agree, assigning it to `f.out[x | :]` writes all three DFs at once.

Notice what's new compared to *Hello World*:

1. **Three node classes, not one.** `BULK` is the interior, `LEFT_WALL` and `RIGHT_WALL` are the two endpoints.
2. **Two of them have no dynamic block.** `LEFT_WALL` and `RIGHT_WALL` list only two blocks, `(InitLeft, Temperature)` and `(InitRight, Temperature)` — that's GeLB's idiom for a **static node class**. For a cell which belongs to a static class, the values are those its initializer set: the DFs are written once and keep that value for the entire run. This is precisely what a fixed-temperature wall needs: the equilibrium distribution for the boundary temperature is set at `t = 0` and stays put forever, so the interior cells keep streaming "from" a fixed-temperature source every timestep.

   The two-block tuple `(initializer, gauge)` is one of GeLB's boundary-condition patterns, explained in more detail in *Boundary Conditions Cookbook*. You can also omit the gauge if you don't want wall cells to appear in the output, or use the `masked` keyword to NaN-fill them.
3. **Every node class shares the same gauge.** `Temperature` is listed in all three class tuples, so every cell — interior, left wall, right wall — writes a `temperature` value to the NetCDF output every timestep. This is how the figure below can show a smooth profile that includes the wall values.

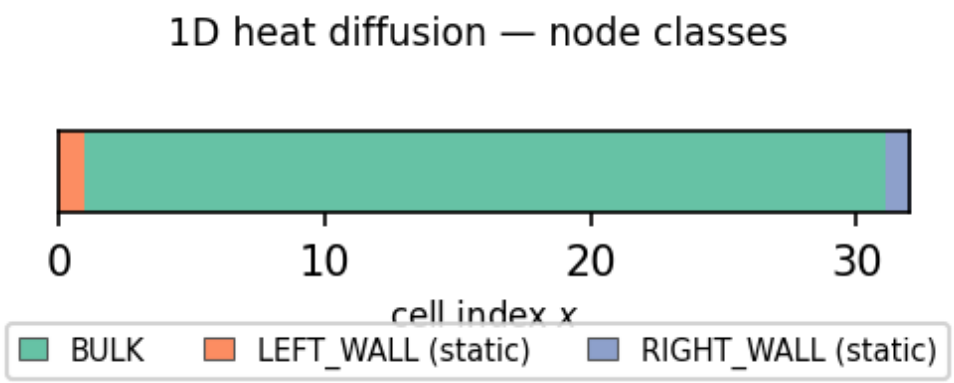


Fig. 2: Node-class layout: BULK (teal) fills the interior, LEFT_WALL (orange, cell 0) and RIGHT_WALL (blue, cell 31) are static boundary cells with no dynamic block.

5.2.4 The dynamic block: BGK collision + pull streaming

```
dynamic CollideAndStream {
    array[real, 3] fIn = upstream_dfs(f, x, EX);
    real T = sum(fIn);
    array[real, 3] feq = DiffusionEq(T);
    f.out[x | :] = (1.0 - omega) * fIn + omega * feq;
}
```

Four lines of physics. Line by line:

Line 1: `upstream_dfs(f, x, EX)` is the streaming intrinsic. It returns a 3-element array where the `i`-th entry is the DF `i` at the upstream cell — that is, the cell you would reach by taking one step *against* the direction of motion encoded in `EX[i]`. For D1Q3:

- DF 0 (rest particle, `EX[0] = 0`) → reads from `f.in[x | 0]` (no offset)
- DF 1 (+x, `EX[1] = +1`) → reads from `f.in[x-1 | 1]`
- DF 2 (-x, `EX[2] = -1`) → reads from `f.in[x+1 | 2]`

The compiler computes these offsets from the `EX` array at compile time and generates the corresponding index expressions — you don't have to write `x - 1` by hand for this common LBM operation.

On a non-periodic grid, GeLB checks that no block reads outside it. `CollideAndStream` runs only on the bulk cells 1 to 30, so its reads at `x - 1` and `x + 1` stay inside the grid. The wall classes are static: their initializers set their DFs, and no collision or streaming code runs there. If you assigned this dynamic to `LEFT_WALL` as well, the compiler would reject the program as an out-of-bounds access, naming the block, the node class and the offending offset, and suggesting a periodic axis or a different block for the boundary cells.

Line 2: `real T = sum(fIn);`. The macroscopic temperature is the sum of the distribution functions (the zeroth moment of the DF population). `sum` is a built-in reduction intrinsic that reduces an array to a scalar.

Line 3: `array[real, 3] feq = DiffusionEq(T);`. Compute the equilibrium distribution at the local temperature, via the user- defined function from earlier. The compiler inlines this so there is no function-call overhead in the inner loop.

Line 4: the BGK collision step proper:

```
f.out[x | :] = (1.0 - omega) * fIn + omega * feq;
```

Element-wise array arithmetic: multiply the 3-element arrays `fIn` and `feq` by scalars, add, assign to the 3-element DF slot `f.out[x | :]`. The compiler checks the shapes: an array must fit the selected DF slice, and an array cannot be assigned to a single scalar DF slot. A scalar on the right broadcasts across the slice, so `f.out[x | :] = 3.0` sets all three DFs to 3.0.

5.2.5 The gauge

```
gauge Temperature -> (real temperature) {
    temperature = sum(f.in[x | :]);
}
```

One scalar output per cell per timestep: the temperature, computed as the sum of the three DFs. Same pattern as the gauge in *Hello World*, but now the underlying computation is physically meaningful. Note that the gauge reads `f.in`, not `f.out`: gauges observe the state the just-completed dynamic produced, which the `f.in` / `f.out` contract of *GeLB in 10 Minutes* presents as the new `f.in`.

5.3 Running it

From the repository root, or in VS Code with *Time steps* 10000 and *Write every* 100 in the Run view (*Working in VS Code*):

```
gelb-run -n 10000 -w 100 \
    examples/lbm/1d/heat_diffusion_1d_dirichlet.gelb
```

Ten thousand timesteps writing a gauge snapshot every 100 steps gives you 101 NetCDF records. That's enough to watch the profile relax from the uniform-hot initial condition down to within 10^{-4} of its steady state.

That steady state is a straight line, but not the one through the two wall cells: a static wall cell holding the equilibrium for T_{wall} imposes that temperature $1-\tau$ cells from its own centre, towards the bulk (a quarter of a cell here, and exactly at the wall cell only for $\tau = 1$). The bulk cells therefore settle on the line from $T = 1$ at $x = 0.25$ to $T = 0$ at $x = N - 1.25 = 30.75$, a length $L = 30.5$, and that is the dashed line in the figure.

A proper way to remove this offset is the anti-bounce-back scheme (Krüger et al. (2017) §8.5.2.1), shown in `examples/lbm/1d/heat_diffusion_1d_antibounceback.gelb`: each end cell takes back the DF it sent towards the wall with its sign flipped, plus twice the equilibrium at the wall temperature, which puts the wall half a cell beyond the end cell for any τ. Its steady profile is the straight line between the two walls to round-off.

The repository also ships `examples/lbm/1d/heat_diffusion_1d_periodic.gelb`: the same kernel on a ring (periodic boundaries, no walls), where one cell heated at the start spreads its heat out until the whole ring is uniformly warm.

5.4 The compiler catches a common mistake

Suppose you write only the first DF in the initializer, thinking "this is fine, I'll set DF 1 and DF 2 in the dynamic block on the first step":

```
initializer InitBulk {
    array[real, 3] feq = DiffusionEq(T_left);
    f.out[x | 0] = feq[0];   // only DF 0!
}
```

Run `gelbc` on that program and it refuses to compile:

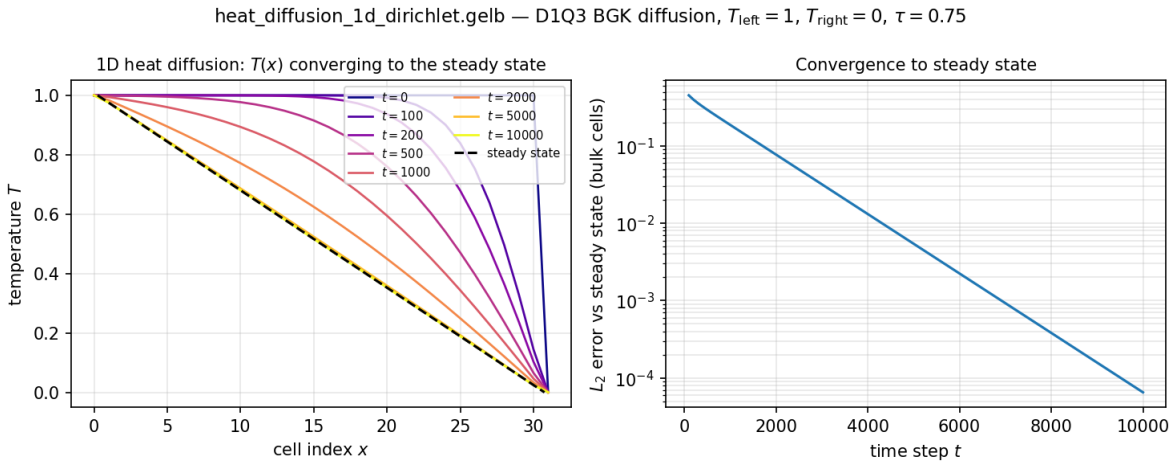


Fig. 3: **Left:** the temperature profile $T(x)$ at eight time slices from `t=0` (hot everywhere, the initial condition) to `t=10000` (indistinguishable from the steady state). The dashed black line is that steady state, $T(x) = 1-(x-0.25)/L$, drawn between the two points where the walls impose their temperatures. Early profiles are curved because the hot interior hasn't yet "seen" the cold right wall; as time progresses the curvature relaxes out and the profile straightens. **Right:** the L_2 error of the bulk cells against the steady state, plotted on a log scale. The exponential approach is the sum of the diffusive eigenmode decays — at large `t` the slowest mode ($k = 1$) dominates and the decay rate tends to $\alpha\pi^2/L^2 \approx 10^{-3}$ per step.

```
[Problem] Initializer 'InitBulk' (for node class 'BULK') does
not write all DFs of lattice 'f': missing DFs [1, 2]

[Hint] Add write statements for the missing distribution
functions, or use the full-range syntax f.out[...|:] to write
all DFs at once.
```

This is the **initializer-coverage check** — one of the LBM- specific compile-time validations GeLB performs by running a stencil analysis pass over every initializer block and checking that every DF of every lattice that participates in the subsequent dynamic is written at least once. If you forget one, you get the error above with the exact lattice name and the exact set of missing DF indices, *before* the first line of code is generated and *before* any simulation has ever run.

The same check catches partial array writes (`f.out[x | 0:2] = ...` when `q = 3`), out-of-range DF indices (`f.out[x | 5] = ...` when `q = 3`), and write statements that only run for some node classes — all the variations on "oops, I didn't actually initialise that DF" that are easy to make in a hand-written LBM code and notoriously hard to spot when the result is just a slightly wrong answer.

5.5 Peeking at the stencil

Assuming your program is correct (undo any changes if you followed the previous error-message showcase), you can ask the compiler to print its stencil analysis:

```
gelbc \
    examples/lbm/1d/heat_diffusion_1d_dirichlet.gelb \
    --stencils
```

That prints, for every initializer, dynamic and gauge block, the per-DF read and write offsets. For `CollideAndStream` you'll see three reads at offsets `{0, -1, +1}` and three writes at offset `0` — exactly what the D1Q3

streaming-in-plus-collision step does. This same stencil information drives the boundary checks, the geometry coverage check, and the visualization in *Source Visualisation: gelb-view source*.

5.6 What's next

The next chapter, *Lid-Driven Cavity*, moves to two dimensions and to fluid flow: the D2Q9 lattice, multi-output functions and bounce-back walls. Everything you learned in this chapter carries forward unchanged.

See also

Mohamad (2019) chapter 5 (§5.1–5.5): a self-contained treatment of pure 1D diffusion with the lattice Boltzmann method, with a Matlab implementation and the Dirichlet boundary condition used here.

Krüger et al. (2017) §3.2 (*The Lattice Boltzmann Equation in a Nutshell*) for the BGK formula; §8.1 (*Lattice Boltzmann Advection-Diffusion in a Nutshell*) and §8.3 (*Lattice Boltzmann for Advection-Diffusion*; §8.3.2 for the scalar equilibrium $f_i^{eq} = w_i T$, §8.3.4 for the Chapman–Enskog argument that the lattice method solves the heat equation).

CHAPTER 6

Lid-Driven Cavity

The lid-driven cavity is a canonical benchmark in computational fluid dynamics. Ghia, Ghia & Shin (1982) published reference centreline velocity profiles across a wide range of Reynolds numbers, and textbooks such as Krüger et al. (2017) and Mohamad (2019) use the square cavity to validate incompressible flow solvers. This chapter runs the setup of Mohamad (2019) §8.8.1 in GeLB.

> **Note**
>
> The example walked here is `examples/lbm/2d/lid_driven_cavity_2d_bgk_re1000.gelb`, which uses the parameters of Mohamad (2019) §8.8.1 (pp. 124–125, Figs. 8.10 and 8.11). It is the first 2D fluid-flow example in the tutorial.

6.1 The physics

Consider a square cavity of side length L filled with a viscous fluid. Three of its walls are stationary (no-slip), while the top wall (the "lid") slides tangentially in the $+x$ direction at a constant speed U_{lid}. Shear at the lid drags the adjacent fluid toward the upper-right corner, where the stationary right wall deflects the stream downward and drives a recirculating vortex across the cavity.

The governing macroscopic equations are the incompressible Navier–Stokes equations,

$$\nabla \cdot \mathbf{u} = 0\,, \qquad \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u}\,,$$

which we solve indirectly in LBM by time-stepping the lattice Boltzmann equation on a D2Q9 lattice and extracting ρ and $\mathbf{u}$ as moments of the distribution function at every step. The flow regime is governed by the Reynolds number

$$\mathrm{Re} = \frac{U_{\text{lid}} L}{\nu}\,.$$

Mohamad (2019) §8.8.1 uses $\mathrm{Re} = 1000$, which is high enough to produce the characteristic three-vortex flow (one primary vortex shifted toward the upper-right by advection, plus two counter-rotating secondary vortices in the lower corners) while remaining laminar and steady.

6.2 D2Q9 and the macroscopic moments

The 1D examples in the preceding chapters used one or three distribution functions per cell. For 2D isothermal fluid flow we move to **D2Q9** — nine populations per cell, consisting of one rest particle ($i = 0$), four cardinal neighbours ($i = 1, \dots, 4$), and four diagonal neighbours ($i = 5, \dots, 8$) arranged in a Moore stencil:

$$\mathbf{c}_i \in \{(0,0), (\pm 1, 0), (0, \pm 1), (\pm 1, \pm 1)\}\,.$$

The macroscopic density and velocity are the zeroth and first moments of the DF population:

$$\rho(\mathbf{x}, t) = \sum_{i=0}^{8} f_i(\mathbf{x}, t)\,, \qquad \rho(\mathbf{x}, t)\,\mathbf{u}(\mathbf{x}, t) = \sum_{i=0}^{8} \mathbf{c}_i\, f_i(\mathbf{x}, t)\,,$$

and the second-order truncated Maxwell–Boltzmann equilibrium distribution (with lattice sound speed $c_s^2 = 1/3$) is

$$f_i^{\mathrm{eq}} = w_i \rho \left(1 + 3\mathbf{c}_i \cdot \mathbf{u} + \tfrac{9}{2}(\mathbf{c}_i \cdot \mathbf{u})^2 - \tfrac{3}{2}|\mathbf{u}|^2\right)$$

with D2Q9 weights $w = \{4/9, 1/9 \times 4, 1/36 \times 4\}$ (Krüger et al. (2017) §3.4.7, Table 3.1).

These two formulas — macroscopic moments and equilibrium — form the bulk core of an isothermal incompressible flow solver in GeLB; the remaining sections of the file specify boundary conditions and geometry.

6.3 The program

Here is the core of `examples/lbm/2d/lid_driven_cavity_2d_bgk_re1000.gelb`, followed by a walk-through of the three new GeLB constructs it introduces:

```
global_constants {
    const int N = 100;
    const real U_LID = 0.1;
    const real NU = 0.01;
    const real TAU = 3.0 * NU + 0.5;      // = 0.53
    const real TAU_INV = 1.0 / TAU;

    const array[real, 9] EX = {0.0, 1.0, 0.0, -1.0,  0.0, 1.0, -1.0, -1.0, ␣
↪1.0};
    const array[real, 9] EY = {0.0, 0.0, 1.0,  0.0, -1.0, 1.0,  1.0, -1.0, -
↪1.0};
    const array[real, 9] W  = {4.0/9.0,
        1.0/9.0, 1.0/9.0, 1.0/9.0, 1.0/9.0,
        1.0/36.0, 1.0/36.0, 1.0/36.0, 1.0/36.0};
}

lattices {
    lattice f(2, 9, real, {N, N}, {false, false});
}

function Macro(array[real, 9] fIn) -> (real rho, real ux, real uy) {
    rho = sum(fIn);
    ux = dot_prod(EX, fIn) / rho;
    uy = dot_prod(EY, fIn) / rho;
}
```

(continues on next page)

(continued from previous page)

```
function Eq(real rho, real ux, real uy) -> (array[real, 9] feq) {
    real usq = ux * ux + uy * uy;
    array[real, 9] eu = EX * ux + EY * uy;
    feq = W * rho * (1.0 + 3.0 * eu + 4.5 * eu * eu - 1.5 * usq);
}

initializer InitFluid { f.out[x, y | :] = W; }

dynamic FluidDynamic {
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    real rho; real ux; real uy;
    {rho, ux, uy} = Macro(fIn);
    f.out[x, y | :] = fIn - TAU_INV * (fIn - Eq(rho, ux, uy));
}

gauge Hydro -> (real rho, real ux, real uy) {
    {rho, ux, uy} = Macro(f.in[x, y | :]);
}

dynamic BounceBack {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY});
}

dynamic MovingLid {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY})
                      + 6.0 * W * (EX * U_LID);
}

gauge BoundaryHydro -> (real rho, real ux, real uy) {
    rho = 1.0;
    ux = U_LID * real(y == N - 1);
    uy = 0.0;
}

node_classes {
    node_class FLUID(InitFluid, FluidDynamic, Hydro);
    node_class WALL(InitFluid, BounceBack, BoundaryHydro);
    node_class LID(InitFluid, MovingLid, BoundaryHydro);
}

geometry {
    nodes[:, :]     = FLUID;
    nodes[0, :]     = WALL;     // left
    nodes[N - 1, :] = WALL;     // right
    nodes[:, 0]     = WALL;     // bottom
    nodes[:, N - 1] = LID;      // top lid
}
```

The three new constructs are multi-output functions, the `dot_prod` intrinsic, and `bounce_back_dfs`.

6.3.1 Multi-output functions: `Macro` and destructuring

The `Macro` function returns *three* values — the density and the two velocity components — via a multi-output signature:

```
function Macro(array[real, 9] fIn) -> (real rho, real ux, real uy) {
    rho = sum(fIn);
    ux = dot_prod(EX, fIn) / rho;
    uy = dot_prod(EY, fIn) / rho;
}
```

Important

A multi-output function cannot be called inside an expression. Instead, destructure its return tuple into pre-declared variables:

```
real rho; real ux; real uy;
{rho, ux, uy} = Macro(fIn);
```

The names after `->` establish the result order. Every named result must have a target on the destructuring left-hand side, in the same order, and every target variable must already be declared with the matching type.

The compiler inlines the function body, so `Macro` carries no function-call overhead in the inner loop — the three outputs become three scalar assignments in the generated target code.

6.3.2 `dot_prod`: fast first moment

```
ux = dot_prod(EX, fIn) / rho;
```

`dot_prod(a, b)` returns the inner product $\sum_i a_i b_i$ as a scalar. Using `dot_prod(EX, fIn)` instead of `sum(EX * fIn)` states the mathematical operation directly and lets backends emit a fused multiply-accumulate reduction. The two forms are semantically equivalent.

The first velocity moment is $\rho u_x = \sum_i c_{i,x} f_i$; since each `EX[i]` multiplies the corresponding `fIn[i]` and the results are summed, `dot_prod(EX, fIn)` is the direct translation.

6.3.3 Bounce-back with `bounce_back_dfs`

The no-slip walls use **full-way bounce-back** (Krüger et al. (2017) §5.3.3.2): a wall cell stores the populations that arrive from the fluid and sends each one back, inverted, on the next step. In GeLB this is a one-liner:

```
dynamic BounceBack {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY});
}
```

`bounce_back_dfs(f, x, y, {EX, EY})` is a streaming intrinsic, like `upstream_dfs`, but it reads the other way round: the `i`-th entry of the returned array is `f.in[x + EX[i], y + EY[i] | opp(i)]`, the population of the neighbour in direction `i` that points back at the wall cell, where `opp(i)` is the index of the DF with the opposite velocity vector $\mathbf{c}_{\mathrm{opp}(i)} = -\mathbf{c}_i$. On the next step the fluid neighbour pulls slot `i` from the wall cell, so every population that reaches the wall goes back along the link it came in on. A link that points out of the grid (the wall cells sit on its edge) gives the cell's own `f.in[x, y | opp(i)]`; no cell ever pulls that slot, so it only has to be defined. The compiler computes the `opp` permutation from the velocity arrays at compile time, avoiding manual index-permutation errors in bounce-back implementations.

For D2Q9 with the standard numbering the permutation is `opp = (0, 3, 4, 1, 2, 7, 8, 5, 6)` — the rest population stays fixed, the four cardinals swap in pairs, and the four diagonals swap in pairs. If you use a different D2Q9 ordering, `bounce_back_dfs` derives the corresponding permutation automatically from your velocity arrays.

The wall cells occupy the outer rows of the grid, but the no-slip surface does not: with bounce-back it lies midway between the wall node and the first fluid node, at $x = 0.5$ and $x = N - 1.5$ (and likewise in y). The fluid therefore spans $L = N - 2$ lattice spacings, and that is the length in the Reynolds number: the program's $N = 100$, $U_{\text{lid}} = 0.1$ and $\nu = 0.01$ give $\text{Re} = 0.1 \times 98/0.01 = 980$, and `-D NU=0.0098` gives exactly 1000.

6.3.4 Moving-wall bounce-back: adding the lid velocity

A moving wall combines bounce-back with a momentum-transfer correction:

```
dynamic MovingLid {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY})
                      + 6.0 * W * (EX * U_LID);
}
```

Following Ladd (1994) and Krüger et al. (2017) §5.3.3 (Eq. 5.26), for a wall moving tangentially at velocity $\mathbf{u}_w$, the bounce-back rule in terms of the incoming pre-reflection direction i (with opposite outgoing direction $\bar{i}$) reads

$$f_{\bar{i}}(\mathbf{x}_b, t+1) = f_i^*(\mathbf{x}_b, t) - 2w_i\rho_w \frac{\mathbf{c}_i \cdot \mathbf{u}_w}{c_s^2},$$

where $\mathbf{x}_b$ is the fluid node next to the wall, $\rho_w \approx 1$ at the wall in the incompressible limit and $c_s^2 = 1/3$ for D2Q9 — giving the prefactor $2/(1/3) = 6$. In the full-way form the reflected population passes through the wall cell and comes back a step later than in Eq. (5.26), carrying the same correction. Because `bounce_back_dfs(f, x, y, {EX, EY})` indexes the wall cell's output slot by the *post-reflection* direction i (whose lattice velocity is opposite to the pre-reflection direction, $\mathbf{c}_i = -\mathbf{c}_{\bar{i}}$), the sign of the momentum correction flips from minus to plus:

$$f_i^{\text{out}}(\mathbf{x}_w, t) = f_{\bar{i}}^{\text{in}}(\mathbf{x}_w + \mathbf{c}_i, t) + 6w_i(\mathbf{c}_i \cdot \mathbf{u}_w),$$

with $\mathbf{x}_w$ the wall node and $\mathbf{x}_w + \mathbf{c}_i$ the fluid node the population came from.

In GeLB that correction is `6.0 * W * (EX * U_LID)`: a 9-element vector that the compiler adds to the bounced-back DFs element-wise (compare also Mohamad (2019) §8.3.1, Eq. 8.17). Here `U_LID` is a scalar, `EX` and `W` are 9-element arrays, `EX * U_LID` broadcasts the scalar across the array, and the remaining products are element-wise.

Because the lid moves in the `+x` direction, only the DFs with non-zero `EX[i]` components (the east/west cardinals and the four diagonals) receive a non-zero correction; the rest and north/south components of `EX` are zero and vanish automatically.

6.3.5 The BGK dynamic block

```
dynamic FluidDynamic {
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    real rho; real ux; real uy;
    {rho, ux, uy} = Macro(fIn);
    f.out[x, y | :] = fIn - TAU_INV * (fIn - Eq(rho, ux, uy));
}
```

The bulk dynamic follows the standard four-step pattern: pull-stream from upstream neighbours, compute macroscopic moments, evaluate the equilibrium distribution, and relax. The BGK relaxation is written in the form $f - \omega(f - f^{\text{eq}})$ rather than $(1-\omega)f + \omega f^{\text{eq}}$; the two are algebraically equivalent, and the subtraction form composes cleanly with body-force source terms (see *Poiseuille with Body Force*). Whereas interior `FLUID` cells pull from all eight neighbours via `upstream_dfs`, the boundary `WALL` and `LID` cells execute `BounceBack` and `MovingLid`, which

use `bounce_back_dfs`: from each neighbour they read the population that points back at them, and the links that leave the grid are guarded, so these classes may sit on the grid edge. A wall cell holds populations on their way back into the fluid, so their moments are not a fluid velocity; the walls report the prescribed wall velocity through their own gauge, `BoundaryHydro` (0 on the walls, U_{lid} on the lid, and $\rho = 1$ as a display value).

6.4 Geometry: painting the walls

```
geometry {
    nodes[:, :]     = FLUID;
    nodes[0, :]     = WALL;     // left
    nodes[N - 1, :] = WALL;     // right
    nodes[:, 0]     = WALL;     // bottom
    nodes[:, N - 1] = LID;      // top lid
}
```

Five assignments define the domain layout: the first line fills the entire grid with `FLUID`, and the next four lines overwrite the outer edges with `WALL` (left, right, bottom) and `LID` (top). Because geometry statements execute in order ("last paint wins"), each of the four corner cells is painted twice: the two bottom corners `(0, 0)` and `(N - 1, 0)` remain `WALL`, while the two top corners `(0, N - 1)` and `(N - 1, N - 1)` are overwritten last by `nodes[:, N - 1] = LID` and become `LID` cells. The physical and numerical consequences of this corner assignment are examined in detail below.

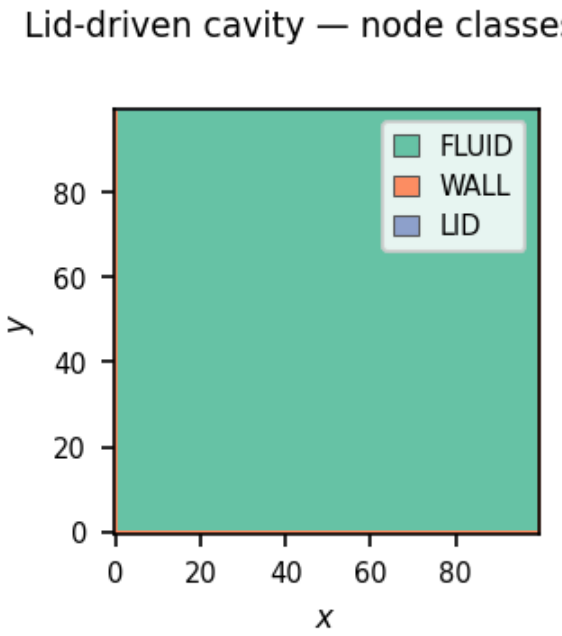


Fig. 1: Node-class layout: FLUID (teal) interior, WALL (orange) on three sides, LID (blue) on the top edge.

6.5 Running it

```
# Default: cpu_highway (portable-SIMD C++ + OpenMP)
gelb-run -n 8000 -w 8000 \
    examples/lbm/2d/lid_driven_cavity_2d_bgk_re1000.gelb
```

(continues on next page)

(continued from previous page)

```
# Single-threaded Fortran reference — same source, no code changes
gelb-run -b cpu_fortran_serial -n 8000 -w 8000 \
    examples/lbm/2d/lid_driven_cavity_2d_bgk_re1000.gelb
```

The `-w 8000` flag instructs the gauge to write only at the final step. A single `gelb-run` invocation takes 2–3 seconds on an 8-core laptop with the default `cpu_highway` backend; the single-threaded `cpu_fortran_serial` reference is several times slower. In VS Code (*Working in VS Code*), set *Time steps* and *Write every* to 8000 in the Run view and press **Run**; the *Backend* row chooses `cpu_fortran_serial` for the second command.

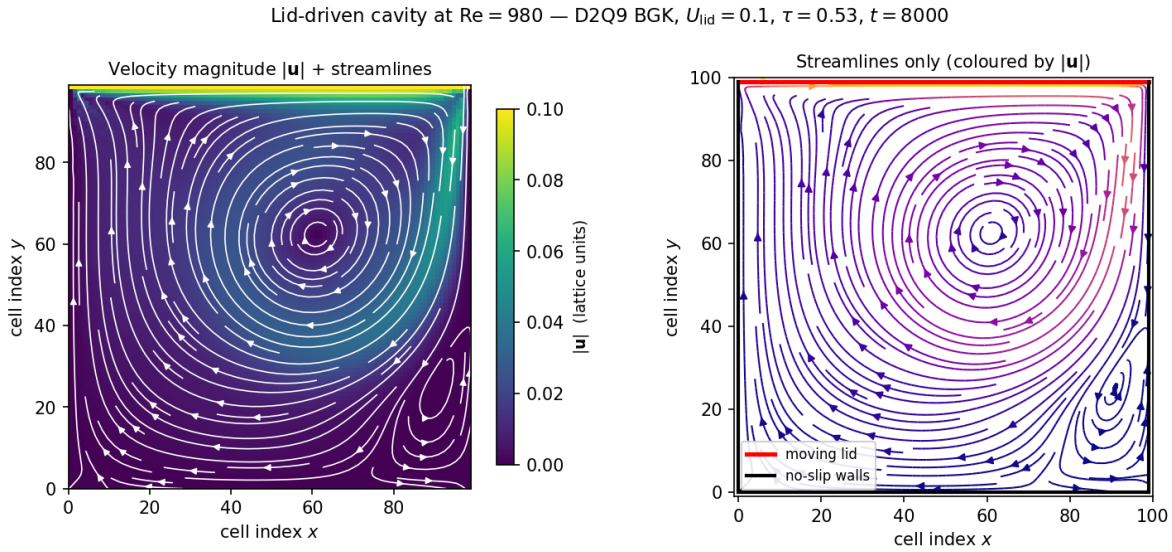


Fig. 2: **Left:** velocity magnitude $|\mathbf{u}|$ with streamlines overlaid in white. The lid (top edge) drives the flow from left to right; the fluid accelerates along the lid, descends the right wall, recirculates through the cavity, and climbs the left wall. The boundary layer along the lid is thin and intense (bright yellow) where the shear is highest. **Right:** streamline topology coloured by speed, showing the **primary vortex** (shifted toward the upper-right by advection at Re $= 980$) and the **counter-rotating vortex** in the lower-right corner; the one in the lower-left corner is still forming at `t = 8000` (the flow along the bottom has just begun to reverse there). This qualitative pattern matches Mohamad (2019) §8.8.1 Fig. 8.10 (p. 125) and Ghia, Ghia & Shin (1982) Fig. 6. At `t = 8000` steps the vortex topology is already established, though the flow is still developing: the viscous time L^2/ν at these parameters is about 10^6 steps. The quantitative comparison with Ghia, Ghia & Shin (1982) below uses runs long enough to reach the steady state.

6.6 A note on the Mach number

The tutorial's example files default to $U_{\text{lid}} = 0.1$ to match the parameters of Mohamad (2019) §8.8.1, which gives a lid Mach number of $\text{Ma}_{\text{lid}} = U_{\text{lid}}/c_s = U_{\text{lid}}\sqrt{3} \approx 0.173$. That exceeds the Ma $\ll 0.1$ ($U^*_{\text{max}} < 0.03$–0.1) accuracy guideline of Krüger et al. (2017) §7.2.1–7.2.3 (Table 7.2) for quantitative incompressible comparisons.

For the quantitative validation below, we override the defaults to $U_{\text{lid}} = 0.05$ (Ma ≈ 0.087) on an $N = 129$ grid, the grid size Ghia, Ghia & Shin (1982) used. The validation script passes N, U_{lid} and the τ that holds Re fixed as `-D` command-line overrides; the `.gelb` source file itself is unchanged.

6.7 Corners and the no-slip discontinuity

The four cavity corners warrant closer inspection. Each corner is the intersection of two walls, and at the **top corners** those two walls have *different* prescribed velocities: the lid moves at U_{lid} while the left and right walls are stationary. Continuum hydrodynamics therefore has a velocity **discontinuity** at the two upper corners (jumping from $u_x = U_{\text{lid}}$ to $u_x = 0$). Krüger et al. (2017) §5.3.6 (footnote 23) note explicitly that standard smooth-corner LBM analysis excludes the lid-driven cavity for this reason.

In our GeLB geometry block:

```
nodes[:, :]     = FLUID;
nodes[0, :]     = WALL;
nodes[N - 1, :] = WALL;
nodes[:, 0]     = WALL;
nodes[:, N - 1] = LID;     // last paint wins at the top corners
```

the top-left and top-right cells are assigned `LID` (by paint order), while the bottom-left and bottom-right cells are assigned `WALL`. There is **no separate corner node class** — each corner cell executes the standard `MovingLid` or `BounceBack` dynamic of its wall. This works because `bounce_back_dfs` sends back every population that arrives at the cell, along whichever link it came in on, which matches the corner prescription of the **link-wise** bounce-back approach (Krüger et al. (2017) §5.3.6.1).

For the **wet-node / Zou–He / NEBB** family of boundary conditions, corners require dedicated closures. At a concave corner like the bottom-left node, five non-rest post-streaming populations (`E`, `N`, `NE`, `NW`, `SE`) plus the rest population f_0 are unknown, whereas the flat-wall NEBB relations supply only three equations. The standard concave-corner closure of Krüger et al. (2017) §5.3.6.2 (Eqs. 5.55–5.59) proceeds in four steps:

1. Apply the non-equilibrium bounce-back rule to the three normal-direction pairs pointing into the fluid domain: $f_1^{\text{neq}} = f_3^{\text{neq}}$, $f_2^{\text{neq}} = f_4^{\text{neq}}$, $f_5^{\text{neq}} = f_7^{\text{neq}}$.
2. Substitute these into the macroscopic momentum equations; the two momentum constraints collapse to a single equation for the difference $f_6 - f_8 = \frac{1}{6c}\,\rho_w\,(u_{w,y} - u_{w,x})$.
3. Close the system by requiring the remaining "buried" link populations f_6 and f_8 to have equal magnitude and opposite sign ($f_6 = -f_8$), yielding $f_6 = -f_8 = \frac{1}{12c}\,\rho_w\,(u_{w,y} - u_{w,x})$ (Krüger et al. (2017) Eq. 5.58), which simplifies to $f_6 = f_8 = 0$ at a stationary corner.
4. Recover f_0 from mass conservation (Eq. 5.59), with ρ_w extrapolated from neighbouring nodes.

At the **top corners**, where the wall velocity is discontinuous, this wet-node closure is not well-posed. Standard practice in the LBM literature is either to extrapolate from the interior or to use link-wise bounce-back, which absorbs the corner singularity locally without destabilising the bulk (the buried links never stream into the fluid interior). Near-corner boundary-layer resolution remains one of the contributors to the residual L_2 error at high Reynolds numbers, where the lid boundary layer thins to a few lattice spacings.

Both `lid_driven_cavity_2d_bgk.gelb` and `lid_driven_cavity_2d_mrt.gelb` use link-wise bounce-back, so the corners are handled by the same wall dynamics as the straight edges.

See also

Krüger et al. (2017) §5.3.6 (*Corners*) for the full taxonomy: convex versus concave corners, link-wise versus wet-node treatments, and the buried-link closure.

6.8 Quantitative validation against Ghia, Ghia & Shin (1982)

To complement the qualitative streamline comparison with a quantitative benchmark, we compare the steady-state velocity profiles against Ghia, Ghia & Shin (1982), who tabulated centreline velocities at Re $= 100, 400, 1000, 3200, \ldots$ for the unit-square lid-driven cavity at 17 non-uniform y/L and x/L stations (Tables I and II of their paper).

The validation script `scripts/validate_ldc.py` runs `examples/lbm/2d/lid_driven_cavity_2d_bgk.gelb` (and its MRT and incompressible siblings, `lid_driven_cavity_2d_mrt.gelb` and `lid_driven_cavity_2d_incomp_zou.gelb`) at each Reynolds number until the flow is steady, extracts $u(y)$ along the vertical centreline ($x = L/2$) and $v(x)$ along the horizontal centreline ($y = L/2$), interpolates onto the 17 stations of Ghia, Ghia & Shin (1982), and reports the relative L_2 error. The table below comes from

```
uv run python scripts/validate_ldc.py --re 100 1000 --n 129 \
    --u-lid 0.05 --collision bgk mrt incomp_zou --backend gpu_cuda_cpp
# → docs/user/figures/data/07_ldc_ghia_validation.json
```

on a CUDA host; on a CPU host substitute `--backend cpu_highway` (portable-SIMD C++ + OpenMP) or `--backend cpu_fortran_serial` (single-threaded reference, the script's default). The reference tables are stored in `gelb/tests/data/ghia_1982_ldc.py` for both Re $= 100$ and Re $= 1000$.

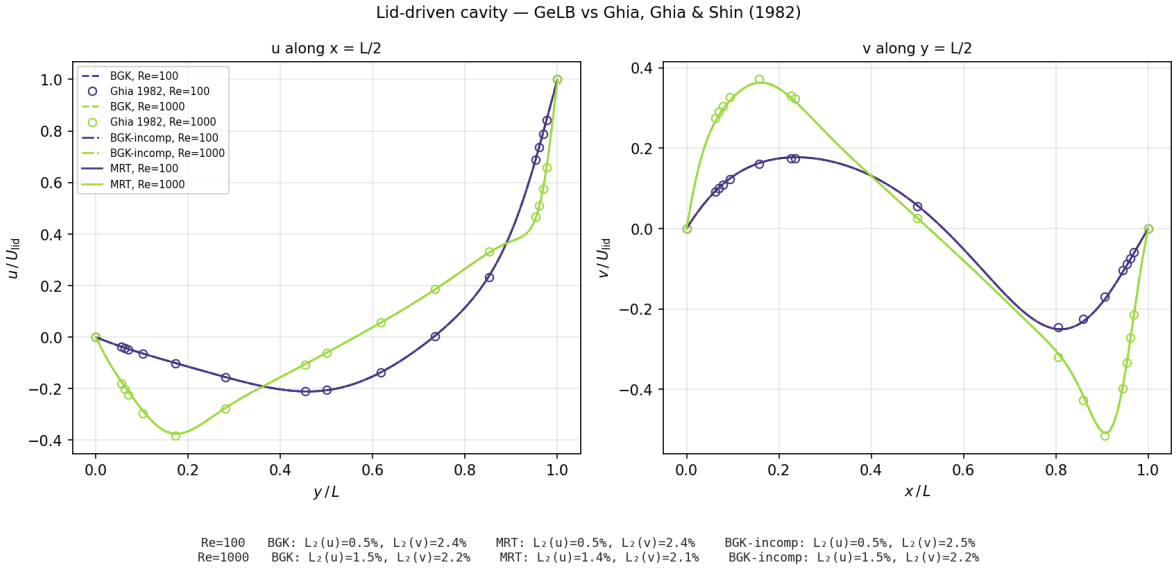


Fig. 3: Centreline velocity profiles at Re $= 100$ (purple) and Re $= 1000$ (green) on an $N = 129$ grid with $U_{\text{lid}} = 0.05$ (Ma ≈ 0.087): GeLB curves (lines) overlaid on the reference points of Ghia, Ghia & Shin (1982) (open circles). **Left:** $u(y)$ along the vertical centreline at $x = L/2$. **Right:** $v(x)$ along the horizontal centreline at $y = L/2$. Both velocities are normalised by U_{lid}.

The relative L_2 errors on an $N = 129$ grid with $U_{\text{lid}} = 0.05$ (Ma ≈ 0.087), run on the `gpu_cuda_cpp` backend, are:

Re	Collision	τ	Steps	$L_2(u)$	$L_2(v)$
100	BGK	0.691	1.3 M	0.50 %	2.40 %
	MRT	0.691	1.3 M	0.50 %	2.41 %
	BGK-incomp (Zou 1995)	0.691	1.3 M	0.49 %	2.46 %
1000	BGK	0.519	13 M	1.52 %	2.21 %
	MRT	0.519	13 M	1.45 %	2.07 %
	BGK-incomp (Zou 1995)	0.519	13 M	1.50 %	2.21 %

At Re $= 100$ the GeLB curves lie on the Ghia, Ghia & Shin (1982) points: the relative L_2 error is 0.5 % on the primary u-profile and about 2.4 % on v, and the three collision operators agree to within 0.06 percentage points. At Re $= 1000$ the errors are 1.5 % on u and 2.1–2.2 % on v; MRT is slightly closer to the benchmark than BGK (1.45 % and 2.07 % against 1.52 % and 2.21 %), and the incompressible equilibrium of Zou et al. (1995) gives the same errors as standard BGK to within a few hundredths of a percentage point.

6.9 What's next

Poiseuille with Body Force keeps the 2D setting and introduces body forcing and MRT collision — the two ingredients needed for pressure- or force-driven channel flow with second-order accuracy, validated directly against the exact parabolic Poiseuille solution.

If you want to push the lid-driven cavity to higher Reynolds numbers, switching from BGK to MRT or adding a subgrid turbulence model improves stability — see *Poiseuille with Body Force* for MRT and `examples/lbm/2d/lid_driven_cavity_2d_bgk_smagorinsky.gelb` for a Smagorinsky LES variant.

See also

Krüger et al. (2017) §5.3.3 (*Solid Boundaries: Bounce-Back Approach*) for the moving-wall bounce-back derivation and the half-grid-spacing wall location that makes bounce-back second-order accurate; §7.2 (*Parameter Selection*, specifically Eq. 7.22 and Table 7.2) for the Mach-number bound and the trade-off between Re, grid size, τ, and timestep count.

Mohamad (2019) §8.8.1 (*Lid-Driven Cavity*, pp. 124–125) for the worked Re $= 1000$, $N = 100$, $U_{\text{lid}} = 0.1$ setup, with streamline topology in Fig. 8.10 and horizontal/vertical centreline velocity profiles compared against a finite-volume benchmark in Fig. 8.11.

Ghia, U., Ghia, K. N. & Shin, C. T. (1982) "High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method." *Journal of Computational Physics*, **48**, 387–411. The canonical reference benchmark for the 2D lid-driven cavity.

CHAPTER 7

Poiseuille with Body Force

The *Lid-Driven Cavity* benchmark ran the Navier–Stokes equations at Re $= 1000$ and compared the centreline velocity profiles against a reference tabulation. Where an exact closed-form solution exists, we can go further and verify the formal order of accuracy across a sequence of grids. This chapter sets up a body-force-driven planar Poiseuille flow with analytical validation and a four-grid convergence study. Along the way, it introduces three ingredients used throughout quantitative LBM simulations: **MRT collision**, **Strang-split body forcing**, and **full-way bounce-back** (the wall midway between the nodes).

> **Note**
>
> The example walked here is `examples/lbm/2d/poiseuille_flow_2d_mrt.gelb`. Its header comment marks it as validated, with second-order convergence; this chapter explains how each of the three ingredients contributes to that rate.

7.1 The physics

Consider incompressible flow between two infinite parallel plates separated by a distance H, driven by a uniform streamwise body force density $G = \rho g_x$ (Krüger et al. (2017) §6.2.1; Mohamad (2019) §8.8.2). In steady state the flow is unidirectional ($\partial_x = 0$, $u_y = 0$), pressure is uniform, and the streamwise momentum balance $\nu\, \mathrm{d}^2 u_x/\mathrm{d}y^2 + G/\rho = 0$ with no-slip walls at $y_{\text{eff}} = 0$ and $y_{\text{eff}} = H$ integrates to the familiar parabola:

$$u(y) = \frac{G}{2\rho\nu}\, y_{\text{eff}}\,(H - y_{\text{eff}})\,, \qquad u_{\max} = \frac{GH^2}{8\rho\nu}\,,$$

with $\rho = \rho_0 = 1$ in lattice units and y_{eff} measured from the bottom no-slip plane. Because both the velocity profile and its L_2 error norm have closed-form expressions, planar Poiseuille flow is the standard benchmark for testing spatial order of accuracy.

Specifically, we can measure the relative L_2 error of the simulated profile against the exact parabola across several grid resolutions and plot the error against the grid spacing $h = 1/H$. For a second-order-accurate scheme the error scales as $\mathcal{O}(h^2)$, which appears on a log-log plot against h as a straight line of slope $+2$ (or slope -2 when plotted against the cell count H). If the measured slope flattens toward 1, at least one component of the discretisation — the collision operator, the forcing scheme, or the wall boundary condition — is introducing a first-order error.

7.2 Why MRT, and why now?

The *Lid-Driven Cavity* example used single-relaxation-time BGK collision, which is compact and effective at moderate viscosity when qualitative or engineering-level accuracy suffices. For strict convergence studies and low-viscosity flows, however, BGK has two well-known limitations (Krüger et al. (2017) §10.1–10.4):

1. **All moments relax at a single rate** ($\omega = 1/\tau$). In the Chapman–Enskog expansion, the conserved moments (density and momentum) do not relax; the shear and normal stress moments relax at the rate $1/\tau$ that fixes the kinematic viscosity $\nu = c_s^2(\tau - \frac{1}{2})\Delta t$; and the higher-order kinetic ("ghost") moments do not enter the incompressible Navier–Stokes equations at leading order. Tying all nine rates together forces a compromise between stability at low τ and small truncation error at high τ.
2. **Viscosity-dependent wall slip with bounce-back.** Under BGK collision, the effective no-slip plane of bounce-back shifts away from the geometric mid-link plane by a term proportional to $(\tau - \frac{1}{2})^2$ (Ginzburg (2008); Krüger et al. (2017) §5.3.3.3 and §10.4). Unless the relaxation rates of the odd and even moments are decoupled, that numerical slip alters the apparent channel width as τ varies.

MRT (Multiple-Relaxation-Time collision; Lallemand & Luo (2000); d'Humières et al. (2002); Krüger et al. (2017) §10.4) resolves both issues. Distribution functions are mapped into *moment space* via a fixed invertible transformation matrix $\mathbf{m} = M\mathbf{f}$, each moment relaxes independently via a diagonal relaxation matrix S, and the post-collision state is mapped back to velocity space:

$$\mathbf{f}^*(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - M^{-1} S\big(\mathbf{m}(\mathbf{x}, t) - \mathbf{m}^{\mathrm{eq}}(\mathbf{x}, t)\big) .$$

For the D2Q9 lattice, the orthogonal Gram–Schmidt basis of Lallemand & Luo (2000) (Krüger et al. (2017) §10.4.2, Eq. 10.30) orders the nine moments as

$$\mathbf{m} = (\rho,\ e,\ \epsilon,\ j_x,\ q_x,\ j_y,\ q_y,\ p_{xx},\ p_{xy})^{\mathsf{T}} ,$$

representing density (ρ), energy (e), energy squared (ϵ), momentum (j_x, j_y), energy flux (q_x, q_y), and the diagonal and off-diagonal viscous stress components (p_{xx}, p_{xy}). The conserved moments have zero relaxation rate ($s_\rho = s_{jx} = s_{jy} = 0$), the two deviatoric stress moments relax at $s_{pxx} = s_{pxy} = 1/\tau$ to set the shear viscosity, and the non-hydrodynamic rates ($s_e, s_\epsilon, s_{qx}, s_{qy}$) are chosen around 1.1 to damp acoustic and kinetic modes cleanly.

In GeLB, MRT is not a special built-in collision keyword — you express it directly in the `dynamic` block using the `matmul` and `inverse` intrinsics. Because `M` is a compile-time constant, `inverse(M)` is evaluated at compile time as well.

7.3 Body forcing with Strang splitting

Adding a body force $\mathbf{g}$ to the lattice Boltzmann equation requires care to avoid discrete lattice artefacts of order $\mathcal{O}(\Delta t)$ (Guo, Zheng & Shi (2002); Krüger et al. (2017) §6.2–6.4). If the force is applied only before or only after collision, the macroscopic momentum recovered by the Chapman–Enskog analysis differs from the physical velocity by half a forcing step. Second-order accuracy requires two conditions:

1. The macroscopic velocity entering the equilibrium distribution must be shifted by half a force impulse: $\rho\mathbf{u} = \sum_i \mathbf{e}_i f_i + \frac{1}{2}\mathbf{G}\Delta t$.
2. The total momentum increment over the time step must equal $\mathbf{G}\Delta t$.

In moment space, both conditions are satisfied cleanly by **Strang splitting** the momentum impulse around the collision step (compare Krüger et al. (2017) §6.2.3 and §10.5):

1. Transform pre-collision distributions to moments: $\mathbf{m} = M\mathbf{f}^{\mathrm{in}}$.
2. Add half the force impulse ($\frac{1}{2}G\Delta t$) to the streamwise momentum moment j_x.
3. Evaluate the equilibrium moments $\mathbf{m}^{\mathrm{eq}}(\rho, \mathbf{u})$ using the half-step velocity $u_x = j_x/\rho$, $u_y = j_y/\rho$, and relax: $\mathbf{m} \leftarrow \mathbf{m} - S(\mathbf{m} - \mathbf{m}^{\mathrm{eq}})$.

4. Add the second half of the force impulse ($\frac{1}{2}G\Delta t$) to j_x.
5. Transform back to populations: $\mathbf{f}^{\text{out}} = M^{-1}\mathbf{m}$.

Because j_x is a conserved moment ($s_{jx} = 0$), the first half-impulse passes through the relaxation step unchanged and automatically supplies the half-step velocity shift $\mathbf{u} = (\sum_i \mathbf{e}_i f_i + \frac{1}{2}\mathbf{G}\Delta t)/\rho$ to $\mathbf{m}^{\text{eq}}$, while the second half-impulse completes the full momentum increment $G\Delta t$. (For a unidirectional channel flow with $u_y = 0$, $G_y = 0$, and $\partial_x = 0$, the higher-order velocity-force product terms in the full MRT forcing source of Krüger et al. (2017) §10.5 Eq. (10.35) vanish identically, so Strang splitting of j_x is exact to second order.)

7.4 Bounce-back — what, and why

In a pull-streaming solver, fluid cells gather their incoming populations from neighbouring cells' `f.out` arrays via `upstream_dfs`. The wall cells at $y = 0$ (bottom) and $y = N_y - 1$ (top) sit outside the fluid domain: they do not run fluid collision, and their sole job is to place the reflected populations into their own `f.out` slots so that the adjacent fluid row ($y = 1$ or $y = N_y - 2$) pulls them on the next time step.

To achieve second-order spatial accuracy, we use **bounce-back** (Krüger et al. (2017) §5.3.3), in which the physical no-slip plane sits *midway* along each lattice link crossing the boundary — at $y = 0.5$ between the bottom wall row ($y = 0$) and the first fluid row ($y = 1$), and at $y = N_y - 1.5$ at the top. The effective channel height spanned by the $N_y - 2$ interior fluid rows is therefore $H = N_y - 2$, with cell centres located at $y_{\text{eff}} = y - 0.5 \in \{0.5, 1.5, \ldots, H - 0.5\}$.

In GeLB the bottom-wall dynamic block implements this reflection by reading the *outgoing* populations of the adjacent fluid row ($y + 1 = 1$) along each link that crosses the wall and storing them in the opposite (upward-pointing) slots of the wall row:

```
dynamic BounceBackBot {
    f.out[x, y | 0] = 0.0;
    f.out[x, y | 1] = 0.0;
    f.out[x, y | 2] = f.in[x,     y + 1 | 4];   // N  ← S  at (x,   1)
    f.out[x, y | 3] = 0.0;
    f.out[x, y | 4] = 0.0;
    f.out[x, y | 5] = f.in[x + 1, y + 1 | 7];   // NE ← SW at (x+1, 1)
    f.out[x, y | 6] = f.in[x - 1, y + 1 | 8];   // NW ← SE at (x-1, 1)
    f.out[x, y | 7] = 0.0;
    f.out[x, y | 8] = 0.0;
}
```

The pattern is: for each *wall-incoming* DF index entering the fluid from the bottom wall (N = 2, NE = 5, NW = 6), read the *opposite* downward-pointing DF (S = 4, SW = 7, SE = 8) from the fluid cell from which that link originated. At steady state, when the fluid row at $y = 1$ pulls `upstream_dfs` from $y = 0$, it receives back the exact post-collision populations it sent toward the wall. The remaining six DFs at the wall node are never read by the fluid interior and are set to zero.

Important

Rule: Every `dynamic` block that writes individual distribution functions must assign all Q populations of the lattice (either via explicit per-DF statements `f.out[... | i] = ...` for each $i \in \{0, \ldots, Q-1\}$ or via the slice shorthand `f.out[... | :] = ...`).

If you omit one of the nine writes, the GeLB compiler rejects the program with an initialiser-coverage diagnostic:

```
[Problem] Dynamic block 'BounceBackBot' (for node class
'WALL_BOT') does not write all DFs of lattice 'f':
missing DFs [8]

[Hint] Add write statements for the missing distribution
functions, or use the full-range syntax f.out[...|:] to
write all DFs at once.
```

Combined with the boundary-stencil check, this prevents silently leaving distribution slots uninitialised or reading across an out-of-bounds domain edge.

7.5 The program

Here is the core of `examples/lbm/2d/poiseuille_flow_2d_mrt.gelb`, starting with the constants and lattice declarations:

```
global_constants {
    const int NY = 32;    // wall-normal resolution
    const int NX = 128;   // streamwise extent
    const real TAU = 0.54;
    const real TAU_INV = 1.0 / TAU;

    // Body force: G ∝ 1/(NY-2)^2 so Mach number is constant
    // as NY grows during a convergence sweep.
    const int  NY_BASE = 32;
    const real G_BASE  = 0.00001;
    const real G = G_BASE * real((NY_BASE - 2) * (NY_BASE - 2))
                          / real((NY - 2) * (NY - 2));

    const array[real, 9] EX = {0.0, 1.0, 0.0, -1.0,  0.0, 1.0, -1.0, -1.0,
↪1.0};
    const array[real, 9] EY = {0.0, 0.0, 1.0,  0.0, -1.0, 1.0,  1.0, -1.0, -
↪1.0};
    const array[real, 9] W  = {4.0/9.0,
        1.0/9.0, 1.0/9.0, 1.0/9.0, 1.0/9.0,
        1.0/36.0, 1.0/36.0, 1.0/36.0, 1.0/36.0};

    // MRT transformation matrix M (Lallemand & Luo, 2000)
    const array[real, {9, 9}] M = {
        { 1.0,  1.0,  1.0,  1.0,  1.0,  1.0,  1.0,  1.0,  1.0},
        {-4.0, -1.0, -1.0, -1.0, -1.0,  2.0,  2.0,  2.0,  2.0},
        { 4.0, -2.0, -2.0, -2.0, -2.0,  1.0,  1.0,  1.0,  1.0},
        { 0.0,  1.0,  0.0, -1.0,  0.0,  1.0, -1.0, -1.0,  1.0},
        { 0.0, -2.0,  0.0,  2.0,  0.0,  1.0, -1.0, -1.0,  1.0},
        { 0.0,  0.0,  1.0,  0.0, -1.0,  1.0,  1.0, -1.0, -1.0},
        { 0.0,  0.0, -2.0,  0.0,  2.0,  1.0,  1.0, -1.0, -1.0},
        { 0.0,  1.0, -1.0,  1.0, -1.0,  0.0,  0.0,  0.0,  0.0},
        { 0.0,  0.0,  0.0,  0.0,  0.0,  1.0, -1.0,  1.0, -1.0}};

    const array[real, {9, 9}] M_INV = inverse(M);
```

(continues on next page)

(continued from previous page)

```
    const array[real, 9] S_DIAG = {0.0, 1.1, 1.1, 0.0, 1.1,
                                   0.0, 1.1, TAU_INV, TAU_INV};
}

lattices { lattice f(2, 9, real, {NX, NY}, {true, false}); }
```

Four points to note in this block:

- **The MRT transformation matrix** `M` is a two-dimensional `array[real, {9, 9}]` constant initialised by a nested array literal. Because `M` is known at compile time, `matmul(M, ...)` compiles into a direct unrolled matrix–vector product in the generated target code.
- **The inverse matrix** `M_INV` is computed at compile time via the `inverse` intrinsic. `inverse` can only be called on a compile-time constant matrix; the compiler evaluates the inverse once during compilation and embeds the resulting constant array directly in the generated solver, avoiding the need to write out 81 fractional coefficients by hand.
- **The relaxation-rate vector** `S_DIAG` has three zeros (for the conserved moments ρ, j_x, j_y), four non-hydrodynamic rates at 1.1 (for e, ϵ, q_x, q_y), and two viscous stress rates at $1/\tau$ (for p_{xx}, p_{xy}). The ordering matches the rows of `M` (Krüger et al. (2017) §10.4.2, Eqs. 10.30–10.33).
- **Diffusive scaling of the body force** (`G`) keeps the peak velocity $u_{\max} = GH^2/(8\rho\nu)$ — and therefore the Mach number $\mathrm{Ma} = u_{\max}/c_s$ and Reynolds number — constant across all grids in the convergence sweep by scaling $G \propto (N_y - 2)^{-2}$ at fixed $\tau = 0.54$ (Krüger et al. (2017) §7.2.2).
- **The lattice is periodic in x** (`{true, false}`) — the exact steady-state flow is translation-invariant along the streamwise direction. We take `NX = 128` so that transient modes excited by the impulsive start from rest damp cleanly before steady state is sampled.

7.5.1 Function blocks

```
function Macro(array[real, 9] fIn) -> (real rho, real ux, real uy) {
    rho = sum(fIn);
    ux = dot_prod(EX, fIn) / rho;
    uy = dot_prod(EY, fIn) / rho;
}

function EqMoments(real rho, real ux, real uy) -> (array[real, 9] meq) {
    real usq = ux * ux + uy * uy;
    meq = {rho,
           -2.0 * rho + 3.0 * rho * usq,
           rho - 3.0 * rho * usq,
           rho * ux,
           -1.0 * rho * ux,
           rho * uy,
           -1.0 * rho * uy,
           rho * (ux * ux - uy * uy),
           rho * ux * uy};
}
```

Two user-defined functions appear here. `Macro` is the same multi-output helper used in *Lid-Driven Cavity* to compute (ρ, u_x, u_y) for VTK output. `EqMoments` returns the standard low-Mach D2Q9 **moment-space equilibrium** $\mathbf{m}^{\mathrm{eq}} = M\mathbf{f}^{\mathrm{eq}}$ of Lallemand & Luo (2000) (Krüger et al. (2017) §10.4.2, Eq. 10.32) as a 9-element array literal.

7.5.2 The dynamic block — Strang-split MRT

```
dynamic MRTCollStream {
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});

    array[real, 9] moments = matmul(M, fIn);

    // 1st half body force: add 0.5*G to the jx moment (index 3)
    moments = moments + {0.0, 0.0, 0.0, 0.5 * G,
                         0.0, 0.0, 0.0, 0.0, 0.0};

    real rho = moments[0];
    real ux = moments[3] / rho;
    real uy = moments[5] / rho;

    array[real, 9] meq = EqMoments(rho, ux, uy);
    moments = moments - S_DIAG * (moments - meq);

    // 2nd half body force.
    moments = moments + {0.0, 0.0, 0.0, 0.5 * G,
                         0.0, 0.0, 0.0, 0.0, 0.0};

    f.out[x, y | :] = matmul(M_INV, moments);
}
```

Step by step:

Streaming. `upstream_dfs(f, x, y, {EX, EY})` gathers the nine incoming distributions from neighbouring cells, just as in *Lid-Driven Cavity*.

Into moment space. `matmul(M, fIn)` multiplies the 9×9 transformation matrix by the 9-vector of incoming populations, yielding the moment vector `moments`. When applied to a compile-time constant matrix like `M`, the compiler emits an unrolled matrix–vector product in the generated target code.

First half body force. Adding `0.5 * G` at index 3 (j_x) applies the first half of the streamwise force impulse. Because `S_DIAG[3] == 0.0`, this increment is preserved through collision.

Extract the macroscopic velocity. Reading `ux = moments[3] / rho` after the first half-force step automatically includes the $\frac{1}{2}G\Delta t/\rho$ velocity shift required for second-order accuracy when `EqMoments(rho, ux, uy)` is evaluated.

Relax. `moments - S_DIAG * (moments - meq)` performs element-wise relaxation of each moment toward its equilibrium at its own rate.

Second half body force. Adding the second `0.5 * G` to `moments[3]` completes the $G\Delta t$ momentum increment.

Back to velocity space. `f.out[x, y | :] = matmul(M_INV, moments);` maps the relaxed moments back to the outgoing populations via the compile-time inverse matrix `M_INV`.

7.6 Running it

```
# Single run, default resolution NY=32
gelb-run -n 300000 -w 300000 \
    examples/lbm/2d/poiseuille_flow_2d_mrt.gelb

# Convergence sweep — override NY on the command line
```

(continues on next page)

(continued from previous page)

```
for NY in 16 32 64 128; do
    N_STEPS=$((5 * (NY-2) * (NY-2) * 75))   # ≈ 5 * H^2 / nu
    gelb-run -D NY=$NY -n $N_STEPS -w $N_STEPS \
        examples/lbm/2d/poiseuille_flow_2d_mrt.gelb
done
```

In VS Code (*Working in VS Code*), the single run is *Time steps* and *Write every* 300000; one point of the sweep is a *Constants* override such as `NY=64` with its own step count, while the loop itself stays a shell command.

The loop form matches the Poiseuille test in `scripts/validate_analytical.py`. Because the viscous diffusion time across the channel scales as $t_{\text{diff}} = H^2/\nu$ (with $\nu = (\tau - \frac{1}{2})/3 = 0.04/3 \approx 1/75$ at $\tau = 0.54$), scaling the step count as $5(N_y - 2)^2/\nu$ ensures that every resolution runs for five full diffusion times and reaches steady state to machine precision. At `NY = 128` this corresponds to roughly five million steps, which executes in under half a minute on a modern multicore CPU with the default `cpu_highway` backend.

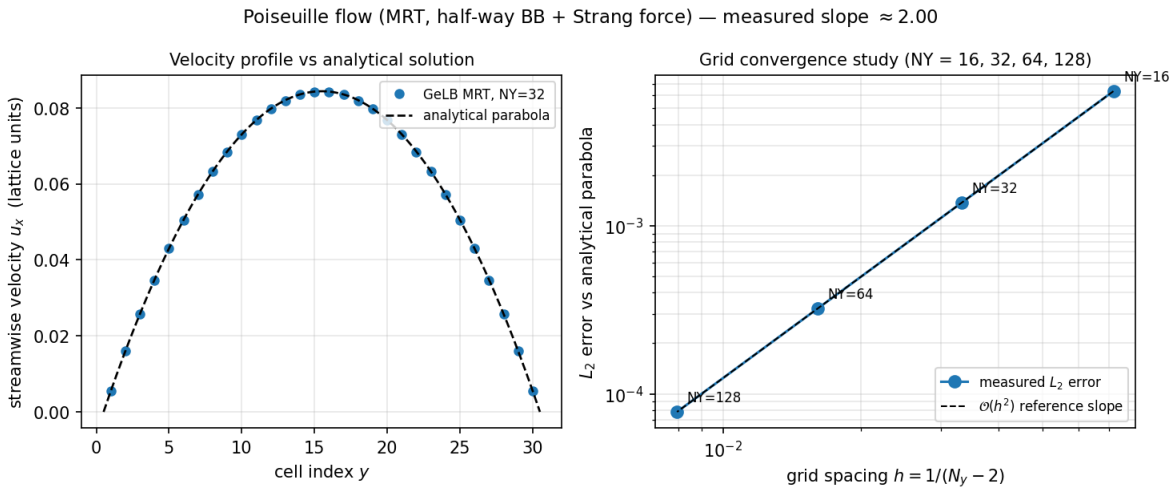


Fig. 1: **Left:** steady-state streamwise velocity profile $u_x(y)$ at `NY = 32` (blue circles, sampled along a vertical column) compared against the analytical parabola $u(y) = Gy_{\text{eff}}(H - y_{\text{eff}})/(2\rho\nu)$ (dashed black). **Right:** log-log plot of the relative L_2 error against grid spacing $h = 1/(N_y - 2)$ for `NY` $\in \{16, 32, 64, 128\}$. The dashed reference line has slope $+2$ ($\mathcal{O}(h^2)$), and the least-squares log-log fit gives a measured slope of 2.00, confirming second-order spatial convergence of the combined MRT collision, Strang-split forcing, and bounce-back scheme.

7.7 Summary

Running the same GeLB model across four grid resolutions via `-D NY=...` overrides and comparing against the exact Poiseuille parabola verifies second-order accuracy quantitatively. Each piece of the numerical method — the 9-moment transformation `matmul(M, fIn)` and its compile-time inverse `inverse(M)`, the Strang-split force impulse in moment space, and the per-population bounce-back stencil — is written directly using the core language constructs introduced in *GeLB in 10 Minutes*.

7.8 What's next

Rayleigh-Bénard Convection introduces **multi-lattice coupling**: one lattice for fluid momentum and a second lattice for temperature, coupled through a Boussinesq buoyancy force in the fluid dynamic block. The MRT collision, Strang-split forcing, and bounce-back patterns from this chapter carry over directly.

GeLB also includes examples of related advanced collision operators: `examples/lbm/2d/poiseuille_flow_2d_trt.gelb` uses the two-relaxation-time (TRT) operator with magic parameter $\Lambda = 3/16$ (Krüger et al. (2017) §10.6), and `examples/lbm/2d/channel_flow_square_obstacle_2d_cascaded.gelb` uses a central-moment (cascaded) collision operator.

See also

Krüger et al. (2017) §5.3.3 (*Solid Boundaries: Bounce-Back*) for bounce-back and its second-order accuracy; Chapter 6 (*Forces*), especially §6.2–6.4, for body-force schemes and the half-step velocity shift; §7.2.2 (*Diffusive Scaling*) for grid refinement at fixed τ; and Chapter 10 (*MRT and TRT*), especially §10.4.2 and §10.5, for the D2Q9 Gram–Schmidt moment basis and MRT forcing.

Mohamad (2019) §8.8.2 (*Developing Flow in a Two-Dimensional Channel*) for the channel-flow benchmark setup.

Lallemand, P. & Luo, L.-S. (2000) "Theory of the lattice Boltzmann method: Dispersion, dissipation, isotropy, Galilean invariance, and stability." *Phys. Rev. E*, **61**, 6546–6562.

d'Humières, D., Ginzburg, I., Krafczyk, M., Lallemand, P. & Luo, L.-S. (2002) "Multiple-relaxation-time lattice Boltzmann models in three dimensions." *Phil. Trans. R. Soc. Lond. A*, **360**, 437–451.

Guo, Z., Zheng, C. & Shi, B. (2002) "Discrete lattice effects on the forcing term in the lattice Boltzmann method." *Phys. Rev. E*, **65**, 046308.

Ginzburg, I. (2008) "Two-relaxation-time lattice Boltzmann scheme: About parametrization, velocity, pressure and mixed boundary conditions." *Commun. Comput. Phys.*, **3**, 519–581.

CHAPTER 8

Rayleigh-Bénard Convection

Rayleigh–Bénard convection is the canonical thermal-convection benchmark: a horizontal fluid layer heated from below and cooled from above, with no externally imposed flow. Below a critical Rayleigh number the fluid remains at rest and heat is transported purely by conduction from the bottom wall to the top wall; above the critical threshold, buoyancy overcomes viscous damping and thermal diffusion, and the layer organises into counter-rotating convection rolls. For rigid (no-slip) isothermal top and bottom boundaries, linear stability theory places the onset at $\mathrm{Ra}_c \approx 1707.8$ with critical wavenumber $k_c \approx 3.117/H$ (Krüger et al. (2017) §8.4.3).

This chapter introduces **multi-lattice simulations** — two coupled lattices in a single GeLB program, one for fluid momentum and one for temperature, exchanging fields every time step through the Boussinesq approximation.

> **Note**
>
> The example walked here is `examples/lbm/2d/rayleigh_benard_2d_mrt.gelb`, which couples a D2Q9 incompressible MRT fluid solver with a D2Q5 MRT thermal solver (Wang et al. (2013); Yoshida & Nagaoka (2010)) and is paired with a snapshot regression test (`gelb/tests/integration/test_rayleigh_benard_mrt_e2e.py`).

8.1 The physics

In the **Boussinesq approximation** (Krüger et al. (2017) §8.4.1; Mohamad (2019) §9.6), material properties are treated as constant everywhere except in the gravitational body force, where density variations are linearised with respect to temperature around a reference state (ρ_0, T_0). The coupled incompressible Navier–Stokes and advection–diffusion equations read:

$$\begin{aligned} \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} &= -\frac{1}{\rho_0}\nabla p + \nu\nabla^2\mathbf{u} + \alpha g\,(T - T_0)\,\hat{\mathbf{e}}_y\,, \\ \frac{\partial T}{\partial t} + (\mathbf{u}\cdot\nabla)T &= \kappa\nabla^2 T\,, \\ \nabla\cdot\mathbf{u} &= 0\,, \end{aligned}$$

where α is the thermal expansion coefficient, g the gravitational acceleration, ν the kinematic viscosity, and κ the thermal diffusivity. The buoyancy term $\alpha g(T - T_0)\hat{\mathbf{e}}_y$ drives warm fluid upward and cool fluid downward.

With channel height $H = N_y - 2$ (between the two no-slip planes at $y = 0.5$ and $y = N_y - 1.5$: full-way bounce-back, the wall midway between the nodes) and imposed wall temperature difference $\Delta T = T_{\text{hot}} - T_{\text{cold}}$, the dynamics are governed by the **Rayleigh number** and **Prandtl number**:

$$\text{Ra} = \frac{\alpha g \Delta T H^3}{\nu \kappa}, \qquad \text{Pr} = \frac{\nu}{\kappa}.$$

For $\text{Ra} > \text{Ra}_c \approx 1708$ and $\text{Pr} = 0.71$ (air), small perturbations grow into steady two-dimensional convection rolls. The example runs at $\text{Ra} = 10\,000$, well inside the steady supercritical regime. Characteristic buoyancy velocity $U_f = \sqrt{\alpha g \Delta T H}$ is mapped to lattice units via the convective Mach parameter `MAX_MACH = 0.1` ($U_f = \text{Ma}\, c_s = \text{Ma}/\sqrt{3}$), which sets

$$\alpha g \Delta T = \frac{\text{Ma}^2}{3H}, \qquad \nu = H\,\text{Ma}\sqrt{\frac{\text{Pr}}{3\,\text{Ra}}}, \qquad \kappa = \frac{H\,\text{Ma}}{\sqrt{3\,\text{Ra}\,\text{Pr}}}.$$

8.2 The LBM strategy

The standard double-distribution LBM for Boussinesq thermal flows (Krüger et al. (2017) §8.4.2; Mohamad (2019) §9.6) uses **two lattices**:

1. A **D2Q9 fluid lattice** `f` carrying the pressure/density fluctuation $\delta\rho$ and velocity $\mathbf{u}$.
2. A **D2Q5 temperature lattice** `g` carrying the temperature T.

The two lattices interact locally inside the interior `dynamic` block:

- **Temperature to fluid (buoyancy):** the fluid collision step reads the local temperature $T = \sum_i g_i^{\text{in}}$ and applies the vertical buoyancy impulse $F_y = \alpha g T$ to the vertical momentum moment j_y via Strang splitting, just as *Poiseuille with Body Force* applied the streamwise body force to j_x.
- **Fluid to temperature (advection):** the temperature collision step uses the half-step fluid velocity $\mathbf{u} = (u_x, u_y)$ to evaluate the advective thermal equilibrium moments $(T,\, Tu_x,\, Tu_y,\, aT,\, 0)^{\mathsf{T}}$.

Both lattices use MRT collision (Wang et al. (2013); Yoshida & Nagaoka (2010)):

- The fluid uses the **incompressible D2Q9 MRT** formulation (He & Luo (1997); Lallemand & Luo (2000)), in which the momentum moments equal the velocity components directly ($j_x^{\text{eq}} = u_x$, $j_y^{\text{eq}} = u_y$) and the stress equilibria are $p_{xx}^{\text{eq}} = u_x^2 - u_y^2$ and $p_{xy}^{\text{eq}} = u_x u_y$ without factors of ρ.
- The temperature lattice uses a **D2Q5 MRT** operator with transformation matrix N mapping $\mathbf{g}$ to $(T,\, j_{Tx},\, j_{Ty},\, e_T,\, \epsilon_T)^{\mathsf{T}}$, relaxation rates $\sigma_k = 3 - \sqrt{3}$ and $\sigma_{\nu e} = 2(2\sqrt{3} - 3)$, and equilibrium parameter $a = 60\kappa/\sqrt{3} - 4$. At the horizontal walls, the fluid uses bounce-back and the temperature uses the Yoshida & Nagaoka (2010) **anti-bounce-back** rule with Dirichlet correction $2\sqrt{3}\kappa T_{\text{wall}}$ (since the cardinal D2Q5 equilibrium weight in this calibration is $w_{1..4} = (4 + a)/20 = \sqrt{3}\kappa$, this matches the standard anti-bounce-back term $2w_i T_{\text{wall}}$ of Krüger et al. (2017) §8.5.2.1, Eq. 8.54; see also *1D Diffusion* and *Boundary Conditions Cookbook*).

8.3 Declaring two lattices

`rayleigh_benard_2d_mrt.gelb` declares both lattices in a single `lattices` block:

```
lattices {
    lattice f(2, 9, real, {NX, NY}, {true, false});   // fluid (D2Q9)
    lattice g(2, 5, real, {NX, NY}, {true, false});   // temperature (D2Q5)
}
```

Important

Rule: All lattices in a GeLB program share the same grid dimensions (here `{NX, NY}`) and periodicity flags (here `{true, false}`), while each lattice defines its own velocity-set size Q and maintains its own independent `.in` and `.out` buffers. Every `initializer` and `dynamic` block must assign all Q outgoing populations of **every** declared lattice.

Because each lattice has its own ping-pong buffers, a `dynamic` block can freely read `f.in[...]` and `g.in[...]` and write both `f.out[...]` and `g.out[...]` in the same time step without race conditions or manual staging passes.

8.4 Equilibrium functions and the coupled MRT dynamic block

The moment-space equilibria for the two lattices are defined in two short `function` blocks:

```
function FluidEqMoments(real dRho, real uX, real uY) -> (array[real, 9] meq) {
    real uX2  = uX * uX;
    real uY2  = uY * uY;
    real e_eq = -2.0 * dRho + 3.0 * (uX2 + uY2);
    meq = { dRho,            // ρ
            uX,              // jx
            uY,              // jy
            e_eq,            // e
            uX2 - uY2,       // pxx
            uX * uY,         // pxy
            -1.0 * uX,       // qx
            -1.0 * uY,       // qy
            -1.0 * e_eq - dRho };  // ε
}

function TempEqMoments(real T, real uX, real uY) -> (array[real, 5] neq) {
    neq = {T, T * uX, T * uY, A_PARAM * T, 0.0};
}
```

Note that `rayleigh_benard_2d_mrt.gelb` orders the nine fluid moments as $(\delta\rho,\ j_x,\ j_y,\ e,\ p_{xx},\ p_{xy},\ q_x,\ q_y,\ \epsilon)$, placing the vertical momentum j_y at index 2. With those two helpers in place, the entire coupled interior update fits in a single `dynamic` block:

```
dynamic BulkDynamic {
    // Pull-stream both lattices.
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    array[real, 5] gIn = upstream_dfs(g, x, y, {EX5, EY5});

    // Moment transforms.
    array[real, 9] fluidMoms = matmul(M, fIn);
    array[real, 5] tempMoms  = matmul(N, gIn);

    // Strang splitting: 1st half of buoyancy force into the jy moment (index␣
↪2).
    fluidMoms = fluidMoms +
        {0.0, 0.0, 0.5 * ALPHA_G * tempMoms[0],
         0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
```

(continues on next page)

(continued from previous page)

```
    // Read macroscopic moments AFTER the half-force.
    real dRho = fluidMoms[0];
    real uX   = fluidMoms[1];
    real uY   = fluidMoms[2];
    real T    = tempMoms[0];

    // Equilibrium moments.
    array[real, 9] fluidMeq = FluidEqMoments(dRho, uX, uY);
    array[real, 5] tempMeq  = TempEqMoments(T, uX, uY);

    // MRT collision in moment space.
    fluidMoms = fluidMoms - S_FLUID * (fluidMoms - fluidMeq);
    tempMoms  = tempMoms  - S_TEMP  * (tempMoms  - tempMeq);

    // Strang splitting: 2nd half of buoyancy force.
    fluidMoms = fluidMoms +
        {0.0, 0.0, 0.5 * ALPHA_G * tempMoms[0],
         0.0, 0.0, 0.0, 0.0, 0.0, 0.0};

    // Back-transform to distribution space.
    f.out[x, y | :] = matmul(M_INV, fluidMoms);
    g.out[x, y | :] = matmul(N_INV, tempMoms);
}
```

Compare this with `MRTCollStream` in *Poiseuille with Body Force*:

1. **Two pull-streaming calls:** `upstream_dfs` is called once on `f` with its 9-velocity stencil `{EX, EY}` and once on `g` with its 5-velocity stencil `{EX5, EY5}` (rest plus the four cardinal directions).
2. **Two moment transforms:** `matmul(M, fIn)` maps `fIn` into the 9-vector `fluidMoms`, and `matmul(N, gIn)` maps `gIn` into the 5-vector `tempMoms` (whose zeroth entry `tempMoms[0]` is the local temperature $T = \sum_{i=0}^{4} g_i^{\text{in}}$).
3. **Two-way Boussinesq coupling:** before relaxation, half of the buoyancy impulse `0.5 * ALPHA_G * tempMoms[0]` is added to `fluidMoms[2]` (j_y). The updated velocity components `uX = fluidMoms[1]` and `uY = fluidMoms[2]` then feed both `FluidEqMoments(dRho, uX, uY)` and `TempEqMoments(T, uX, uY)`.
4. **Independent MRT relaxation and back-transform:** each lattice relaxes with its own rate vector (`S_FLUID` and `S_TEMP`), the second half of the buoyancy impulse is added to `fluidMoms[2]`, and `M_INV` and `N_INV` transform both moment vectors back to `f.out` and `g.out`.

8.5 The geometry and wall boundary conditions

```
node_classes {
    node_class FLUID(InitBulk, BulkDynamic, RBOutput);
    node_class WALL_BOT(InitWallBot, WallBotDynamic) masked;
    node_class WALL_TOP(InitWallTop, WallTopDynamic) masked;
}

geometry {
    nodes[:, :]      = FLUID;
```

(continues on next page)

(continued from previous page)

```
    nodes[:, 0]       = WALL_BOT;
    nodes[:, NY - 1] = WALL_TOP;
}
```

Three node classes partition the domain: `FLUID` in the interior, `WALL_BOT` at `y = 0` (held at `T_HOT = 1.0`), and `WALL_TOP` at `y = NY - 1` (held at `T_COLD = 0.0`). The two wall classes are marked `masked` so their boundary dynamics execute every time step while excluding the ghost rows `y = 0` and `y = NY - 1` from the saved NetCDF gauge output.

In `WallBotDynamic`, the fluid populations undergo bounce-back exactly as in *Poiseuille with Body Force*, while the northward temperature population (slot 2) is reflected from the first fluid row `y + 1` with the isothermal anti-bounce-back source:

```
g.out[x, y | 0] = 0.0;
g.out[x, y | 1] = 0.0;
g.out[x, y | 2] = -g.in[x, y + 1 | 4]
                  + 2.0 * sqrt(3.0) * DIFFUSIVITY * T_HOT;
g.out[x, y | 3] = 0.0;
g.out[x, y | 4] = 0.0;
```

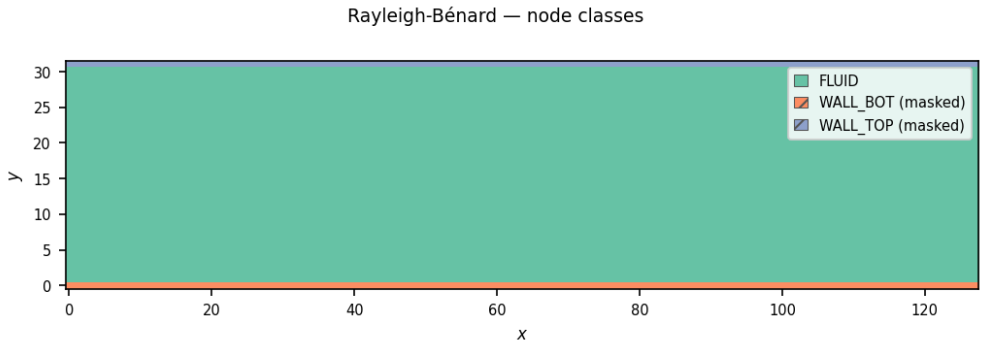


Fig. 1: Node-class layout for `rayleigh_benard_2d_mrt.gelb`: `FLUID` (teal) fills the interior rows `y = 1 .. NY - 2`, while `WALL_BOT` (orange, hot) and `WALL_TOP` (blue, cold) are single-row masked boundary classes at `y = 0` and `y = NY - 1`.

Note

In a multi-lattice program, coverage checks run independently on every declared lattice. If a wall block omits writes to the temperature lattice `g`, the compiler reports specifically which lattice and DF indices are missing:

```
[Problem] Dynamic block 'WallBotDynamic' (for node class
'WALL_BOT') does not write all DFs of lattice 'g':
missing DFs [0, 1, 2, 3, 4]

[Hint] Add write statements for the missing distribution
functions, or use the full-range syntax g.out[...|:] to
write all DFs at once.
```

8.6 Running it

Because the instability is seeded with a small random perturbation on top of the conductive linear temperature profile, pass `--seed` for a reproducible roll pattern:

```
gelb-run --seed 42 -n 30000 -w 30000 \
    examples/lbm/2d/rayleigh_benard_2d_mrt.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* and *Write every* to 30000 and *Seed* to 42; set *Write every* to 500 to watch the rolls form.

At the default resolution (`NY = 32`, `NX = 128`), 30 000 steps are sufficient for the initial perturbation to grow and saturate into steady convection rolls at Ra $= 10\,000$. With the default `cpu_highway` backend, the run finishes in about 15 seconds on a modern multicore CPU.

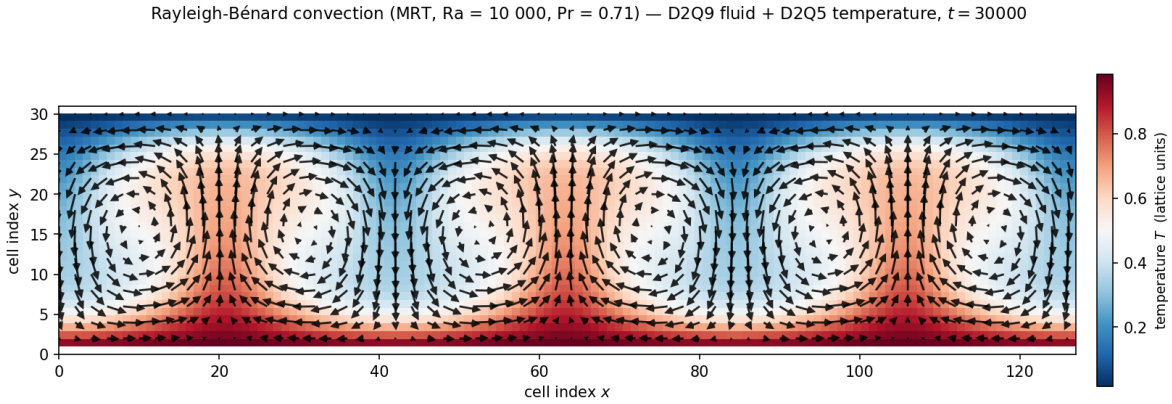


Fig. 2: Steady-state temperature field T (colour map: red hot, blue cold) and velocity vectors (black arrows) at $t = 30\,000$ for Ra $= 10\,000$, Pr $= 0.71$. The periodic domain of aspect ratio $\Gamma = N_x/(N_y - 2) = 128/30 \approx 4.27$ supports three pairs of counter-rotating convection rolls (six rolls in total): three warm upwelling plumes rise near $x \approx 20, 63, 106$ while three cool downwelling plumes descend near $x \approx 0, 42, 85$.

8.7 Validation: the snapshot regression test

Analytical solutions exist for the linear stability threshold Ra_c, but the finite-amplitude post-critical roll state is governed by the full nonlinear coupled equations. To guard against numerical regressions, GeLB pairs this example with an end-to-end snapshot test (`gelb/tests/integration/test_rayleigh_benard_mrt_e2e.py`) that compiles

and runs the model and compares the resulting temperature and velocity fields against a reference NetCDF snapshot (`gelb/tests/data/rb_mrt_snapshot.nc`) to a tolerance of $\sim 10^{-10}$. Whereas *Poiseuille with Body Force* illustrated validation against a closed-form analytical profile, snapshot regression provides automated verification for nonlinear multi-physics benchmarks once a reference solution has been established.

8.8 What's next

Multi-lattice coupling completes the core set of GeLB language constructs. The remaining chapters of the guided tour combine the features you have already seen on new physical regimes: *Kármán Vortex Street* uses `border()` and coordinate-expression paints to place a circular cylinder in an open channel flow; *Shan-Chen Multiphase* computes non-local pseudopotential forces via `upstream_dfs` on an auxiliary lattice; and *Beyond LBM: Conway's Game of Life* applies the same stencil primitives to a cellular automaton outside fluid dynamics.

See also

Krüger et al. (2017) §8.4 (*Thermal Flows*) for the Boussinesq approximation (§8.4.1), the double-distribution LBM formulation (§8.4.2), and the Rayleigh–Bénard benchmark (§8.4.3); §8.2.2 and §8.6.2 for D2Q5 advection–diffusion models; and §8.5.2.1 (Eq. 8.54) for Dirichlet anti-bounce-back boundary conditions.

Mohamad (2019) Chapter 9 (*Nonisothermal Incompressible Fluid Flow*), especially §9.6 (*Natural Convection*), for worked two-lattice thermal convection examples.

Wang, J., Wang, D., Lallemand, P. & Luo, L.-S. (2013) "Lattice Boltzmann simulations of thermal convective flows in two dimensions." *Computers & Mathematics with Applications*, **65**, 262–286.

Yoshida, H. & Nagaoka, M. (2010) "Multiple-relaxation-time lattice Boltzmann model for the convection and anisotropic diffusion equation." *Journal of Computational Physics*, **229**, 7774–7795.

CHAPTER 9

Kármán Vortex Street

Flow past a circular cylinder is a classic unsteady fluid-dynamics benchmark: above a critical Reynolds number ($\mathrm{Re}_c \approx 47$) the steady, symmetric recirculation bubble behind the cylinder loses stability and transitions into a staggered train of alternating counter-rotating vortices shed downstream — the **Kármán vortex street**. For $\mathrm{Re} \in [100, 200]$ the shedding is laminar and periodic with dimensionless Strouhal frequency $\mathrm{St} = fD/U \approx 0.16$--$0.17$.

This chapter walks a setup adapted from Sukop & Thorne (2006) §5.3 and introduces three pieces of GeLB geometry machinery: **conditional painting** via a user-defined Boolean predicate function, the `border()` statement for automatic boundary-layer assignment around solid obstacles, and the `masked` modifier for excluding solid interiors from gauge output.

> **Note**
>
> The example walked here is `examples/sukop/karman_vortex_street_2d_bgk.gelb`. Its header is tagged `STATUS: EXPERIMENTAL`: in a periodic channel driven by a body force, the flow takes its viscous time to spin up, and the wake that leaves the domain re-enters it upstream, so the shedding it shows is a demonstration, not a benchmark. For a cylinder with an inflow and an outflow (Zou–He velocity boundaries), see `examples/softwarex-2026-paper/flow_past_cylinder_2d_bgk_zouhe.gelb`.

9.1 The physics

A circular cylinder of diameter $D = 2R$ is placed in a stream of characteristic velocity U and kinematic viscosity ν. The wake regime is governed by the cylinder Reynolds number $\mathrm{Re} = UD/\nu$ (Mohamad (2019) §8.8.3):

- $\mathrm{Re} \lesssim 1$: creeping (Stokes) flow, fore–aft symmetric with no separation.
- $\mathrm{Re} \sim 5$--47: steady separated flow with a closed pair of symmetric counter-rotating recirculation vortices attached to the rear of the cylinder.
- $\mathrm{Re} \sim 47$--200: laminar periodic vortex shedding (the two-dimensional Kármán vortex street), with vortices of alternating sign detaching from the upper and lower sides of the cylinder at Strouhal number $\mathrm{St} = fD/U \approx 0.16$--$0.18$.
- $\mathrm{Re} \gtrsim 200$: onset of three-dimensional wake instabilities and transition toward a turbulent wake.

The example's defaults aim at Re ≈ 100 once the channel has spun up, while keeping the lattice Mach number Ma $= U/c_s < 0.1$ (Krüger et al. (2017) §7.2.1). *Running it* below shows how far a run gets towards that.

9.2 The GeLB strategy

Following Sukop & Thorne (2006) §5.3, the flow is driven through a streamwise-periodic channel by a uniform body force G rather than imposed inlet and outlet profiles. The top and bottom channel walls use full-way bounce-back (the wall midway between the nodes); the circular cylinder is painted as `SOLID` via a user-defined predicate function `InCircle(x, y, CX, CY, R)`, and its one-cell bounce-back perimeter is generated automatically by `border(SOLID) = WALL`. The cylinder centre sits one lattice unit off the channel midline (`CY = NY / 2 - 1`), a small, deterministic asymmetry for the wake instability to grow from, without random noise.

9.3 Conditional paint and the `InCircle` predicate

The disk is defined by a pure Boolean helper function:

```
function InCircle(int x, int y, int cx, int cy, int r) -> (bool hit) {
    hit = (x - cx) * (x - cx) + (y - cy) * (y - cy) < r * r;
}
```

`InCircle` takes five integer arguments — a cell coordinate `(x, y)`, a disk centre `(cx, cy)`, and a radius `r` — and returns `true` whenever $(x - c_x)^2 + (y - c_y)^2 < r^2$. It is then called inside a `where` clause in the `geometry` block:

```
geometry {
    nodes[:, :]      = FLUID;
    nodes[:, 0]      = WALL;
    nodes[:, NY - 1] = WALL;
    nodes[where InCircle(x, y, CX, CY, R)] = SOLID;
    border(SOLID) = WALL;
}
```

In `nodes[where InCircle(x, y, CX, CY, R)] = SOLID;`, `where` introduces a Boolean predicate in which `x` and `y` (and `z` in 3D) represent the coordinates of the candidate cell. During compilation, GeLB evaluates the predicate at every grid point and assigns `SOLID` wherever it evaluates to `true`.

The same pattern extends to multiple obstacles or arbitrary analytic shapes:

```
nodes[where InCircle(x, y, CX1, CY1, R1)] = SOLID;
nodes[where InCircle(x, y, CX2, CY2, R2)] = SOLID;
```

Any shape expressible as a pure Boolean function of coordinates and constants — squares, ellipses, or lookup masks stored in a compile-time `array[bool, {NX, NY}]` — can be painted in the same way.

9.4 `border()` — automatic wall-layer inference

```
border(SOLID) = WALL;
```

This statement instructs the compiler: after all `nodes[...]` paint statements have been applied, inspect every cell currently classified as `SOLID`; if at least one of its neighbours is *not* `SOLID`, reclassify that cell as `WALL`. The neighbours are all the cells one step away along an axis or a diagonal (8 in 2D, 26 in 3D); a periodic axis wraps around its seam, and a non-periodic edge has no neighbour beyond it. Because the rule reads a single snapshot of the node map and applies all replacements simultaneously, the resulting one-cell `WALL` shell is independent of traversal order.

The diagonals matter on D2Q9. A fluid cell on one of the circle's diagonal steps touches the disk only along a diagonal link; if the cell behind that link stayed `SOLID`, an empty class that holds zero populations, the fluid cell would pull zero from it on every step and the flow would lose mass there. Because `border()` inspects the diagonals, every disk cell that a fluid cell reaches along any of the nine links is a wall.

Without `border()`, you would have to construct the rasterised staircase perimeter of the disk manually. With the border rule, the domain is partitioned cleanly into three regions:

- `FLUID` cells in the channel interior (running BGK collision, forcing, and pull-streaming).
- `WALL` cells along the top and bottom channel walls and in the staircase ring of the disk (running bounce-back).
- `SOLID` cells in the strict interior of the disk (inactive and masked).

9.5 The `masked` modifier and empty node classes

The three node classes are declared as follows:

```
node_classes {
    node_class FLUID(InitFluid, FluidDynamic, Hydro);
    node_class WALL (InitFluid, BounceBack,   Hydro);
    node_class SOLID() masked;
}
```

Important

Rules for geometry predicates, `border()`, and `masked` classes:

- **Predicate paints** (`nodes[where expr] = CLASS;`) may use cell coordinates (`x`, `y`, `z`), global constants, scalar math intrinsics, and pure single-output functions returning `bool`. They may not read lattice populations or the runtime step counter `t` because geometry is resolved at compile time.
- **Border rules** (`border(SOURCE) = TARGET;`) run in their own source order *after* all `nodes[...]` paint statements, reclassifying any `SOURCE` cell that has at least one non-`SOURCE` neighbour along an axis or a diagonal.
- **Empty node classes** (`node_class SOLID();`) execute no initialiser, dynamic, or gauge; all distribution functions at their cells are guaranteed to hold **zero**.
- The `masked` modifier excludes a node class from NetCDF gauge output (its cells are written as `NaN`, which `gelb-view` renders as transparent) and forbids attaching a gauge to that class. A `masked` class that specifies an initialiser and dynamic still executes them on every step (as used for wall rows in *Rayleigh-Bénard Convection* and *Reduced-Gravity Ocean Circulation*).

Combining an empty parameter list `()` with `masked` means the interior `SOLID` cells of the cylinder require no initialiser or dynamic block, hold zero populations in memory, are skipped during the time-stepping loop, and appear automatically masked as `NaN` in the output file.

9.6 The program

```
global_constants {
    const int NX = 1200;
    const int NY = 200;
    const real TAU   = 0.56;              // ν = 0.02
```

(continues on next page)

(continued from previous page)

```
    const real OMEGA = 1.0 / TAU;
    const real G = 2.0e-7;                // body force

    const int CX = NX / 4;
    const int CY = NY / 2 - 1;            // slightly off-centre to break␣
↪symmetry
    const int R  = 20;                    // disk radius (D = 40)

    const array[real, 9] EX = {0.0, 1.0, 0.0, -1.0,  0.0, 1.0, -1.0, -1.0, ␣
↪1.0};
    const array[real, 9] EY = {0.0, 0.0, 1.0,  0.0, -1.0, 1.0,  1.0, -1.0, -
↪1.0};
    const array[real, 9] W  = {4.0/9.0,
        1.0/9.0, 1.0/9.0, 1.0/9.0, 1.0/9.0,
        1.0/36.0, 1.0/36.0, 1.0/36.0, 1.0/36.0};
}

lattices { lattice f(2, 9, real, {NX, NY}, {true, false}); }

function InCircle(int x, int y, int cx, int cy, int r) -> (bool hit) {
    hit = (x - cx) * (x - cx) + (y - cy) * (y - cy) < r * r;
}

function Macro(array[real, 9] fIn) -> (real rho, real ux, real uy) {
    rho = sum(fIn);
    ux  = dot_prod(EX, fIn) / rho;
    uy  = dot_prod(EY, fIn) / rho;
}

function Feq(real rho, real ux, real uy) -> (array[real, 9] feq) {
    real usq = ux * ux + uy * uy;
    array[real, 9] cu = EX * ux + EY * uy;
    feq = W * rho * (1.0 + 3.0 * cu + 4.5 * cu * cu - 1.5 * usq);
}

initializer InitFluid { f.out[x, y | :] = W; }

// Shifted-velocity body force
dynamic FluidDynamic {
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    real rho; real ux; real uy;
    {rho, ux, uy} = Macro(fIn);
    ux = ux + 0.5 * G;        // velocity shift
    f.out[x, y | :] = fIn - OMEGA * (fIn - Feq(rho, ux, uy));
}

dynamic BounceBack {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY});
}

gauge Hydro -> (real rho, real ux, real uy) {
    {rho, ux, uy} = Macro(f.in[x, y | :]);
```

(continues on next page)

(continued from previous page)

```
}

node_classes {
    node_class FLUID(InitFluid, FluidDynamic, Hydro);
    node_class WALL (InitFluid, BounceBack,   Hydro);
    node_class SOLID() masked;
}

geometry {
    nodes[:, :]      = FLUID;
    nodes[:, 0]      = WALL;
    nodes[:, NY - 1] = WALL;
    nodes[where InCircle(x, y, CX, CY, R)] = SOLID;
    border(SOLID) = WALL;
}
```

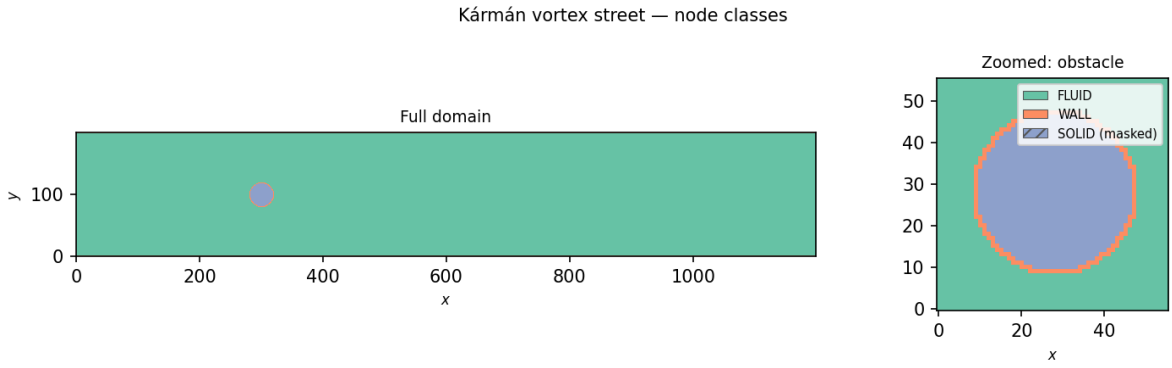


Fig. 1: Node-class layout: `FLUID` (teal), `WALL` (orange, top/bottom channel edges and the `border()` ring around the disk), and `SOLID` (blue, disk interior — masked, excluded from gauge output).

All collision and streaming primitives here are familiar from *1D Diffusion* and *Lid-Driven Cavity*. In `FluidDynamic`, the body force is introduced via an equilibrium velocity shift `ux = ux + 0.5 * G` before evaluating `Feq(rho, ux, uy)` (compare the equilibrium-velocity forcing schemes of Krüger et al. (2017) §6.4.2, eq. (6.29): because BGK relaxation multiplies f^{eq} by $\omega = 1/\tau$, shifting the equilibrium velocity by $\Delta u_x = \frac{1}{2}G$ injects an effective streamwise momentum increment of $\omega\rho\Delta u_x = \rho G/(2\tau)$ per step, which at $\tau = 0.56$ is $\approx 0.89\,\rho G$). For qualitative wake visualisation at low Mach number this simple shift is sufficient; for second-order forcing accuracy, use the Strang-split or Guo schemes of *Poiseuille with Body Force*.

9.7 Running it

```
gelb-run -n 250000 -w 500 \
    examples/sukop/karman_vortex_street_2d_bgk.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* to 250000 and *Write every* to 500.

Running 250 000 steps on the 1200×200 grid with the default `cpu_highway` backend takes roughly five minutes on a modern multicore laptop, and writing every 500 steps gives 501 records for animating the wake in `gelb-view`. The channel starts at rest, and the body force spins it up over its viscous time $H^2/\nu \approx 2 \times 10^6$ steps ($H = 198$), so at the end of this run the centreline speed is 0.019, a Reynolds number of about 37 on the centreline speed instead of the 100 the defaults aim at. That is below $\mathrm{Re}_c \approx 47$: the wake is a steady, symmetric pair of shear layers behind the disk,

and nothing is shed yet.

A wider channel driven twice as hard gets there within the run:

```
gelb-run -b gpu_cuda_cpp -n 300000 -w 2000 \
    -D NY=256 -D G=4.0e-7 \
    examples/sukop/karman_vortex_street_2d_bgk.gelb
```

(about 80 s on `gpu_cuda_cpp`). The centreline Reynolds number reaches about 75 around $t = 160\,000$; the symmetric wake then turns unstable, its cross-stream velocity on the axis oscillating and growing from 10^{-5}, and from about $t = 250\,000$ the disk sheds vortices alternately from its two sides. The period is about 6200 steps, a Strouhal number of about 0.14 on the centreline speed at Re ≈ 90, and the channel is still speeding up. Because the channel is periodic in x, the street that leaves on the right comes back in on the left.

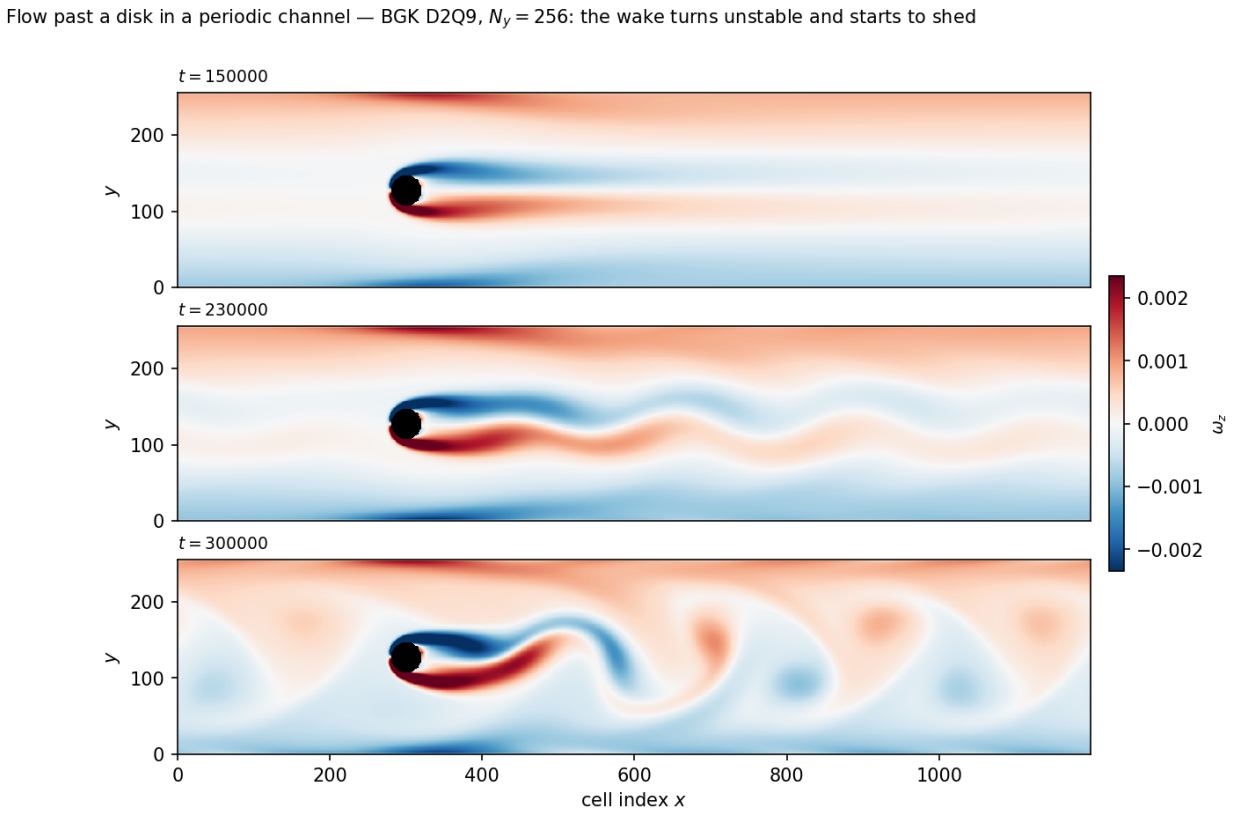


Fig. 2: Spanwise vorticity $\omega_z = \partial_x u_y - \partial_y u_x$ from the second command (a 1200×256 channel, `G = 4.0e-7`, a disk of radius $R = 20$, $D = 40$) at $t = 150\,000$, $230\,000$ and $300\,000$. Red denotes positive (counter-clockwise) vorticity and blue negative (clockwise) vorticity; the colour along the top and bottom is the channel's own shear. At $t = 150\,000$ the wake is steady and symmetric; at $230\,000$ it oscillates; at $300\,000$ the disk sheds a vortex street, which re-enters the periodic channel from the left.

9.8 Summary

Using a pure coordinate predicate (`InCircle`), automatic boundary peeling (`border(SOLID) = WALL`), and `masked` empty interior nodes (`node_class SOLID() masked;`), an obstacle flow on a Cartesian grid is specified in a few declarative lines without manual index lists or runtime branching inside the fluid kernel. As introduced in *GeLB in 10 Minutes*, resolving the geometry at compile time lets the generated solver iterate over each node class cleanly while omitting inactive interior obstacle cells from both computation and NetCDF output.

9.9 What's next

Shan-Chen Multiphase introduces single-component multiphase flow via the Shan–Chen pseudopotential method, where phase separation arises from a non-local interaction force computed across neighbouring cells using `upstream_dfs` on an auxiliary scalar lattice.

See also

Krüger et al. (2017) §5.3.3 (*Solid Boundaries: Bounce-Back*) for staircase bounce-back on curved obstacles; §6.4.2 (*Forcing Schemes*; Shan and Chen's velocity shift in §6.4.2.2) for velocity-shift forcing; §7.2.1 for Mach-number constraints in weakly compressible BGK; and Chapter 11 (*Curved and Moving Boundaries*) for higher-order interpolated bounce-back schemes.

Mohamad (2019) §8.8.3 (*Flow over Obstacles*) for cylinder-wake benchmarks and vortex-shedding regimes.

Sukop, M. C. & Thorne, D. T. (2006) *Lattice Boltzmann Modeling: An Introduction for Geoscientists and Engineers* (Springer), §5.3 and Fig. 10.

CHAPTER 10

Shan-Chen Multiphase

This chapter walks through the **Shan–Chen single-component multiphase** (SCMP) model (Shan & Chen (1993, 1994); Krüger et al. (2017) §9.3; Mohamad (2019) §11.1). By adding a short-range attractive interaction force between neighbouring lattice nodes, a single BGK fluid spontaneously separates into coexisting liquid and vapour phases with diffuse interfaces and emergent surface tension, without explicit interface tracking.

From a GeLB perspective, the new ingredient is a **non-local body force** that depends on the density at all eight neighbouring cells. This is written directly via offset slice reads `sum(f.in[x + dx, y + dy | :])` (or, equivalently, via a dedicated 1-DF auxiliary lattice as in `examples/lbm/2d/shan_chen_multiphase_2d_bgk.gelb`).

> **Note**
>
> The example walked here is `examples/sukop/shan_chen_2d.gelb`, adapted from Sukop & Thorne (2006) §6.3. Its header is tagged `STATUS: EXPERIMENTAL` because the coexistence densities in the original single-relaxation-time Shan–Chen scheme depend on the pseudopotential functional form and relaxation time τ; see `examples/lbm/2d/shan_chen_multiphase_2d_bgk.gelb` for the companion two-lattice formulation using the bounded exponential pseudopotential of Krüger et al. (2017) §9.3.1, Eq. (9.102).

10.1 The physics

In an isothermal ideal lattice gas, the bulk pressure is $p_b = c_s^2 \rho$, which is monotonically increasing with ρ and cannot support two coexisting phases. To model a non-ideal fluid whose molecules attract at short range, Shan & Chen (1993) introduced a nearest-neighbour interaction force (Krüger et al. (2017) §9.3.1, Eq. 9.105):

$$\mathbf{F}_{\mathrm{SC}}(\mathbf{x}) = -G\,\psi\big(\rho(\mathbf{x})\big) \sum_i w_i\,\psi\big(\rho(\mathbf{x} + \mathbf{c}_i \Delta t)\big)\,\mathbf{c}_i \Delta t\,,$$

where $\psi(\rho)$ is the **pseudopotential** (or effective density), $G < 0$ is the interaction strength (negative for attraction), and w_i are the standard D2Q9 weights. A Taylor expansion of $\psi(\mathbf{x} + \mathbf{c}_i \Delta t)$ shows that $\mathbf{F}_{\mathrm{SC}}$ modifies the bulk equation of state to (Krüger et al. (2017) §9.3.2.1, Eq. 9.111)

$$p_b(\rho) = c_s^2 \rho + \frac{c_s^2 \Delta t^2 G}{2}\,\psi^2(\rho)\,,$$

while its higher-order gradient terms supply an isotropic surface tension across diffuse phase interfaces.

In `examples/sukop/shan_chen_2d.gelb`, the pseudopotential has the form used by Sukop & Thorne (2006) §6.3:

$$\psi(\rho) = \psi_0 \exp(-\rho_0/\rho)\,,$$

with $\psi_0 = 4$, $\rho_0 = 200$, and $G = -120$. Below the critical coupling threshold ($G < G_c < 0$), the equation of state $p_b(\rho)$ develops a van der Waals loop ($\mathrm{d}p_b/\mathrm{d}\rho < 0$ inside the spinodal interval), and a homogeneous fluid at $\rho \approx \rho_0$ is mechanically unstable. Starting from a uniform density with small random perturbations, the system evolves through four characteristic stages:

1. **Random initial state** ($t = 0$): uniform base density $\rho_0 = 200$ plus small white noise.
2. **Spinodal decomposition** ($t \sim 100$): unstable wavelengths grow exponentially into a fine interconnected pattern of dense and dilute regions.
3. **Coarsening** ($t \sim 1000$): filaments pinch off and merge into circular liquid droplets suspended in vapour, driven by surface tension to reduce total interface length.
4. **Late-stage coalescence** ($t \sim 10\,000$): droplets continue to coalesce into a few large drops whose interior and exterior densities approach the planar coexistence values ($\rho_{\mathrm{liq}} \approx 524$, $\rho_{\mathrm{vap}} \approx 80$; see Krüger et al. (2017) §9.1.1 and §9.3.2.2, Eqs. 9.113–9.114).

10.2 How Shan–Chen force injection works

The interaction force $\mathbf{F}_{\mathrm{SC}}$ can be incorporated into the collision step via several forcing schemes — including the Strang-split or Guo schemes of *Poiseuille with Body Force* or the **equilibrium-velocity shift** originally introduced by Shan & Chen (1993) (Krüger et al. (2017) §6.4.2.2, with the equilibrium velocity of eq. (6.29)). In the Shan–Chen equilibrium-velocity shift used by `shan_chen_2d.gelb`, the velocity entering the BGK equilibrium distribution is shifted by $\tau\mathbf{F}_{\mathrm{SC}}/\rho$:

$$\mathbf{u}^{\mathrm{eq}} = \mathbf{u} + \frac{\tau\,\mathbf{F}_{\mathrm{SC}}}{\rho}\,,$$

where $\mathbf{u} = (\sum_i \mathbf{c}_i f_i)/\rho$ is the pre-collision velocity. The BGK collision step then reads

$$f_i^{*}(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{1}{\tau}\big(f_i(\mathbf{x}, t) - f_i^{\mathrm{eq}}(\rho, \mathbf{u}^{\mathrm{eq}})\big)\,.$$

Because the first moment of $f_i^{\mathrm{eq}}(\rho, \mathbf{u}^{\mathrm{eq}})$ is $\rho\mathbf{u} + \tau\mathbf{F}_{\mathrm{SC}}$, multiplying by $1/\tau$ in the BGK relaxation operator cancels τ and increments the momentum density by exactly $\mathbf{F}_{\mathrm{SC}}\Delta t$ per step (with the physical half-step velocity given by $\mathbf{u} + \frac{1}{2}\mathbf{F}_{\mathrm{SC}}\Delta t/\rho$).

10.3 Non-local reads in GeLB

Here is the top of `BulkDynamic` in `examples/sukop/shan_chen_2d.gelb`:

```
dynamic BulkDynamic {
    // --- Nonlocal densities ---
    real rho_C  = sum(f.in[x,     y     | :]);
    real rho_E  = sum(f.in[x + 1, y     | :]);
    real rho_W  = sum(f.in[x - 1, y     | :]);
    real rho_N  = sum(f.in[x,     y + 1 | :]);
    real rho_S  = sum(f.in[x,     y - 1 | :]);
    real rho_NE = sum(f.in[x + 1, y + 1 | :]);
```

(continues on next page)

(continued from previous page)

```
    real rho_NW = sum(f.in[x - 1, y + 1 | :]);
    real rho_SW = sum(f.in[x - 1, y - 1 | :]);
    real rho_SE = sum(f.in[x + 1, y - 1 | :]);

    // --- Pseudopotentials ---
    real psi_C  = Psi(rho_C);
    real psi_E  = Psi(rho_E);
    ...
```

The expression `f.in[x + 1, y + 1 | :]` reads the 9-element distribution vector at neighbour `(x + 1, y + 1)` from the previous time step, and `sum(...)` reduces it to that neighbour's scalar density. Because `f` is declared doubly periodic (`lattice f(2, 9, real, {N, N}, {true, true});`), offsets at domain edges wrap around automatically on every backend.

Important

Rule: Any `dynamic` or `gauge` block that reads a lattice at a non-zero spatial offset `[x + dx, y + dy | ...]` is checked against the geometry map and the lattice periodicity flags at compile time. If a node class using that block is painted on a non-periodic boundary where `x + dx` or `y + dy` would fall outside the grid, `gelbc` rejects the program:

```
[Problem] Out-of-bounds lattice access: block 'BulkDynamic'
(for node class 'BULK') reads/writes lattice 'f' at offset
-1 in x, but 'BULK' is painted at boundary position(s)
where this would access outside the grid

[Hint] Either make the x-dimension periodic, or avoid
painting 'BULK' at the affected boundary, or use a
different block for boundary nodes.
```

10.4 The rest of the dynamic block

Once the nine pseudopotential values `psi_C .. psi_SE` have been evaluated, the weighted discrete gradient $\sum_i w_i \psi(\mathbf{x}+\mathbf{c}_i\Delta t)\mathbf{c}_i$ and interaction force $\mathbf{F}_{\mathrm{SC}}$ are assembled using the D2Q9 cardinal weight $1/9$ and diagonal weight $1/36$:

```
real sx = (1.0 /  9.0) * (psi_E  - psi_W)
        + (1.0 / 36.0) * (psi_NE - psi_NW + psi_SE - psi_SW);
real sy = (1.0 /  9.0) * (psi_N  - psi_S)
        + (1.0 / 36.0) * (psi_NE + psi_NW - psi_SE - psi_SW);

real Fx = -G_SC * psi_C * sx;
real Fy = -G_SC * psi_C * sy;
```

The remainder of `BulkDynamic` pull-streams `f`, computes the pre-collision velocity, shifts the equilibrium velocity by `TAU * F / rho`, and relaxes:

```
// --- Pull-streamed DFs and local velocity ---
array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
real rho = sum(fIn);
```

(continues on next page)

(continued from previous page)

```
real ux  = dot_prod(EX, fIn) / rho;
real uy  = dot_prod(EY, fIn) / rho;

// --- Shan–Chen equilibrium-velocity force injection ---
real ux_eq = ux + TAU * Fx / rho;
real uy_eq = uy + TAU * Fy / rho;

f.out[x, y | :] = fIn - OMEGA * (fIn - Feq(rho, ux_eq, uy_eq));
```

Note that `BulkDynamic` reads `f.in` both in the non-local density sums at the top and via `upstream_dfs` below. Because `f.in` is read-only throughout the step and all writes go to `f.out`, both reads observe the same previous-step state without conflict.

10.5 The initializer: random initial density

```
initializer InitRandom {
    real rho_init = RHO_MEAN + random();
    f.out[x, y | :] = W * rho_init;
}
```

The `random()` intrinsic returns a uniform pseudo-random number in $[0, 1)$, giving each cell an initial density $\rho_{\text{init}} \in [200, 201)$ and initial populations $f_i = w_i \rho_{\text{init}}$ corresponding to a fluid at rest.

Important

Rule: Bind `random()` to a scalar local variable first (as in `real rho_init = RHO_MEAN + random();`) before multiplying by the weight array `W`. Writing `W * (RHO_MEAN + random())` inside an array or slice expression evaluates `random()` independently for each of the Q lattice directions rather than drawing a single scalar density per cell.

Pass `--seed <N>` to `gelb-run` for a deterministic initial perturbation; without `--seed`, every backend seeds the generator from the system clock, so each run produces a statistically independent droplet realisation.

10.6 Running it

```
# Small grid for quick iteration
gelb-run --seed 42 -D N=256 -n 10000 -w 100 \
    examples/sukop/shan_chen_2d.gelb

# Full resolution from Sukop & Thorne (2006) §6.3 (N=1024)
gelb-run --seed 42 -n 10000 -w 100 \
    examples/sukop/shan_chen_2d.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* to 10000, *Write every* to 100, and *Seed* to 42, with `N=256` in *Constants* for the quick grid.

With the default `cpu_highway` backend, the 256×256 grid completes 10 000 steps in roughly 10 seconds on a modern multicore CPU, while the full 1024×1024 domain takes about three minutes.

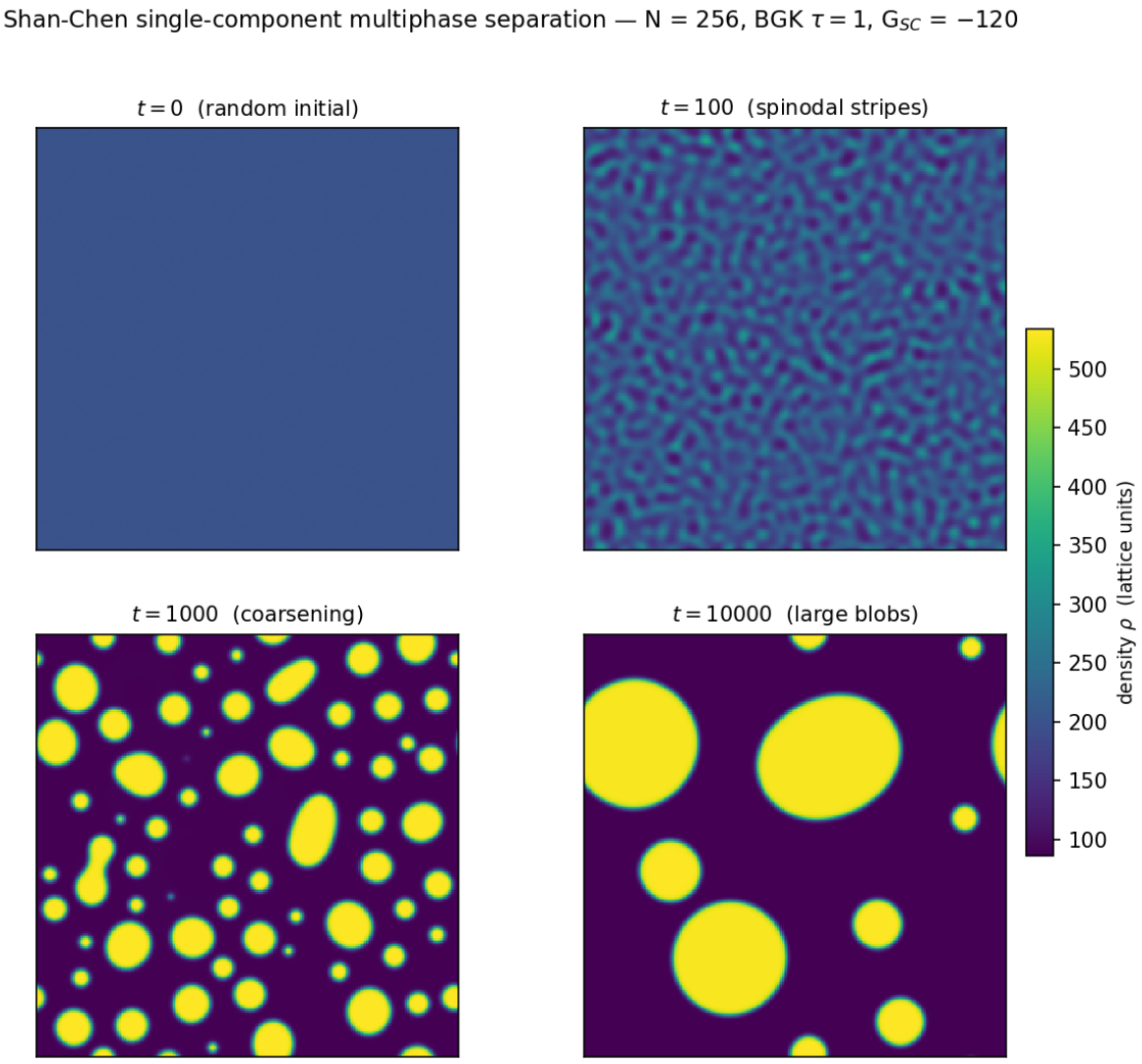


Fig. 1: Four snapshots of the density field $\rho(x, y)$ over 10 000 time steps at $N = 256$, $\tau = 1$, $\psi(\rho) = 4\exp(-200/\rho)$, and $G_{\mathrm{SC}} = -120$. **Top left** ($t = 0$): near-uniform initial density $\rho \in [200, 201)$. **Top right** ($t = 100$): spinodal decomposition amplifies unstable wavelengths into fine interconnected stripes. **Bottom left** ($t = 1000$): coarsening breaks the stripes into circular liquid droplets. **Bottom right** ($t = 10\,000$): coalescence leaves a few large liquid droplets ($\rho_{\mathrm{liq}} \approx 524$) in a low-density vapour background ($\rho_{\mathrm{vap}} \approx 80$).

10.7 Summary

Non-local stencil interactions such as the Shan–Chen pseudopotential force are written directly in GeLB using offset lattice reads `f.in[x + dx, y + dy | :]` (or a coupled 1-DF scalar lattice `psi` as in `examples/lbm/2d/shan_chen_multiphase_2d_bgk.gelb`). The compiler verifies stencil bounds against the declared lattice periodicity and handles periodic wrapping and parallel execution across all supported backends.

10.8 What's next

Beyond LBM: Conway's Game of Life steps outside LBM. It is not a fluid simulation at all — it implements Conway's Game of Life in GeLB to demonstrate that the same lattice, stencil, initializer, dynamic, and gauge primitives apply generally to explicit cellular automata and grid-based stencil updates.

See also

Krüger et al. (2017) §9.1 (*Introduction*) for liquid–gas coexistence, the Maxwell equal-area construction, and Laplace pressure, and §9.3 (*Shan-Chen Pseudopotential Method*) for the pseudopotential force (Eqs. 9.102 and 9.105), the non-ideal equation of state (Eq. 9.111), and mechanical coexistence across a planar interface (§9.3.2.2, Eqs. 9.113–9.114).

Mohamad (2019) §11.1 (*Shan–Chen Potential Method*) for a concise overview of pseudopotential multiphase models.

Sukop, M. C. & Thorne, D. T. (2006) *Lattice Boltzmann Modeling: An Introduction for Geoscientists and Engineers* (Springer), §6.3 and Fig. 46.

Shan, X. & Chen, H. (1993) "Lattice Boltzmann model for simulating flows with multiple phases and components." *Phys. Rev. E*, **47**, 1815–1819.

Shan, X. & Chen, H. (1994) "Simulation of nonideal gases and liquid–gas phase transitions by the lattice Boltzmann equation." *Phys. Rev. E*, **49**, 2941–2948.

CHAPTER 11

Beyond LBM: Conway's Game of Life

This chapter of the guided tour is not an LBM simulation at all — it is Conway's Game of Life (Gardner (1970)), expressed in GeLB. The purpose is to show that everything introduced in the preceding chapters (lattices, initializers, dynamics, gauges, geometry, and the `f.in`/`f.out` ping-pong contract) works unchanged on problems that are not LBM, provided they can be expressed as updating each cell from its own and its neighbours' previous state.

Conway's Game of Life is a canonical cellular-automaton stencil computation: a binary grid, an 8-neighbour Moore stencil, and three transition rules. It maps onto GeLB without any new language constructs — 63 lines including comments (37 lines of GeLB code), shorter than the 1D heat-diffusion program in *1D Diffusion*.

Note

The example walked here is `examples/lgca/2d/game_of_life_glider.gelb`. It runs a classic *glider* pattern on a 32×32 doubly-periodic torus (`GRID = 32`), small enough that individual cells are clearly visible in the plots below.

11.1 The rules

Conway's Game of Life operates on a 2D grid of cells, each either **alive** (1) or **dead** (0). At each discrete generation, every cell inspects its eight Moore-neighbourhood neighbours and applies three rules simultaneously:

1. **Birth**: a dead cell with exactly 3 live neighbours becomes alive.
2. **Survival**: a live cell with 2 or 3 live neighbours stays alive.
3. **Death**: all other cells die (or stay dead).

A *glider* is the smallest pattern that translates across the grid: a 5-cell shape with period 4 that shifts by one cell diagonally every four generations. On a doubly-periodic torus it wraps around both boundaries indefinitely.

11.2 The GeLB program

Here is the top of `examples/lgca/2d/game_of_life_glider.gelb`:

```
simulation_description("2D Game of Life with a glider (deterministic ICs)");

global_constants {
    const int GRID = 32;
}

lattices {
    lattice f(2, 1, int, {GRID, GRID}, {true, true});
}
```

The program declares a single 2D lattice with **one** scalar slot per cell (`q = 1`, rather than 5 or 9), **integer** element type (`int` rather than `real`), and doubly-periodic boundaries `{true, true}` (`GRID = 32` by default). It stores the cell state directly as `1` (alive) or `0` (dead).

11.3 The initializer: a glider at (1, 1)

```
initializer GliderInit {
    f.out[x, y | 0] = (x == 2) * (y == 3) +
                      (x == 3) * (y == 2) +
                      (x == 1) * (y == 1) +
                      (x == 2) * (y == 1) +
                      (x == 3) * (y == 1);
}
```

The glider pattern consists of five live cells: `(2, 3)`, `(3, 2)`, and the horizontal bar `(1, 1)`, `(2, 1)`, `(3, 1)`. With this orientation, the glider translates by $(\Delta x, \Delta y) = (+1, -1)$ every four steps.

> **Important**
>
> **Rule:** GeLB has no `if` statement inside kernel blocks; instead, relational comparisons such as `(x == 2)` evaluate to `bool` and promote to `int` (`1` when true, `0` when false) when used in integer arithmetic. Multiplying two comparisons `(x == a) * (y == b)` yields `1` at cell `(a, b)` and `0` elsewhere, and summing mutually exclusive indicator products sets the five glider cells to `1` without branching.

This is the same pure-expression pattern used by the `InCircle` geometry predicate in *Kármán Vortex Street*.

11.4 The dynamic: Conway's rules in four lines

```
dynamic LifeEvolve {
    int neighborSum;

    neighborSum = f.in[x - 1, y - 1 | 0] + f.in[x - 1, y | 0] +
                  f.in[x - 1, y + 1 | 0] + f.in[x, y - 1 | 0] +
                  f.in[x, y + 1 | 0] + f.in[x + 1, y - 1 | 0] +
                  f.in[x + 1, y | 0] + f.in[x + 1, y + 1 | 0];
```

(continues on next page)

(continued from previous page)

```
    int alive;
    int birth;
    int survive;

    alive = f.in[x, y | 0];
    birth = (neighborSum == 3);
    survive = alive * (neighborSum == 2);
    f.out[x, y | 0] = birth + survive - birth * survive;
}
```

Two aspects of `LifeEvolve` are worth noting:

- **The 8-neighbour Moore stencil.** Eight explicit `f.in[x ± 1, y ± 1 | 0]` reads gather the surrounding cell states — the same non-local offset access pattern used in *Shan-Chen Multiphase* for the pseudopotential force. Because `f` is declared with `{true, true}` periodicity, offset reads at the domain edges wrap around automatically on every backend. Declaring `{false, false}` periodicity while painting `BULK` on the boundary would trigger the compile-time out-of-bounds stencil diagnostic shown in *Shan-Chen Multiphase*.
- **Branch-free transition rule.** A cell is alive in the next generation if and only if it has 3 live neighbours (either a birth or a 3-neighbour survival), *or* it is currently alive and has 2 live neighbours:
  - `birth = (neighborSum == 3)` evaluates to `1` whenever a cell (alive or dead) has exactly 3 live neighbours.
  - `survive = alive * (neighborSum == 2)` evaluates to `1` only when the cell is currently alive *and* has 2 live neighbours.
  - `birth + survive - birth * survive` computes the logical OR of the two mutually exclusive conditions (with `- birth * survive` included as the general inclusion–exclusion guard).

11.5 The gauge and geometry

```
gauge CellStatus -> (int status) {
    status = f.in[x, y | 0];
}

node_classes {
    node_class BULK(GliderInit, LifeEvolve, CellStatus);
}

geometry {
    nodes[:, :] = BULK;
}
```

A single node class covers the entire doubly-periodic grid.

11.6 Running it

```
gelb-run -n 120 -w 1 \
    examples/lgca/2d/game_of_life_glider.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* to 120 and *Write every* to 1.

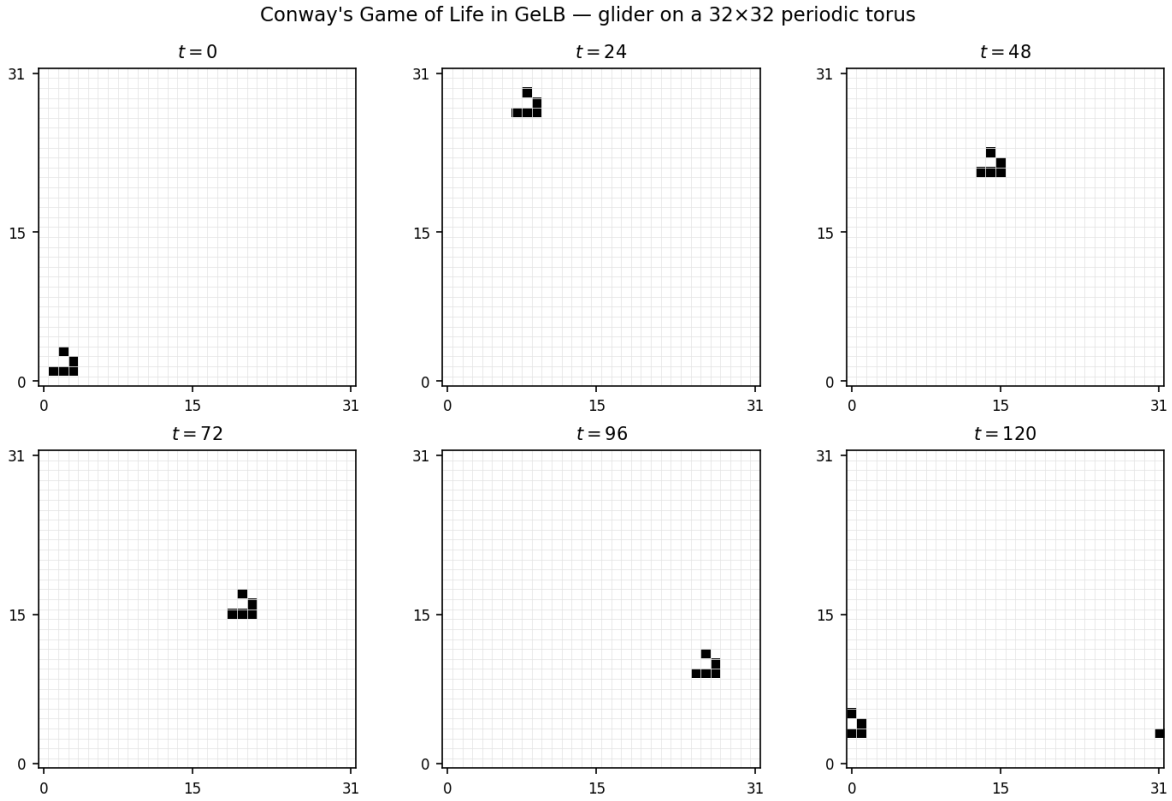


Fig. 1: Six snapshots ($t = 0, 24, 48, 72, 96, 120$) of the 5-cell glider on a 32×32 doubly-periodic torus (black = alive, white = dead). Starting near the bottom-left corner at $t = 0$, the glider shifts by $(\Delta x, \Delta y) = (+1, -1)$ every four generations: it wraps across the bottom periodic boundary ($y = 0 \to 31$) to the top-left before $t = 24$, marches diagonally down and right at six cells per 24-step interval ($t = 24 \ldots 96$), and is caught mid-wrap across the right periodic boundary ($x = 31 \to 0$) at $t = 120$.

11.7 Why this matters

Conway's Game of Life has no distribution functions, equilibrium moments, relaxation time, or conservation laws. Yet every GeLB construct in the program — the lattice declaration, initializer, dynamic block, gauge, `f.in`/`f.out` ping-pong contract, non-local offset reads, and periodic boundary wrapping — works identically to the fluid examples. GeLB's core execution model is a general **structured-grid stencil compiler** equipped with LBM-specific intrinsics (`upstream_dfs`, `bounce_back_dfs`, `dot_prod`). Any explicit cellular automaton or finite-difference stencil update that computes each cell's next state from a bounded neighbourhood of the previous step compiles through the same validation and multi-backend pipeline.

11.8 What's next

The next chapter, *Reduced-Gravity Ocean Circulation*, returns to fluid mechanics with a geophysical application — the wind-driven reduced-gravity ocean circulation model of Salmon (1999), which concludes the guided tour.

After that, Part C provides reference chapters:

- *Boundary Conditions Cookbook* — a consolidated catalogue of boundary condition patterns used across the tutorial, with code snippets and cross-references.
- *Source Visualisation: gelb-view source* — how to use `gelb-view source` to inspect geometry maps and stencil patterns extracted by the compiler.
- *Performance and Backends* — `cpu_highway`, `gpu_cuda_cpp`, and `cpu_fortran_serial` backends, when to use each, and throughput benchmarks.
- *Validation and Convergence* — quantitative validation against analytical solutions and literature benchmarks.
- *Numerical Semantics* — floating-point and integer semantics guaranteed across backends.
- *Further Reading* — a cross-reference table mapping tutorial chapters to textbook sections in Krüger et al. (2017), Mohamad (2019), and Sukop & Thorne (2006).

CHAPTER 12

Reduced-Gravity Ocean Circulation

This chapter applies the LBM techniques from the tutorial to a geophysical fluid dynamics problem: **wind-driven ocean circulation** in a closed rectangular basin. The walked example is a port of the shallow-water D2Q9 model introduced by Salmon (1999), "The lattice Boltzmann method as a basis for ocean circulation modeling" (§3, Eqs. 3.13–3.15, Fig. 3a), which reproduces the classical Stommel/Munk double-gyre circulation and western boundary current intensification using explicit local collision and streaming rules.

For climate scientists and physical oceanographers, this chapter shows how a rotating shallow-water basin model maps onto GeLB. For LBM practitioners, it demonstrates two forcing techniques — **exact-rotation Coriolis** and **Strang-split Guo momentum injection** — combined with a modified hydrostatic equilibrium distribution.

> **Note**
>
> The example walked here is `examples/lbm/2d/reduced_gravity_2d_sw_d2q9.gelb`.

12.1 The physics

Consider a single active upper ocean layer of thickness $h(x, y, t)$ overlying a deep quiescent abyss, with reduced gravity $g' = g\,\Delta\rho/\rho_0$. The layer is confined to a closed rectangular basin of side L with no-slip lateral walls and driven at the surface by a steady zonal wind stress

$$\tau_x(y) = -\tau_0 \cos\!\left(\frac{2\pi y}{L}\right), \qquad \tau_y = 0\,,$$

corresponding to easterly trades near the southern and northern walls and westerlies at mid-latitudes. This wind-stress curl drives a clockwise **subtropical anticyclone** (thickened upper layer, high h) in the southern half of the basin and a counter-clockwise **subpolar cyclone** (thinned upper layer, low h) in the northern half.

On a mid-latitude β-plane where the Coriolis parameter varies linearly with latitude as $f(y) = f_0 + \beta y$, the depth-integrated rotating shallow-water equations with lateral eddy viscosity ν and Ekman bottom-drag regularisation δ_E

read (Salmon (1999) §3):

$$\frac{\partial(hu)}{\partial t} + \nabla\cdot(hu\,\mathbf{u}) = -g'h\frac{\partial h}{\partial x} + f(y)\,hv + \tau_x(y)\frac{h}{h+\delta_E} + \nu\nabla^2(hu)\,,$$
$$\frac{\partial(hv)}{\partial t} + \nabla\cdot(hv\,\mathbf{u}) = -g'h\frac{\partial h}{\partial y} - f(y)\,hu + \nu\nabla^2(hv)\,,$$
$$\frac{\partial h}{\partial t} + \nabla\cdot(h\,\mathbf{u}) = 0\,.$$

Because $\beta = \partial f/\partial y > 0$, planetary vorticity conservation (βv balance in the Sverdrup interior) shifts the return flow of both gyres into an intense, narrow **western boundary current** (the Stommel/Munk analogue of the Gulf Stream or Kuroshio) of characteristic Munk width $\delta_M = (\nu/\beta)^{1/3}$.

Salmon (1999) §3 (Eqs. 3.13–3.15) showed that the D2Q9 lattice Boltzmann equation recovers these shallow-water equations at leading order when the mass density ρ is identified with the layer depth h and the isothermal pressure $c_s^2\rho$ in the second moment of the equilibrium distribution is replaced by the hydrostatic shallow-water pressure $\frac{1}{2}g'h^2$ (see also Krüger et al. (2017) §3.2 and §6.3 for the D2Q9 BGK and forcing framework, and chapter 10 for the MRT collision used below).

12.2 The GeLB program

The file `examples/lbm/2d/reduced_gravity_2d_sw_d2q9.gelb` is 395 lines including comments. Most of the source consists of explicit full-way bounce-back dynamic blocks (the wall midway between the nodes) for the four basin walls and four corner cells; the shallow-water physics lives in `FeqSW` and the 43-line `BulkDynamic` block.

12.2.1 The shallow-water equilibrium

```
function FeqSW(real h, real ux, real uy) -> (array[real, 9] feq) {
    real usq = ux * ux + uy * uy;
    array[real, 9] cu = EX * ux + EY * uy;

    // Standard D2Q9 BGK equilibrium with ρ ↦ h, plus Salmon's
    // pressure-like term P_COEF · g · h² (encoding −∇(gh²/2)).
    feq = W * h * (1.0 + 3.0 * cu + 4.5 * cu * cu - 1.5 * usq)
          + P_COEF * (G * h * h);
}
```

Here `FeqSW` starts from the standard D2Q9 quadratic equilibrium with $\rho \mapsto h$ and adds `P_COEF * (G * h * h)`, where

```
const array[real, 9] P_COEF = {-5.0/6.0,
                               1.0/6.0,  1.0/6.0,  1.0/6.0,  1.0/6.0,
                               1.0/24.0, 1.0/24.0, 1.0/24.0, 1.0/24.0};
```

The coefficient vector `P_COEF` encodes the quadratic hydrostatic pressure contribution $\frac{1}{2}g'h^2\delta_{\alpha\beta}$ of Salmon (1999) Eqs. (3.13)–(3.15) while satisfying three moment constraints:

- **Zeroth moment:** $\sum_{i=0}^{8}\mathrm{P_COEF}_i = -\frac{5}{6} + 4(\frac{1}{6}) + 4(\frac{1}{24}) = 0$, so mass conservation $\sum_i f_i^{\mathrm{eq}} = h$ is preserved exactly.
- **First moment:** $\sum_{i=0}^{8}\mathbf{c}_i\,\mathrm{P_COEF}_i = \mathbf{0}$ by lattice symmetry, so $\sum_i \mathbf{c}_i f_i^{\mathrm{eq}} = h\mathbf{u}$.
- **Second moment:** since $\mathrm{P_COEF}_i = \frac{3}{2}w_i$ for $i = 1\ldots8$, $\sum_{i=0}^{8} c_{i\alpha}c_{i\beta}\,\mathrm{P_COEF}_i(g'h^2) = \frac{3}{2}c_s^2 g'h^2\delta_{\alpha\beta} = \frac{1}{2}g'h^2\delta_{\alpha\beta}$, whose divergence supplies the shallow-water hydrostatic pressure force $-\nabla(\frac{1}{2}g'h^2) = -g'h\nabla h$.

12.2.2 The bulk dynamic — Coriolis + wind + MRT

```
dynamic BulkDynamic {
    // Pull-stream and extract conserved moments.
    array[real, 9] fIn = upstream_dfs(f, x, y, {EX, EY});
    real h   = sum(fIn);
    real hu0 = dot_prod(EX, fIn);
    real hv0 = dot_prod(EY, fIn);

    // Ekman-scaled double-gyre wind stress, y-only profile, with
    // a linear spin-up ramp τ₀ · min(1, t / T_RAMP) that reaches
    // full strength after T_RAMP outer-loop steps.
    real bf    = h / (h + DELTA_E);
    real ramp  = min(1.0, real(t) / T_RAMP);
    real tau_x = -TAU0 * cos(TWO_PI * real(y) / real(NY - 1)) * ramp;

    // Half-angle rotation coefficients for Coriolis.
    real f_loc = F0 + BETA * real(y);
    real ca    = cos(0.5 * f_loc);
    real sa    = sin(0.5 * f_loc);

    // --- 1st half force: wind/2 then rotate/2 ---
    real hu_a = hu0 + 0.5 * bf * tau_x;
    real hu_h =  ca * hu_a + sa * hv0;
    real hv_h = -sa * hu_a + ca * hv0;

    // Inject pre-collision momentum delta so mom(fInForced) = (hu_h, hv_h).
    real d1_hu = hu_h - hu0;
    real d1_hv = hv_h - hv0;
    array[real, 9] fInForced = fIn + W * 3.0 * (EX * d1_hu + EY * d1_hv);

    // --- MRT collision: feq built from (hu_h, hv_h) matches mom(fInForced);
    // h, j_x, j_y are conserved (S_DIAG = 0), so the first moment
    // after collision is exactly (hu_h, hv_h).
    array[real, 9] feq   = FeqSW(h, hu_h / h, hv_h / h);
    array[real, 9] mom   = matmul(M, fInForced);
    mom = mom - S_DIAG * (mom - matmul(M, feq));
    array[real, 9] fPost = matmul(M_INV, mom);

    // --- 2nd half force: rotate/2 then wind/2 (symmetric to the first half)␣
↪---
    real hu_b =  ca * hu_h + sa * hv_h;
    real hv_f = -sa * hu_h + ca * hv_h;
    real hu_f = hu_b + 0.5 * bf * tau_x;

    // Inject post-collision momentum delta.
    real d2_hu = hu_f - hu_h;
    real d2_hv = hv_f - hv_h;
    f.out[x, y | :] = fPost + W * 3.0 * (EX * d2_hu + EY * d2_hv);
}
```

Four physical and numerical ingredients work together in `BulkDynamic`:

1. **Exact-rotation Coriolis.** In Salmon's dimensionless units, $f_0 \Delta t \approx 0.4712$. Discretising the Coriolis acceleration $\mathrm{d}(hu)/\mathrm{d}t = f\,hv$, $\mathrm{d}(hv)/\mathrm{d}t = -f\,hu$ with explicit forward Euler amplifies the kinetic energy $|hu|^2 + |hv|^2$

by a factor of $1 + (f\Delta t)^2 \approx 1.22$ per step, blowing up long before viscous spin-up completes. Instead, `BulkDynamic` rotates (hu, hv) through the exact half-angle $\frac{1}{2} f(y)\Delta t$ using `cos` and `sin`, preserving $|hu|^2 + |hv|^2$ to machine precision.

2. **Symmetric Strang splitting with Guo momentum injection.** As in *Poiseuille with Body Force*, the forcing is split symmetrically around collision: half wind stress followed by half Coriolis rotation before collision, and half Coriolis rotation followed by half wind stress after collision.

3. **Smooth wind spin-up via the implicit `t` symbol.** The built-in time-step index `t` (introduced in *GeLB in 10 Minutes*) drives the linear ramp `min(1.0, real(t) / T_RAMP)`, bringing the wind stress smoothly to full amplitude over the first 20 000 steps so initial basin gravity waves can damp out cleanly.

4. **MRT collision at Salmon's relaxation rate.** The example runs at Salmon's $\lambda = 0.95\,\lambda_{\max}$ (`OMEGA = 1.9`, $\nu = c_s^2(1/\omega - \frac{1}{2}) \approx 8.8 \times 10^{-3}$ in lattice units), where the Munk layer $\delta_M = (\nu/\beta)^{1/3} \approx 1.4$ cells is barely resolved. With a single relaxation rate (BGK), grid-scale oscillations of the non-hydrodynamic moments then grow along the western boundary until the run fails, near step 17 000. The collision is therefore done in moment space (`matmul(M, ...)`, as in *Poiseuille with Body Force*): the energy and stress moments relax at `OMEGA`, so the shear and bulk viscosities are exactly those of BGK, while the three non-hydrodynamic moments (ϵ, q_x, q_y) relax at rate 1, which damps the grid-scale modes without changing the hydrodynamics. `-D OMEGA=1.2` gives a more viscous, smoother basin ($\nu \approx 0.11$, $\delta_M \approx 3.4$ cells).

Important

Rule: When combining Strang-split forcing with a collision that conserves momentum, the pre-collision momentum increment $(\Delta(hu)_1, \Delta(hv)_1)$ must be injected into the distribution functions *before* the collision (`fInForced = fIn + W * 3.0 * (EX * d1_hu + EY * d1_hv)`), in addition to passing the half-step velocity into `FeqSW`. The MRT collision leaves the momentum moments unchanged, so without the injection the first half of the force would be lost; with BGK, whose post-collision first moment is $(1-\omega)\,\mathrm{mom}(f^{\mathrm{in}}) + \omega\,\mathrm{mom}(f^{\mathrm{eq}})$, it would be weighted by ω instead of 1.

12.3 The geometry — 9 node classes

```
node_classes {
    node_class FLUID(InitAll, BulkDynamic, OceanGauge);
    node_class WALL_BOT  (InitAll, WallBotDynamic)   masked;
    node_class WALL_TOP  (InitAll, WallTopDynamic)   masked;
    node_class WALL_LEFT (InitAll, WallLeftDynamic)  masked;
    node_class WALL_RIGHT(InitAll, WallRightDynamic) masked;
    node_class WALL_BL   (InitAll, WallBLDynamic)    masked;
    node_class WALL_BR   (InitAll, WallBRDynamic)    masked;
    node_class WALL_TL   (InitAll, WallTLDynamic)    masked;
    node_class WALL_TR   (InitAll, WallTRDynamic)    masked;
}
```

The interior `FLUID` region is enclosed by four straight edge walls (`WALL_BOT`, `WALL_TOP`, `WALL_LEFT`, `WALL_RIGHT`) and four single-cell corner classes (`WALL_BL`, `WALL_BR`, `WALL_TL`, `WALL_TR`), giving nine node classes in total. Each corner cell sits at the intersection of two walls and reflects the single diagonal DF pointing toward its sole interior fluid neighbour (for example, slot 5 `NE` at `WALL_BL` reads slot 7 `SW` from fluid cell `(1, 1)`). All eight boundary classes carry the `masked` modifier so their gauge fields are written as `NaN` in NetCDF, outlining the 100×100 fluid basin automatically.

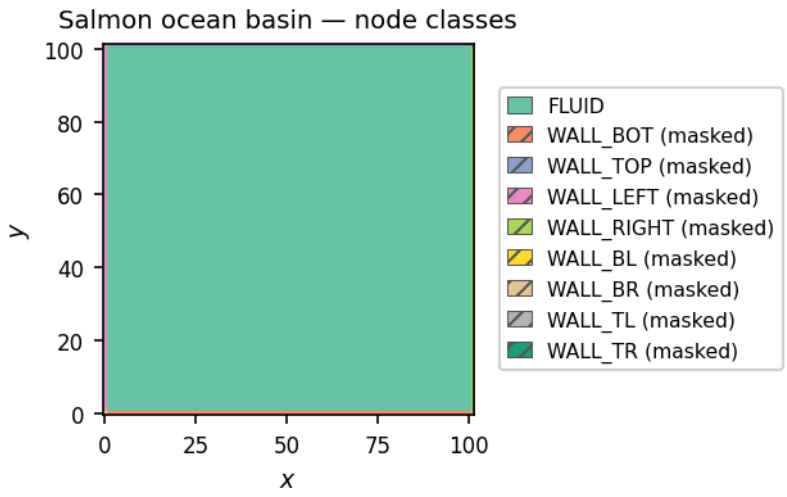


Fig. 1: Node-class layout for the 102×102 domain: `FLUID` (100×100 interior, teal) surrounded by four edge-wall and four corner node classes (all `masked`, shown with hatching in the legend on the right).

12.4 Running it

```
gelb-run -n 100000 -w 1000 \
    examples/lbm/2d/reduced_gravity_2d_sw_d2q9.gelb
```

In VS Code (*Working in VS Code*), set *Time steps* to 100000 and *Write every* to 1000.

Running 100 000 steps on the 102×102 grid with the default `cpu_highway` backend takes about 10 seconds, compilation included, on a 16-core desktop CPU. The wind ramp reaches full strength at $t = 20\,000$, and the asymmetric double-gyre pattern is established by $t = 50\,000$.

12.5 What this demonstrates

This example combines a non-standard hydrostatic equilibrium, exact trigonometric Coriolis rotation, a time-dependent wind spin-up ramp, and two-stage Strang-split Guo momentum injection and an MRT collision inside a single 47-line `BulkDynamic` block that mirrors the governing equations directly. Across the eight wall and corner classes, the compiler statically verifies that all nine D2Q9 distribution slots are written at every boundary cell and that no stencil read reaches outside the non-periodic 102×102 grid.

See also

Salmon, R. (1999) "The lattice Boltzmann method as a basis for ocean circulation modeling." *Journal of Marine Research*, **57**, 503–535. Section 3 (Eqs. 3.13–3.15) derives the shallow-water D2Q9 formulation; Figure 3a shows the 100×100 double-gyre benchmark.

Krüger et al. (2017) §3.2 (*The Lattice Boltzmann Equation in a Nutshell*) and §6.3 (*LBE with Body Forces*) for the D2Q9 BGK and Guo/Strang forcing foundations; chapter 10 (*MRT and TRT Collision Operators*) for the moment-space collision.

The repository also provides two planetary-geostrophic (PG) companions from Salmon (1999): `examples/lbm/2d/reduced_gravity_2d_pg_d2q9.gelb` (linearised PG on D2Q9, Salmon Fig. 3c–d) and `examples/lbm/2d/reduced_gravity_2d_pg_d2q5.gelb` (PG on a 5-velocity D2Q5 lattice, Salmon Fig. 6a),

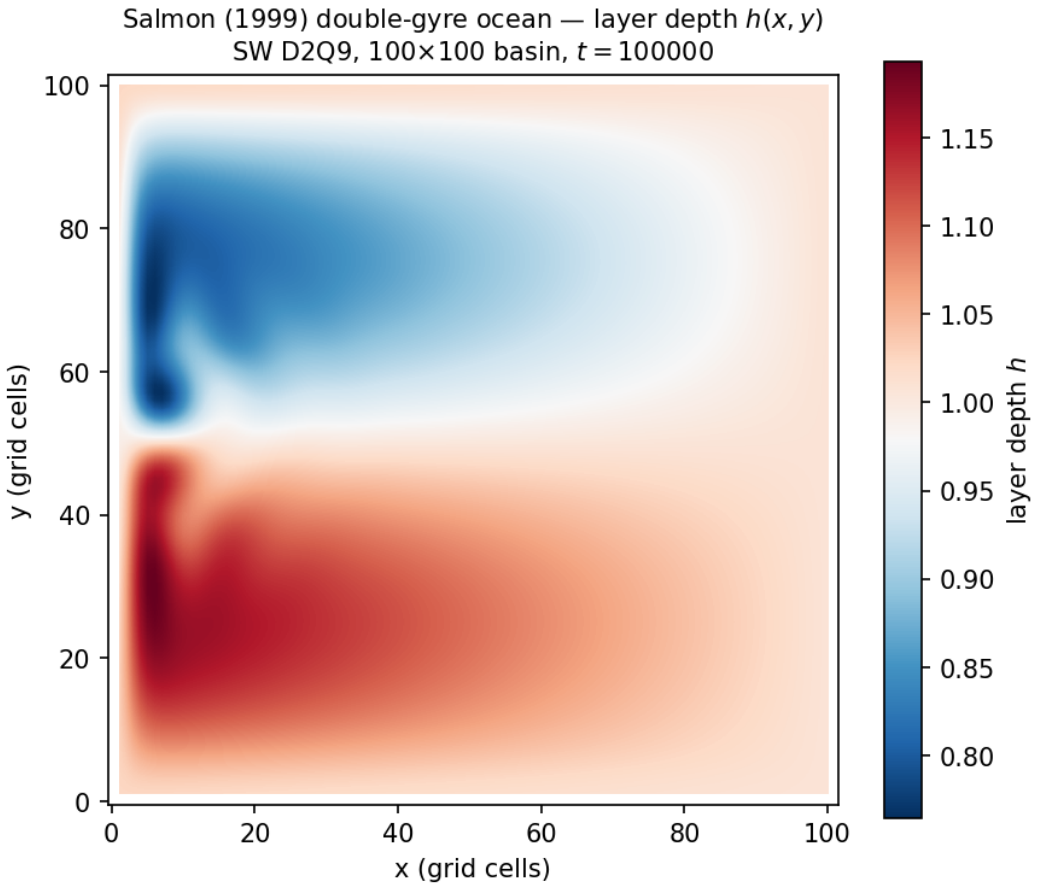


Fig. 2: Upper-layer depth $h(x, y)$ at $t = 100\,000$ in the 100×100 fluid basin (102×102 including masked perimeter walls). The sinusoidal zonal wind stress drives a subtropical anticyclone (thickened upper layer, $h > 1$, warm colours) in the southern half and a subpolar cyclone (thinned upper layer, $h < 1$, cool colours) in the northern half, with strong **western boundary intensification** in a narrow band along the left wall, produced by the β-plane Coriolis gradient; the two western boundary currents meet near mid-basin ($y \approx 50$) and turn east together.

which drop the $\mathcal{O}(u^2)$ advective terms in the equilibrium to solve the planetary-geostrophic balance $f\hat{\mathbf{z}} \times h\mathbf{u} + g'h\nabla h = \boldsymbol{\tau} + \nu\nabla^2(h\mathbf{u})$.

Part C

Reference material

CHAPTER 13

Boundary Conditions Cookbook

This chapter is a single-page reference for every boundary condition pattern used in the tutorial. Each entry gives the GeLB code snippet, the physical BC it implements, and a pointer to the chapter where it was first introduced.

13.1 Static Dirichlet (fixed value)

A node class with an initializer but **no dynamic block** holds its DFs at the initialized values forever. The gauge still reports the cell's state, but nothing updates it.

```
initializer InitWall { f.out[x | :] = DiffusionEq(T_wall); }
node_class WALL(InitWall, Temperature);   // no dynamic
```

This is the simplest Dirichlet BC. The wall cell's temperature stays at `T_wall` for all time because no dynamic block runs on it. The neighbouring cells see that temperature imposed $1 - \tau$ cells from the wall cell, not at it; `examples/lbm/1d/heat_diffusion_1d_antibounceback.gelb` puts the wall half a cell beyond the end cell instead, with the anti-bounce-back rule.

First used in *1D Diffusion* (left and right walls).

13.2 Neumann flux (time-dependent)

To impose a heat flux $q''(t)$ at a wall, modify the unknown incoming DF using the LBM Neumann formula (Krüger et al. (2017) §8.5.3):

```
dynamic FluxWall {
    real f0 = f.in[x | 0];
    real f2 = f.in[x + 1 | 2];
    real I_now = FLUX_AMP * sin(PI * real(t) / PERIOD);
    f.out[x | 0] = f0;
    f.out[x | 2] = f2;
    f.out[x | 1] = f2 + I_now;
}
```

The per-DF writes set DF 1 (the unknown incoming direction) to the outgoing DF 2 plus the flux correction. DFs 0 and 2 are pulled from this cell and its right neighbour and copied through unchanged. The block reads them directly rather than through `upstream_dfs`, which would also read DF 1 from `x - 1`, outside the grid: the compiler rejects that read at a wall.

Not used by a tutorial chapter; the `t` symbol is listed in the cheat-sheet of *GeLB in 10 Minutes*.

13.3 Moving-wall bounce-back

A bounce-back wall sends every population that reaches it back along its link; `bounce_back_dfs` makes that a one-liner. For a wall moving at velocity u_w, add a momentum-transfer term to each reflected DF:

```
dynamic MovingLid {
    f.out[x, y | :] = bounce_back_dfs(f, x, y, {EX, EY})
                      + 6.0 * W * (EX * U_LID);
}
```

For each direction `i`, `bounce_back_dfs` reads the population of the neighbour in direction `i` that points back at this cell, and the fluid neighbour pulls it on the next step. This is full-way bounce-back (Krüger et al. (2017) §5.3.3.2): the wall lies midway between the wall node and the first fluid node. The `6 w_i (c_i · u_w)` term injects the wall's momentum, with the wall density taken as 1. For a stationary wall, drop the second term entirely.

First used in *Lid-Driven Cavity*.

13.4 Bounce-back written out per link

`bounce_back_dfs` can also be written out one DF at a time. At a bottom wall cell, the three DFs that enter the fluid read the fluid neighbours' populations travelling the other way:

```
dynamic WallBot {
    f.out[x, y | 2] = f.in[x,     y + 1 | 4];   // N ← S
    f.out[x, y | 5] = f.in[x + 1, y + 1 | 7];   // NE ← SW
    f.out[x, y | 6] = f.in[x - 1, y + 1 | 8];   // NW ← SE
    // ... remaining DFs set to 0 ...
}
```

These are entries 2, 5 and 6 of `bounce_back_dfs(f, x, y, {EX, EY})` at a bottom wall; the fluid never pulls the other six, so setting them to zero changes nothing for the flow. When the fluid cell runs `upstream_dfs` on the next step, it pulls from the wall cell the population it sent toward the wall two steps earlier; the effective wall lies midway between the two cells. Writing the links out is useful when a wall needs its own rule per link, as the thermal wall below does.

First used in *Poiseuille with Body Force*. Also used extensively in *Rayleigh-Bénard Convection* and *Reduced-Gravity Ocean Circulation*.

13.5 Yoshida anti-bounce-back (fixed-T thermal wall)

For the temperature lattice, the anti-bounce-back scheme (Yoshida & Nagaoka 2010) sets the wall temperature to a fixed value T_w by reflecting with a sign change plus a correction term:

```
// Temperature lattice g, hot bottom wall: the north-going DF
g.out[x, y | 2] = -g.in[x, y + 1 | 4]
                  + 2.0 * sqrt(3.0) * DIFFUSIVITY * T_HOT;
```

The $2\sqrt{3}\,\kappa\,T_w$ correction term (with `DIFFUSIVITY` the thermal diffusivity κ) enforces the Dirichlet condition at the wall, midway between the nodes; the snippet is from `examples/lbm/2d/rayleigh_benard_2d_mrt.gelb`.

First used in *Rayleigh-Bénard Convection*.

13.6 Periodic boundaries

Periodic boundaries are declared in the lattice definition, not in a dynamic block:

```
lattice f(2, 9, real, {N, N}, {true, true});
```

No special wall node class is needed: a read past one edge comes from the opposite edge, so the grid simply wraps around.

First used in *Hello World*. Also used in *Shan-Chen Multiphase* and *Beyond LBM: Conway's Game of Life*.

13.7 Zou-He velocity / pressure

The Zou-He scheme (Zou & He 1997) imposes velocity or pressure at an inlet/outlet by solving for the unknown DFs from the known macroscopic constraint. GeLB expresses this as a dynamic block with per-DF writes derived from the constraint equations.

See `examples/sukop/poiseuille_2d_bgk_pressure_zouhe.gelb` for a pressure-driven Poiseuille flow using Zou-He pressure BCs, and `examples/lbm/2d/poiseuille_flow_2d_mrt_zouhe.gelb` for the MRT variant.

13.8 Masked node classes (obstacle interiors)

Cells inside a solid obstacle need no physics — they just need to exist so wall cells can read from them. No fluid cell may reach them along any link, so the wall layer around them has to cover the diagonals too, as the layer `border()` paints does (see *Kármán Vortex Street*). The `masked` modifier suppresses the gauge output of a node class (its cells emit NaN in the NetCDF output, so visualisation tools render them as blank space or a distinct colour) and disallows an attached gauge; an initializer and a dynamic, when present, still run. An *empty* masked class holds defined zero DFs:

```
node_class SOLID() masked;
```

Other masked storage follows the ordinary initialization and coverage rules.

First used in *Kármán Vortex Street*.

CHAPTER 14

Source Visualisation: `gelb-view source`

GeLB ships a single tool for inspecting the *compile-time* state of a `.gelb` program: `gelb-view source`. It bundles two views:

- **Geometry** — the per-cell node-class layout (which block runs where). This is what the geometry figures of the Part B chapters show.
- **Stencils** — the per-block I/O pattern (which DFs each `dynamic` / `gauge` block reads from and writes to, expressed as spatial offsets from the current cell).

Both views answer different questions about the same program, and `gelb-view source` lets you see them together.

14.1 Quick start

```
# Geometry only (1D bar / 2D heatmap / 3D slices)
gelb-view source examples/sukop/karman_vortex_street_2d_bgk.gelb \
    --show=geo -o geometry.png

# Stencils only (per-block velocity-set diagrams)
gelb-view source examples/sukop/karman_vortex_street_2d_bgk.gelb \
    --show=algo -o stencils.png

# Side-by-side
gelb-view source examples/sukop/karman_vortex_street_2d_bgk.gelb \
    --show=both -o both.png

# Combined: geometry + arrows from one cell to its read neighbours
gelb-view source examples/lbm/2d/rayleigh_benard_2d_mrt.gelb \
    --show=combined --cell 64,1 -o combined.png

# ASCII text mode (useful over SSH or in CI logs)
gelb-view source examples/lbm/1d/heat_diffusion_1d_dirichlet.gelb \
    --text
```

(continues on next page)

(continued from previous page)

```
# Statistics table
gelb-view source examples/sukop/karman_vortex_street_2d_bgk.gelb \
    --stats

# JSON export (for tooling — used by the VSCode extension)
gelb-view source examples/lbm/2d/rayleigh_benard_2d_mrt.gelb \
    --export-json analysis.json
```

You can also drop the subcommand entirely — `gelb-view` auto-detects the file type and routes `.gelb` → `source`, `.nc` → `data`:

```
gelb-view examples/sukop/karman_vortex_street_2d_bgk.gelb
```

14.2 Single-view subcommands

Two subcommands open one view on its own:

- `gelb-view geo file.gelb` — same as `source --show=geo`
- `gelb-view algo file.gelb` — opens the standalone PySide6 stencil viewer (`source` is the recommended entry point)

14.3 Reading the combined view

The combined mode is the most informative — it shows **which cells your dynamic block actually reads**. Click (or pass `--cell x,y`) to pick a cell; the figure draws an arrow from that cell to each cell its block reads from, labelled with the DF index.

This is invaluable for debugging boundary conditions. If you write a wall block that's supposed to bounce DF 5 (NE) but accidentally reads `f.in[x+1, y+1 | 7]` instead of `f.in[x+1, y+1 | 5]`, the arrow points at the wrong DF — a visual mismatch that's much easier to spot than a numerical bug in the output.

14.4 Lint warnings (`--lint`)

GeLB includes a set of geometry-lint checks that run over the resolved node-class map and flag common mistakes:

```
gelb-view source mygeometry.gelb --lint
```

Checks performed:

- **Disconnected NC regions** — a node class painted in many small pieces may indicate a paint mistake (using full 8/26-connectivity, warns at >4 components)
- **Unreachable fluid pockets** — fluid cells separated from the bulk by walls

Lint warnings are also surfaced by `gelbc` itself, so they appear at compile time.

14.5 VSCode integration

The `vscode-gelb` extension provides a live interactive preview pane that re-renders the geometry / stencils on every save. See `vscode-gelb/README.md` for setup and the full feature list.

Once installed:

- Open any `.gelb` file
- Run "GeLB: Show Source Preview" from the command palette (or click the *GeLB: Show Source Preview* button in the editor's title bar)
- An interactive webview opens beside the editor: pan / zoom in 2D, orbit / fly-through in 3D, click a cell to inspect it, press **Space** to animate the reads / writes stencil
- Edit the file → save → preview refreshes within ~1 sec

The extension calls `gelb-view source --export-json` and renders the result in the webview.

The sidebar's **Run** view runs the file with `gelb-run` as a VS Code task: the progress bar and the MLUPS report appear in a terminal of the panel, and *Stop* ends the run with everything it started. It exposes the everyday parameters as rows you click to edit: time steps (`-n`), write-every (`-w`), constant overrides (`-D NAME=VALUE`, picked from the file's `global_constants`), the seed (`-s`, shown only when the program calls `random()`), the backend (`-b`), whether to open the viewer at the end (`--view`) and the output folder, i.e. the working directory where `<name>.nc` lands (next to the source file by default). The values are remembered per file. One run goes at a time, numbered, with a *Status* row; pressing *Run* while a run is still going (compiling, simulating, or with its viewer window open) asks before stopping it. Every other flag of `gelb-run` (`--keep`, `--verbose`, `--fast-math`, `--output-mode stdout`, the `--cuda-*` knobs) is for the command line: the view's *Show Run Log* gives the command line of each run, to copy into a terminal and extend. The same run is a play button in the editor title bar and a *Run* CodeLens above `simulation_description(...)`. In a source checkout the view uses its `scripts/gelb-run`, in an installation its `bin/gelb-run`, with nothing to configure; the setting `gelb.gelbRunCommand` names another `gelb-run` when needed.

14.6 When to use it

- **Verify geometry layout** before running a long simulation — catch off-by-1 errors in paint statements, wrong border() inference, etc. Use `--show=geo`.
- **Debug a wall BC** — pick a wall cell and check the read footprint. Use `--show=combined --cell x,y`.
- **Quick stats** — count cells per node class, check periodicity flags. Use `--stats`.
- **Programmatic analysis** — integrate the geometry/stencils into your own tooling. Use `gelb-view source --export-json` for the full geometry + stencils bundle (the same JSON the VSCode extension consumes), or `gelb-view geo --export-npy` for a raw numpy `nc_map` + sidecar metadata.

The `gelb-view source` command works on any `.gelb` file, including programs that fail later in compilation (for example a type error in an expression) as long as their structure is valid: the geometry and stencil analysis runs before code generation.

CHAPTER 15

Performance and Backends

GeLB compiles to three backends. The same `.gelb` source runs on all three without changes — you choose the backend at compile time.

15.1 The three backends

``**cpu_highway**`` (default). Portable-SIMD C++ via Google Highway, compiled with `g++` and parallelised with OpenMP. Resolves to AVX-512 / AVX-2 (x86) or NEON (ARM / Apple Silicon) at compile time. This is the production CPU backend.

```
gelb-run -n 10000 examples/lbm/2d/poiseuille_flow_2d_mrt.gelb
```

``**gpu_cuda_cpp**`` (CUDA C++). Requires the CUDA toolkit (`nvcc`). The compiler emits CUDA C++ kernels and links against the same NetCDF-C library that `cpu_highway` already needs.

```
gelb-run -b gpu_cuda_cpp -n 10000 \
    examples/lbm/2d/poiseuille_flow_2d_mrt.gelb
```

``**cpu_fortran_serial**`` (educational reference). `gelbc` emits plain Fortran 90, compiled with `gfortran`. **Strictly single-threaded by design** — its role is to serve as the human-readable source-level reference and the correctness anchor that the SIMD and CUDA backends are tested against. Use it for debugging, small grids, and reference runs; reach for `cpu_highway` for actual performance work.

```
gelb-run -b cpu_fortran_serial -n 10000 \
    examples/lbm/2d/poiseuille_flow_2d_mrt.gelb
```

15.2 When to use which

- ``**cpu_highway**``: the default for everything on a CPU. Multi-threaded via OpenMP; scales with core count up to the memory bandwidth limit. For a single-threaded Highway baseline, set `OMP_NUM_THREADS=1` in the environment.

- ``**gpu_cuda_cpp**``: large 3D grids (128³+) on NVIDIA GPUs. The kernel launch overhead means CUDA only wins at large problem sizes.
- ``**cpu_fortran_serial**``: grids under ~100×100, debugging, human-readable reference, correctness anchor for the other backends.

15.3 Performance numbers

The figure below shows Compute MLUPS (million lattice updates per second) for three canonical 2-D examples — lid-driven cavity (D2Q9 BGK, 512²), Poiseuille (D2Q9 MRT, 256²), and Taylor-Green vortex (D2Q9 BGK, 512²) — across all three backends. Each bar is the median of three samples; CPU runs come from an Apple M4 (MacBook Air), CUDA runs from an NVIDIA RTX 4080 Laptop GPU. The benchmark harness is `scripts/benchmark_backends.py`; the underlying data lives in `docs/user/figures/data/16_mlups_backend_comparison.json`.

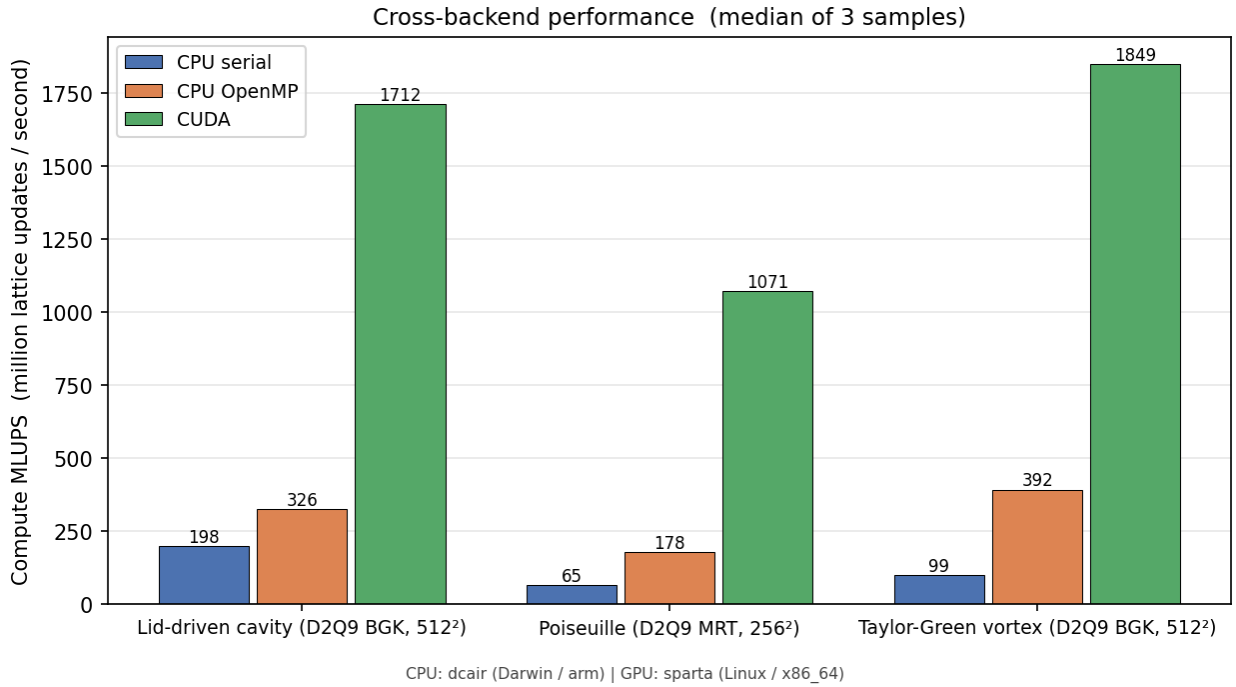


Fig. 1: Cross-backend performance. Median of three samples per case; gauges fire only at `t=0` and `t=N` so NetCDF I/O is amortised away.

A few patterns to notice:

- **CUDA dominates by an order of magnitude** at 256²–512² — `CUDA / serial` is ~9× for the BGK lid-cavity, ~16× for the MRT Poiseuille (where the per-step matmul amortises well on the GPU), and ~19× for the BGK Taylor-Green vortex. The payoff grows with the per-step arithmetic intensity, which is why MRT widens the GPU's lead more than its CPU lead.
- **OpenMP scaling varies by case**. Taylor-Green (BGK, 512²) hits ~4× over serial — close to the practical ceiling on this CPU. Lid-driven cavity at the same resolution gets only ~1.65× because BGK at 512² is largely memory-bandwidth bound; threads do not buy more bandwidth. Poiseuille MRT at 256² lands at ~2.7× — middling because the smaller grid gives less work per thread, but MRT's arithmetic still benefits.
- **Serial Fortran is competitive at small problem sizes**. If the grid fits in cache and you do not need a pretty animation, the serial backend is the simplest reproducibility story (deterministic, no thread-affinity surprises, no CUDA toolkit dependency).

What the numerical guarantees cost. *Numerical Semantics* fixes the arithmetic; a few of its rules have a measured price. The CPU backends compile with contraction off (`-ffp-contract=off`), so the native compiler never fuses a multiply and an addition on its own: 2.3–4.4 % on an Apple M4 across channel flow, TGV-2D BGK and Shan–Chen (the measurements behind that decision). Every term of a reduction counts, zero coefficients included, with one fusion rule in every code path: 2.1 % on TGV-2D BGK and 2.3 % on channel flow (circle obstacle, BGK) on a Ryzen 9 7945HX, default `cpu_highway` build, median of five interleaved rounds. CUDA default builds keep nvcc's fusion because turning it off costs 7.5–30.6 % on an RTX 4080 Laptop GPU (`scripts/bench-cuda-fusion`); a strict build (`--strict-fp`) pays that on CUDA and, on the CPU, the fusion inside the reduction helper: 1.3 % on channel flow and nothing measurable on the 64^2 Taylor–Green vortex, same machine and protocol.

Multi-threaded CPU versus GPU — mind the cache. Against the multi-threaded `cpu_highway` backend on a large desktop CPU the picture at 512^2 is different from the figure above: on a 16-core Zen 4 (AMD Ryzen 9 7945HX, 64 MB L3) the TGV-2D BGK anchor reaches about 4660 MLUPS, *twice* the RTX 4080 Laptop GPU (about 2360), because the ~38 MB D2Q9 working set fits in L3. That is a cache artifact, not a GPU weakness: at 2048^2 the same comparison is 327 versus 2332 MLUPS (GPU 7.1×) and at 128^3 it is 144 versus 825 (GPU 5.7×), with the GPU at roughly 96 % of its GDDR6 bandwidth ceiling in FP64. When you benchmark backends yourself, include at least one grid that does not fit in your CPU's last-level cache (2D N=2048 or 3D N=128 on this class of hardware); the reference numbers live in `docs/perf_baseline_zen4.md`.

15.4 Practical notes

OpenMP stack size. MRT and multi-lattice examples allocate large per-thread private arrays. If a `cpu_highway` simulation segfaults under OpenMP, raise the stack size in the shell before running (`gelb-run` inherits your shell's limits):

```
ulimit -s unlimited    # or: ulimit -s hard
OMP_STACKSIZE=64M gelb-run -n 10000 program.gelb
```

Reproducibility. Pass `--seed N` to `gelbc` or `gelb-run` to seed the PRNG deterministically. Without `--seed`, the runtime seeds from the system clock and every run produces different results for programs that call `random()`.

macOS (Apple Silicon). `gelb-run` prefers Homebrew GCC (`g++-16` down to `g++-13`; `brew install gcc netcdf`) and otherwise uses Apple clang from Xcode or the Command Line Tools, which needs only the OpenMP runtime: `brew install libomp`, or `GELB_OMP_ROOT` pointing at a directory that holds `include/omp.h` and `lib/libomp.dylib`. The clang build compiles with `-Xclang -fopenmp` and `-mcpu=native`, links `-lomp`, and matches the GCC results to round-off. `GELB_CXX` picks the compiler explicitly, and `GELB_NETCDF_C_ROOT` names a NetCDF-C prefix (`include/` + `lib/`) when `nc-config` is not on the PATH. `GELB_PYTHON` names an interpreter that can `import gelb`; when it is set, `gelb-run` runs `gelbc`, the progress bar and `gelb-view` through it and does not need `uv` (the relocatable macOS bundle built by `scripts/mac_bundle/build_bundle.sh` works this way). `OMP_NUM_THREADS` defaults to the performance cores.

15.5 Hand-tuning hot kernels

Most `.gelb` programs hit memory-bandwidth or arithmetic limits where the *form* of the kernel — slice-style versus explicit per-direction — barely matters. For long-running production jobs at large grid sizes, though, a small (~5 %) gain is sometimes available by rewriting the bulk dynamic in a **fully-fused form** that mirrors what a hand-coded LBM author would write.

15.5.1 The technique

Three textual rewrites, all valid GeLB source:

1. **Inline the equilibrium expansion.** Instead of computing `feq[i] = W[i]·rho·(1 + 3·eu[i] + 4.5·eu[i]² − 1.5·usq)` for each direction inside a slice loop, expand it directly into the BGK update

`f.out[i] = (1−ω)·fIn[i] + ω·feq[i]` for each `i`. This removes the intermediate `eu[]` and `feq[]` arrays and gives the compiler 9 independent expressions to schedule.

2. **Hoist shared sub-expressions.** In D2Q9, the four diagonal directions share two scalars: `u0Pu1 = ux + uy` (used by NE and SW) and `u0Mu1 = ux − uy` (used by SE and NW). `f_eq_common = rho·(1 − 1.5·usq)` is shared across all 9 directions.

3. **Pre-multiply constant coefficients.** Replace `ω·W[i]` with named constants so the compiler sees a single literal per direction:

```
const real K0     = TAU_INV * 4.0 / 9.0;
const real K_CARD = TAU_INV * 1.0 / 9.0;
const real K_DIAG = TAU_INV * 1.0 / 36.0;
```

The two key shapes look like this for the rest direction (i=0) and the cardinal east direction (i=1):

```
f.out[x, y | 0] = (1.0 - TAU_INV) * fIn[0]
                + K0 * f_eq_common;

f.out[x, y | 1] = (1.0 - TAU_INV) * fIn[1]
                + K_CARD * (f_eq_common
                            + 3.0 * rho * ux
                            + 4.5 * rho * ux * ux);
```

A complete example lives at `examples/lbm/2d/lid_driven_cavity_2d_bgk_fused.gelb` — identical physics to the canonical `lid_driven_cavity_2d_bgk.gelb`, just with the bulk dynamic written this way. Output agrees with the original to within one ULP, so it works as a drop-in replacement.

15.5.2 Going further: incompressible algebra

For low-Mach flows (Ma 0.1), one more rewrite is available: work with **momentum** (`u_x = sum(c_x · fIn)`, no division by rho) instead of velocity, and with `delta_rho = rho − 1` instead of `rho`. This is the **incompressible** LBM equilibrium.

The win is one fewer `vdivpd` per cell — the compressible BGK needs `ux = …/rho` and `uy = …/rho` which are expensive on modern x86 (~30 cycles each). The incompressible form sidesteps both divisions.

The cost is a small accuracy delta of `O(Ma²)`: at the LDC's `U_LID = 0.1` (Ma ≈ 0.17), the per-step cell values differ from the compressible form at the third decimal place but converge to the same steady-state flow within plotting precision.

A complete example is at `examples/lbm/2d/lid_driven_cavity_2d_bgk_incomp_fused.gelb`.

15.5.3 Measured gains

On the Zen 4 AVX-512 reference system (AMD Ryzen 9 7945HX, 16 physical cores), LDC N=512, 5000 steps, OpenMP at 16 threads, `--write-every=5000` to isolate dynamic-block throughput from gauge I/O. Three samples per variant, median reported:

Variant	MLUPS	vs default
Default (slice-style, with `Eq`)	1700	1.00 ×
Fused-compressible (CSE only)	1755	1.03 ×
Fused-incompressible (no 1/rho)	1801	1.06 ×

Going from the default to the fused-incompressible form recovers ~6 % at 16 T. Most of that comes from removing the two divisions; the CSE rewrite alone is worth ~3 %. The remaining gap to a hand-tuned C kernel (about 50 %) lies in

the generated code, not in the source-level shape, and is not reachable from the `.gelb` level.

15.5.4 When to bother

Use the fused form when:

- The bulk dynamic is the dominant cost (large grid, low gauge frequency, long run).
- You're already happy with the algebra and the example will run unchanged for many wall-clock-hours.

Stick with the textbook `Macro` / `Eq` user-function form when:

- You're prototyping or teaching: the slice-style code reads closer to a paper's notation.
- Numerical experiments: changing the equilibrium or collision is two lines in `Eq`, vs nine lines in the fused form.
- The kernel isn't the bottleneck — at small grids or with per-step gauges enabled, the gauge sweep + I/O dominates.

The default form should always be your starting point. Reach for the fused form only when `-w` is large, the simulation is running long enough that 6 % matters, and you've measured that the bulk dynamic is the bottleneck.

CHAPTER 16

Validation and Convergence

A simulation that looks right is not necessarily right. This chapter describes how to validate GeLB simulations against analytical solutions, using the convergence-rate methodology from *Poiseuille with Body Force* and the validation script `scripts/validate_analytical.py`.

16.1 The validation script

```
uv run python scripts/validate_analytical.py
```

This script runs several GeLB examples at multiple grid resolutions, compares the numerical output against exact analytical solutions, computes L2 error norms, and plots convergence curves. Its default run covers three families of cases (`--cases` selects a subset):

1. **Poiseuille flow**: parabolic velocity profile in a channel — in 2D (MRT, body force) and in 3D with full-way bounce-back walls (the wall midway between the nodes), and in 2D with Zou-He walls (the wall on the boundary node). Expected: O(h²).
2. **Taylor-Green vortex decay** (2D BGK and MRT): exponential decay of a sinusoidal velocity field. Expected: O(h²) until the time-discretization floor is hit around N=256.
3. **Couette flow** (2D BGK and MRT, with bounce-back or Zou-He walls): linear velocity profile between two walls moving in opposite directions.

Two further cases, `ra_critical_2d` and `ra_critical_3d`, sweep the critical Rayleigh number of *Rayleigh-Bénard Convection*; they are expensive and run only when asked for.

16.2 Poiseuille convergence — the reference result

The Poiseuille MRT convergence study from *Poiseuille with Body Force* is the cleanest validation in GeLB. The figure in that chapter shows:

- L2 error decreasing as h^2 on a log-log plot (slope ≈ 2.0)
- The error is measured against the exact parabolic profile $u(y) = G/(2\nu)\, y(H - y)$ at a single x-column in the fluid interior

- The simulation is run long enough for the flow to reach steady state: $t > 5H^2/\nu$

This second-order convergence is the hallmark of a correct MRT implementation with bounce-back walls and Strang-split Guo body forcing.

16.3 Taylor-Green vortex decay

The Taylor-Green vortex is an exact solution of the incompressible Navier-Stokes equations:

$$u_x(x, y, t) = -u_0 \cos(k_x x) \sin(k_y y)\, e^{-\nu(k_x^2+k_y^2)t}$$
$$u_y(x, y, t) = +u_0 \sin(k_x x) \cos(k_y y)\, e^{-\nu(k_x^2+k_y^2)t}$$

where $k_x = k_y = 2\pi/N$ and the domain is doubly periodic. The velocity amplitude decays exponentially, and the LBM should reproduce the decay rate to within its spatial discretization error.

At coarse grids (N < 64), the L2 error decreases as h^2. At finer grids (N > 256), the error plateaus because the time-discretization error of the LBM (which is first-order in Δt) becomes dominant. This floor is a known property of single-relaxation-time LBM and is not a GeLB bug.

16.4 Numerical conformance

Physical validation says the model is right; the tests below say the compiler is. They establish every guarantee of *Numerical Semantics*, on all three backends, and run with the rest of the test suite (`uv run pytest`; the CUDA cases need a GPU and are marked `gpu`). Four families:

- **The expression battery** (`gelb/tests/numeric`): a corpus of C++ expressions, translated to GeLB, is compiled strictly and run on each backend in four surfaces (a scalar, a slice, an array declaration, an array assignment) and in both pass modes, and every stored value is compared *bit for bit* with an independently compiled C++ oracle of the same expression; the same expressions are also folded as constants and compared. It covers the widths and conversions, integer and real arithmetic, signed zeros, and the special values (an infinity, a NaN, a negative zero, IEEE `min` and `max`).
- **Per-decision programs** (`gelb/tests/integration/test_real_width_contract.py`): one small program per rule of *Numerical Semantics*, run on three backends and compared with the rule's value — the build modes and the rejected flags, the promotion table and the intrinsic widths, the exact `mod`, `min`/`max` and integer powers, folding, constant arrays in their declared width, the library functions' identical bits in every cell, the random-number draw order and count, and one regression per optimizer invariant.
- **Reproducibility** (`test_reproducibility.py`): an ordinary D2Q9 program and two sensitive programs (a reduction whose fused and unfused results differ, the six library functions at one argument) across thread counts, storage layouts, CUDA block sizes, thread coarsening and tiling, compared bit for bit within a build mode, with the paths and the sensitive arithmetic verified inside the generated kernels.
- **Accuracy** (`test_math_library_accuracy.py`): each library function on each backend against a 60-digit reference, asserted within that implementation's documented bound; the measured maxima are printed with `-s`.

16.5 Known limitations

Couette flow with MRT. The BGK Couette cases reproduce the linear profile to round-off, but the MRT ones converge more slowly than second order: at about order 1.5 with bounce-back walls and order 1 with Zou-He walls, from relative errors of 5×10^{-7} and 2.5×10^{-6} at $N_y = 16$ (`examples/VALIDATION.md`).

16.6 Adding your own validation

To add a new analytical test case to the validation script:

1. Write a function that returns the exact solution at a given grid resolution, timestep count, and physical parameters.
2. Write a function that compiles and runs the corresponding `.gelb` example at those parameters, reads the NetCDF output, and returns the numerical solution.
3. Compute the L2 error: $\|u - u_{\text{exact}}\|_2 / \|u_{\text{exact}}\|_2$.
4. Run at 4–5 grid resolutions (e.g. N = 16, 32, 64, 128, 256) and plot the error vs h on a log-log scale. The slope gives the convergence order.

The existing functions in `validate_analytical.py` (`run_poiseuille`, `run_taylor_green`) serve as templates.

CHAPTER 17

Numerical Semantics

This chapter states what a GeLB program's floating-point results are, and which of them you can count on. It is the authoritative statement of GeLB's numerical semantics: where the compiler disagrees with it, the compiler is wrong. The rule it follows is *minimal surprise*: the semantics are the conventional ones (IEEE 754 binary32 and binary64, round to nearest with ties to even, evaluated as written, with no program changing the rounding mode), each guarantee below is one the compiler is tested for on every backend, and anything not promised here is **unspecified**, not merely undocumented.

Each section ends with a note naming the tests that establish it (*Validation and Convergence* describes the test families). What this chapter leaves unspecified, in one place: results across backends beyond what the section *What must agree* states (reductions, real powers and library functions may differ between backends); results across compilers, compiler versions, CPU and GPU models; a NaN's sign and payload; the sign of an exactly zero reduction; the order of a reduction's terms (fixed per backend, but not stated); integer overflow, an integer zero divisor and an out-of-range real-to-integer conversion at run time (a real divided by zero follows IEEE 754); and anything a fast-math build does.

Three terms recur. A **plain build** is a program compiled with GeLB's two optional passes off (`--no-slice-simplify --no-scalar-cse`); constant folding and lowering still run, so a plain build is the reference for those passes, not an independent semantics (the independent reference is the C++ oracle of the test battery). An **execution configuration** is the backend, the native compilers and their flags, the target instruction set and SIMD width, the storage layout, the thread count and the CUDA launch options (block size, CUDA graphs, `--cuda-coarsening`, `--cuda-tiled`); the generated code is not part of it, since a default and a plain build generate different code on purpose. The **IEEE basic operations** are `+ - * /`, negation, `sqrt`, comparisons and conversions between numeric types; IEEE 754 defines each of them exactly, correctly rounded.

17.1 Build modes

Every backend has three build modes, chosen on the `gelb-run` command line. They differ in what the native compiler and GeLB's own runtime helpers may do to your program's floating-point arithmetic.

Default build. On the CPU backends (`cpu_highway`, `cpu_fortran_serial`) the native compiler runs with contraction off (`-ffp-contract=off`), so it never fuses a multiply and an addition on its own: every `+` and `*` you wrote rounds once. The one fusion GeLB decides itself is inside its reduction helper (`dot_prod` with coefficients other than ±1), which uses a fused multiply-add on targets that have one, and then on every code path alike: the SIMD body, the peeled edges and the scalar tail of a `cpu_highway` loop round the same way, so two cells with identical inputs

give identical bits whatever their position. Every term of a reduction counts, including those with a zero coefficient, so an infinity or a NaN in any direction reaches the result. On the CUDA backend a default build keeps nvcc's fusion, because it is worth 7–15 % on the GPU; all threads of a kernel run the same instructions, so identical cells agree, but a CUDA default build is bitwise reproducible only against another build of the same kernel code — not across `--cuda-coarsening` or `--cuda-tiled`, and not against the plain build below.

Strict build (`gelb-run --strict-fp`). No multiply-add is fused in the generated code or in GeLB's runtime helpers: `gelb-run` passes `-ffp-contract=off` to the CPU compilers and `-fmad=false` to nvcc, and the generated C++ defines `GELB_STRICT_FP` before including the runtime header, which then accumulates its reductions with a separate multiply and addition. No value-changing native flag is accepted either: `-F` with `--strict-fp` is an error, and so is a `GELB_CXXFLAGS`, `GELB_FFLAGS` or `GELB_NVCC_FLAGS` override that carries `-ffast-math`, `-Ofast`, `--use_fast_math`, a flush-to-zero or an approximate-division flag — `gelb-run` names the flag and stops rather than edit your flag string. What a strict build does *not* reach is the inside of the math library: `sin`, `exp` and the rest are each backend's implementation, and whatever they fuse internally is part of that implementation. Use a strict build when you compare runs bit for bit on one backend — across launch configurations on CUDA, or default against plain. Strict mode removes GeLB's fusion and the native compiler's contraction as sources of variation; it does not by itself guarantee identical results across backends: the section *What must agree* says which operations do, and reductions, real powers and library functions may differ. What a strict build costs depends on the program and the backend: on a Ryzen 9 7945HX (`cpu_highway`, five interleaved rounds of 2000 steps) it was 1.3 % on channel flow with a circle obstacle and indistinguishable from the default on a 64^2 Taylor–Green vortex, whose run-to-run scatter is wider than any difference; on CUDA it gives up nvcc's fusion, 7.5–30.6 % on the examples measured (*Performance and Backends*).

Fast math (`gelb-run -F`). The native compiler's fast-math mode (`-ffast-math`, nvcc `--use_fast_math`): re-association, reciprocal approximations, no signed zeros, flushed subnormals, approximate transcendentals on CUDA. On CUDA this concerns `real32` operations (flush-to-zero, the approximate intrinsics); the binary64 programs measured did not change under it. Not reproducible and not covered by anything in this chapter.

Fusion rounds once instead of twice, so it is usually at least as accurate; what it gives up is reproducibility and some symmetries. In `x*x - y*y` with `x == y`, fusing one product leaves the other's rounding error, so the result can be a tiny non-zero — even negative, so that its `sqrt` is NaN. If a result like that must be exactly zero, build strictly.

> **Note**
>
> Tested by `test_real_width_contract.py`: the default build runs the native compiler without contraction; the strict build reaches the compiler and the generated source; `-F` and `--strict-fp` exclude each other; a strict build wins over an ordinary flag override and rejects fifteen value-changing spellings; the fusion probe (a `dot_prod` with coefficients `1, 1, 1, 1, 1 + 2^-52, 1, 1, 1, 1` over `-1, 0, 0, 0, 1 - 2^-52, 0, 0, 0, 0`, which is exactly `-2^-104` when its multiply-add is fused once and `0` when it is not) fuses nothing in a strict build and gives identical cells in a default build on every target.

17.2 Types and promotion

`real` is binary64 and `real32` binary32; `int` is a 64-bit integer, and `int8`, `int16` and `int32` are as named. An integer literal is `int` and a real literal is `real`, whatever the lattices' type; a typed constant has its declared type. `real16` (IEEE 754 binary16, half precision) is a **reserved keyword**: a declaration, lattice type or cast spelled with it is rejected with a diagnostic.

One table gives the type of every arithmetic expression, and it is the same table on every backend:

- If either operand is real, the result is real, of the widest real operand's width; an integer operand converts to that width, so `int + real32` is `real32` and is computed in binary32.
- Otherwise the result is integer, of the widest operand's width. Narrow integers are not first widened: `int8 + int8` is `int8`, while `int8 + 1` is `int` because the literal is.

- `real32` with `real` is `real`; so is `real32 * 1.0`, whose literal is `real`. To keep an expression in single precision, make its literals `real32` constants or write `real32(1.0)`.
- In arithmetic, `bool` is 0 or 1: `bool` with a numeric type takes that type, and `bool` with `bool` gives `int`.
- Comparisons give `bool`. Two `bool` values compare with `==` and `!=` only: they have no order, so `<`, `>`, `<=` and `>=` on `bool` operands are type errors (order them as 0 and 1 with `int()` if that is meant). Integer division truncates toward zero.

Every operation rounds to its type: a `real32` product is one binary32 rounding, and no intermediate is kept wider. A conversion happens where the table says, and at a store into a narrower variable or lattice. Each conversion is one rounding, the same on every backend:

- An integer converts to a real width directly, rounded once to that width and never through binary64 first: `real32(2^54 + 2^30 + 1)` is `2^54 + 2^31`, the nearest binary32, where rounding to binary64 first would give `2^54`.
- A real converts to an integer by truncation toward zero: `int(-3.9)` is `-3`. A NaN, an infinity or a value outside the integer's range is a compile-time error where the compiler sees the conversion and undefined at run time, as in C and Fortran.
- An integer converts to a narrower integer by keeping its low bits: `int8(300)` is `44`, as in C++20, Java and Rust's `as`.
- A real converts to a narrower real by rounding to nearest, ties to even; a value past the narrower range becomes an infinity, at run time and in a folded constant alike (see *Constant folding*).

The intrinsics follow the same table. `min`, `max` and `mod` convert both arguments to their common type and return it, so `min(w, 1)` with `w` a `real32` is `real32`. `atan2` converts both arguments to the common *real* type (`real` for two integers) and returns it. `abs` returns its argument's type, integers included, so `abs(x - 3) / 2` is an integer quotient. `sqrt`, `exp`, `log`, `sin`, `cos`, `tan` and `tanh` return the width of their real argument, `sqrt` of a `real32` being a `real32`; an integer argument is converted to `real` first. These are the conventions of Fortran's generic intrinsics, C++'s overloads, NumPy, Julia and CUDA.

At run time these intrinsics are one helper each, the same on every backend and every code path (Highway's SIMD body and scalar tail, CUDA, Fortran): `mod` is the truncated remainder with the sign of the dividend, for integers and reals alike, so `mod(-7, 2)` is `-1` and `mod(-7.5, 2.0)` is `-1.5` (Fortran's `MOD`, C's `%` and `fmod`); integer `min` and `max` are integers; real `min` and `max` are IEEE 754-2019 minimum and maximum, so a NaN operand gives NaN and `-0.0` is less than `+0.0`; an integer raised to an integer is exact integer arithmetic, with `x^-n` being `1 / x^n` in integer division as in Fortran, so `2^(-1)` is `0`; an integer converted to a narrower integer (`int8(x)`) wraps. An integer zero divisor, `0` to a negative integer power and an integer overflow are undefined at run time, as in C and Fortran; a real divided by zero, or `mod` of a real by zero, follows IEEE 754 (an infinity or a NaN), and so does a real overflow. The compiler rejects the zero divisors it can see, for integers and reals, and the integer overflows it can see; a real overflow it can see folds to the infinity (see *Constant folding*).

Note

Tested by `gelb/tests/unit/type_inference/test_type_rules.py` (a test per row of the table, the passes' agreement on programs, the `real16` spellings) and by `test_real_width_contract.py`: `sqrt` of a `real32`, `min` of a `real32` and an integer, `abs` of an integer, `real16` rejected end to end, the integer intrinsics and casts on the coordinate, IEEE `min`/`max` with a NaN and a signed zero on every path; and by the battery's width, conversion and special-value cases against the C++ oracle.

17.3 Special values

IEEE 754 semantics hold in default and strict builds: a zero keeps its sign, infinities and NaNs propagate, and subnormals are preserved (no backend flushes them in those builds). A NaN's sign and payload are unspecified, as in IEEE 754. The floating-point exceptions (invalid, division by zero, overflow) propagate silently into the output, as C, Fortran and CUDA do by default; the one exception is the compile-time error for a zero divisor the compiler can see. Constant folding follows the same rules: an operation on constants that overflows or is invalid (`exp(1000.0)`, `log(0.0)`, `sqrt(-1.0)`, `1.0e300 * 1.0e300`, `real32(1.0e300)`) folds to the infinity or the NaN the run time would give, and every backend writes that value explicitly (Fortran's `ieee_value`, C++'s `std::numeric_limits`), never as a spelled-out `inf` and never as the invalid call itself, which a native compiler would reject (see *Constant folding*).

Note

Tested by the battery's special-value cases (an infinite, a NaN and a negative-zero input through the operators, `min`/`max` with a NaN and mixed zeros, a reduction with an infinity in a zero-coefficient direction), by `test_real_width_contract.py` (a real zero keeps its sign through the optimizer, a NaN or an infinity in a zero-coefficient direction propagates, a real narrowed past its range gives an infinity), by `test_const_eval.py` (the folder writes a non-finite result as its IEEE value) and by `gelb/tests/unit/midware/test_nonfinite_folding.py` (the folded special values reach the output on three backends).

17.4 What the optimizer promises

Expressions are evaluated as written: parentheses and left-to-right associativity decide the grouping, GeLB never re-associates in a default build, and the native compilers run without re-association. GeLB's own optimization passes never change a result. A default build is bitwise identical to a *plain* build on the CPU backends in default and strict builds, and on CUDA in a strict build: the same values, signed zeros, infinities and NaNs included (a NaN's sign and payload aside). This is checked on every backend against an independently compiled C++ reference, at zero tolerance, by the compiler's test suite. The passes apply only identities that hold bit for bit (`1*x`, `-1*x`, `x/1`, `-(-x)`; a real zero only where its sign cannot change, as in `x + (-0)`, `x - (+0)` and `(-0) - x`; every zero identity in integer arithmetic), fold literal-only `+ - * /` in the expression's width (never to an infinity or a NaN, which are left to run time), and share identical *ordered* sub-expressions that have no side effects. Three invariants hold in every build: an array-valued expression stays array-valued and only scalar statements take part in the scalar sharing; an expression is shared only while every name it reads has one value, so a run of shared expressions ends at an assignment to a name it has read; and a subtree containing a stateful call (`random()`) is never shared, dropped, duplicated or moved across another such call, inside the arguments of intrinsics too. Rewrites that re-associate floating-point operations, such as the BGK collinearity rewrite of `x - C*(x - y)` into `(1-C)*x + C*y`, which loses Sterbenz exactness near `x == y`, are not performed.

> **Note**
>
> Tested by the battery (`gelb/tests/numeric`), which compares both pass modes with the oracle bit for bit on every backend, and by `test_real_width_contract.py`: the scalar CSE does not share a value across a write, a shared `real32` subtree stays `real32`, whole-array assignments stay arrays, and the pipeline test keeps the collinearity rewrite off.

17.5 Library functions

`sin`, `cos`, `tan`, `tanh`, `exp`, `log` and `atan2` are each backend's own implementation, at the argument's width, and one implementation serves every code path of a backend, so two cells with the same argument give the same bits whatever their position:

- `cpu_fortran_serial` and `gpu_cuda_cpp` call the platform's library (glibc's scalar `libm`, the CUDA math library). The generated Fortran marks a loop that calls a library function `!GCC$ novector`, since gfortran would otherwise vectorize it through glibc's vector library, a different implementation (four ULP) in the vector body than in the loop's remainder.
- `cpu_highway` uses Highway's vector math for binary64 in the SIMD body and, through a one-lane vector, in the scalar tail and the edge cells of a loop. Highway's functions are valid on a range (Highway 1.4.0: `sin` and `cos` for a magnitude up to 39000, `exp` for x up to +706, `log` on positive finite x); outside it, and for a NaN or an infinite argument, the platform's function serves that lane, chosen by the argument's value, never by the cell's position. Highway has no `tan`, so the platform's `tan` serves every lane. A `real32` argument uses the platform's library.

Accuracy is that of the implementation, and no general bound is promised. What each implementation states is a measured maximum, in its own metric, not a guarantee: the CUDA Math API tables (CUDA 13.2) give the largest observed distance, in ULPs, from the correctly rounded result (binary64: `exp`, `log` and `tanh` 1; `sin`, `cos` and `tan` 2) and say the figures come from extensive but not exhaustive testing; glibc's manual (2.39) lists known maximum errors from the correctly rounded result (x86-64 binary64: `sin`, `cos`, `exp` and `log` 1, `tanh` 2; `tan` has no listed error); Highway's `math-inl.h` (1.4.0) gives a maximum ULP delta from the platform's `std::` function on sampled inputs (`exp` 1, `sin` and `cos` 3, `log` and `tanh` 4). `sqrt` is correctly rounded everywhere (IEEE 754). The test suite measures each function on each backend against a 60-digit reference in the implementation's own metric and asserts that implementation's figure; the distance from the unrounded value is recorded alongside, not asserted. A function of a constant is folded with the compiler host's library (next section), so a constant argument and a variable one can differ in the last bits; the folded values, in binary64 and in binary32, are measured against the same reference.

> **Note**
>
> Tested by `test_real_width_contract.py` (a uniform argument gives identical bits in every cell for the six functions on three backends; beyond Highway's range the platform's value serves the lane) and by `test_math_library_accuracy.py` (each function on each backend, and each folded value in binary64 and binary32, against a 60-digit reference, within the implementation's documented figure in its own metric).

17.6 Reproducibility

Within one build of one program, the result does not depend on how the run is configured, except where this chapter says so:

- **Cell position.** Two cells with identical inputs give identical bits whatever their position: in Highway's SIMD body, in a peeled edge or in the scalar tail of a loop, on any CUDA thread, in any Fortran loop iteration. Reductions, library functions and the intrinsics are all covered (the sections above say how).

- **Threads and storage layouts.** On `cpu_highway` the same bits are produced with 1, 2 or 4 OpenMP threads and with each of the `soa`, `aos` and `cbt` storage layouts, in default and strict builds alike. `cbt` exists only where the program is compiled for AVX-512: elsewhere `gelbc` rejects it, and a `cbt` program does not compile.
- **CUDA launch configuration.** In a strict build the same bits are produced with any block size, with `--cuda-coarsening 4` and with `--cuda-tiled`. In a default build, which keeps nvcc's fusion, the promise is narrower: the same bits across block sizes, which run the same kernel code; across coarsening and tiling only within each variant, since nvcc may fuse differently in a restructured kernel. On the GPU the tests were run on, the coarsened and the tiled variants gave the same bits as the plain one, for an ordinary D2Q9 program, for the fusion probe and the library probe described in the test note below, and for a program written to provoke a difference (a product in the inner-axis coordinate added to a cell value, which a coarsening loop may hoist); the default build did fuse the probe's reduction (every cell gave the fused value, alike in every variant). That is an observation, not a promise.
- **Default and plain builds** of one program draw the same random numbers and, on the CPU backends and in CUDA strict builds, give the same bits; the optimizer section says why.
- **Seeds.** The same seed, program, options and configuration reproduce a random program; a different thread count, layout, SIMD width or backend draws a different stream.

Nothing is promised across backends beyond what *What must agree* states, nor across compilers, compiler versions, or CPU and GPU models.

Note

Tested by `test_reproducibility.py` (the configuration matrix on `cpu_highway` and `gpu_cuda_cpp`, for an ordinary D2Q9 program and for two sensitive programs on a two-dimensional grid with identical inputs in every cell: the *fusion probe*, a `dot_prod` with coefficients `1, 1, 1, 1, 1 + 2^-52, 1, 1, 1, 1` over `-1, 0, 0, 0, 1 - 2^-52, 0, 0, 0, 0`, whose result is `-2^-104` when fused once and `0` when not, and the *library probe*, `sin`, `cos`, `tan`, `tanh`, `exp` and `log` at one argument each — computed in the gauge and, in a second pair, in the dynamic block so that CUDA's coarsened and tiled kernels run the sensitive arithmetic, which the generated kernel is checked to contain; every cell agreeing; and the coordinate-product probe) and by the same two programs in `test_real_width_contract.py` (identical bits by cell position on three backends).

17.7 What must agree

The table below says which results must be bitwise identical. Comparisons apply to the same typed operand values and defined inputs; a difference introduced upstream by a library function or a real power may propagate, and integer overflow and invalid real-to-integer conversions are undefined, not compared.

Operation	Default vs plain build, same execution configuration	Across backends, strict builds	Across backends, default builds
IEEE basic operations	bitwise (CUDA: strict builds)	bitwise	may differ (fusion)
folded constants	bitwise	bitwise	bitwise (same compiler host and toolchain)
`min`, `max`, `mod`	bitwise	bitwise	bitwise
integer powers	bitwise	bitwise	bitwise
real powers `x^n`	bitwise	may differ	may differ
reductions	bitwise (CUDA: strict builds)	may differ (order)	may differ
library functions	bitwise	may differ	may differ
`random()`	bitwise, same seed	not compared	not compared

A real raised to a power is backend-defined: `x^n` with a compile-time integer exponent from 0 to 8 is multiplied left to right on Highway and CUDA, a variable exponent is not unrolled, and Fortran uses `**`; no bound on the difference between backends is promised, and within a backend default and plain builds agree because GeLB never rewrites a power. Within one backend and build, identical inputs give identical results whatever the cell's position, for arithmetic, reductions and library functions; for non-random programs the same holds across the execution configurations of one backend, except that CUDA default builds agree only where the kernel code is the same (see *Reproducibility*).

17.8 Random numbers

`random()` returns a `real` in [0, 1); each call draws the next number of the stream. In a slice or array expression it draws once per element, in element index order, as NumPy's `rng.random(9)` or a C loop would; to use one draw for a whole slice, draw into a scalar first (`real r = random(); f.out[x | :] = W * r;`). Several calls in one expression draw left to right, and a call in an array expression draws all its elements before the next call: `random() - random()` over nine elements draws nine numbers and then nine more. The compiler sequences the draws of a statement itself, so this holds on every backend, whatever C++ or Fortran leave unspecified, and in a default and a plain build alike.

The same seed (`gelb-run --seed`), the same program, the same GeLB options and the same execution configuration reproduce the same numbers. Nothing is promised across execution configurations: a different thread count, storage layout, SIMD width or backend draws a different stream (each backend has its own generator: xoshiro256+ on `cpu_highway`, one vector of lanes per draw; cuRAND's XORWOW per thread on `gpu_cuda_cpp`; Fortran's `random_number` on `cpu_fortran_serial`).

Note

Tested by `test_real_width_contract.py`: a slice draws once per element in every build; several draws in one expression equal their hand-sequenced form bit for bit, inside an intrinsic argument too; draws inside and around user-function calls (an argument, a function body, nested calls) keep their source order; an array constructor's elements are scalar draws, and a broadcast `sqrt` or `min` of a draw draws once per element; a draw whose contribution simplifies away still happens; a seed reproduces; default and plain builds draw alike.

17.9 Constant folding

An expression whose operands are all constants (literals, constants, and intrinsics of such) is evaluated once at compile time, by one evaluator shared by the folder, the geometry's `where` predicates and the optimizer. A folded value is the value the same expression has in the same width at run time, with these rules:

- Arithmetic, conversions, `abs`, `min`, `max`, `mod`, `sqrt` and integer powers are exact or correctly rounded in the expression's width: `mod(-7, 2)` is `-1` (the truncated remainder, as at run time), `2^(-1)` is `0` (an integer power with a negative exponent is `1 / x^n` in integer division, as in Fortran), `int(-3.9)` is `-3`, `min(0.0, -0.0)` is `-0.0`.
- An integer result that does not fit its width (`9223372036854775807 + 1`, `2^63`, `abs` of the minimum integer) is a compile-time error, and so is a real converted to an integer when it is NaN, infinite or out of range (`int(1.0e300)`, `int8(300.0)`). A conversion between integer widths wraps (`int8(300)` is `44`). Division or `mod` by a zero the compiler can see, for integers and reals, and `0` raised to a negative integer power, are errors too. At run time the integer operations are undefined, as they are in C and Fortran, and the real ones follow IEEE 754.
- The library functions (`exp`, `log`, `sin`, `cos`, `tan`, `tanh`, `atan2`, and a real raised to a power) fold with the compiler host's binary64 library, on arguments already rounded to their width, and the result is rounded once to the expression's width: `sin(W)` with `W` a `real32` is the binary64 sine of `W`'s value rounded to binary32. For one host and toolchain every backend shares the folded value. The same function of a *variable* is the backend's own implementation (see *Library functions*), so a constant argument and a variable one can differ in the last bits; no bound on that difference is promised. A call outside the function's domain or range (`sqrt(-1.0)`, `log(0.0)`, `exp(1000.0)`, a real base raised to a power that overflows) folds to its IEEE 754 result, a NaN, `-Inf` or `+Inf`, the value the run time gives; the backends write it as an explicit special value, so no native compiler sees `sqrt(-1.0)` as a constant argument.
- A reduction of constants (`sum`, `dot_prod`) and `matmul` of constants are evaluated term by term in the expression's width, by the rules above: an integer sum is exact and its overflow an error; a `real32` dot product rounds each product and each sum to binary32; a real result that overflows is the infinity. `inverse` and `determinant` of constants are evaluated by SymPy at binary64 precision and rounded once to the declared width.
- A real narrowed past its range (`real32(1.0e300)`, or a `real32` declared from a binary64 value that large) folds to the `real32` infinity, the value the conversion gives at run time. A real converted to an integer is still an error when it is NaN, infinite or out of range, a folded infinity included (`int(1.0e300 * 1.0e300)`).
- A block-local `const` is, after folding, a typed local declaration holding its value, a special value included; its immutability is checked before, by the frontend.
- A constant array's elements are the values its declared width stores, on every backend and in every reading: `const array[int8, 2] A = {300, 1}` holds 44 and 1, a `real32` array holds the binary32 roundings of its literals. An element a narrow real width cannot hold, written as a literal (`1.0e300` in a `real32` array), as a conversion (`real32(1.0e300)`) or as an expression that overflows (`1.0e300 * 1.0e300`), is an infinity, and a reduction or `matmul` over such an array folds with it (`sum` of it is the infinity). `inverse` and `determinant` of a matrix with such an element are errors: they are evaluated exactly and have no run-time form.

Note

Tested by `gelb/tests/unit/midware/test_const_eval.py` (a rule per test on the evaluator; the folder, the geometry evaluator and the simplifier agree; the diagnostics) and by `test_real_width_contract.py`: folded `mod`, powers and `real32` library calls on three backends, reductions of constants in their width (an exact integer sum, a `real32` dot product, an overflowing sum giving the infinity), a real narrowed past its range giving the infinity, constant arrays holding their declared width (`int8`, `int16` and `real32` elements, an overflowing `real32` element written as a literal, a conversion or a product, in a 1-D and a 2-D array, read, summed and multiplied), five constants this chapter makes errors rejected by `gelbc`; by `gelb/tests/unit/midware/test_nonfinite_folding.py` (programs with `sqrt(-1.0)`, `log(0.0)`, `exp(1000.0)`, an overflowing product and a narrowed `real32`, as literals, global constants, constant arrays and block-local constants, folded to their special values and read back from the output on three backends); and by the battery's constant forms.

17.10 Not in this version

Some controls are planned and deliberately absent, so that nothing here is promised before it is implemented and tested on every backend:

- `real16`: a declaration, lattice type or cast spelled with it is rejected with a diagnostic that says so.
- A user function called in a `global_constants` initializer; the call is rejected with a diagnostic that says so. A global constant is an expression of literals, constants and intrinsics.
- An array-valued gauge output (`-> (array[real, 2] u)`), rejected with a diagnostic that says so; a gauge writes one scalar field per declared output.
- Opt-in relaxations, each meaning exactly what its name says, as gcc's do: `--reciprocal-math` (`a/r; b/r` as `t = 1/r; a*t; b*t`), `--no-signed-zeros` (`x + 0` as `x`), `--finite-math-only` (no infinity or NaN assumed; with `--no-signed-zeros`, `0*x` as `0`; zero-coefficient reduction terms skipped), `--associative-math` (regrouping, the collinearity rewrite, combining literals) and `--flush-subnormals` (FTZ/DAZ on the CPU backends, nvcc `--ftz=true`, which flushes `real32` only); a `--fast-math` that sets all of them, passing the matching native flags but not nvcc's `--use_fast_math`, which also selects approximate division, `sqrt` and math functions. A strict build with any relaxation will be rejected with an error naming both flags, and the optimizer's three invariants above hold under every relaxation.
- `--wide-intermediates`, evaluating `real32` subexpressions in binary64 and rounding at the store; `--fpe-trap`, trapping the first floating-point exception where the platform supports it (not on CUDA); `--check-overflow`, trapping integer overflow at run time as gcc's `-ftrapv` does; `--no-fold-transcendentals`, evaluating a library function of constants at run time instead of on the compiler host.
- A reduction across cells, which must use a fixed-shape tree, not atomics, so that the guarantees above extend to it.

CHAPTER 18

Further Reading

This chapter maps each tutorial chapter to the relevant sections of three standard LBM textbooks, and provides a short bibliography of the original papers cited throughout the tutorial.

18.1 Textbook cross-reference

Chapter	Krüger et al. (2017)	Mohamad (2019)
Hello World	§3.2, §3.3.1–3.3.2	§2.2–2.3
1D Diffusion	§3.2, §8.1, §8.3, §8.5.2.1	§5.1–5.5
Lid-Driven Cavity	§5.3.3, §7.2	§8.3.1, §8.8.1
Poiseuille with Body Force	§6 (Forces), §10 (MRT)	§8.3, §8.8.2, §10.1
Rayleigh-Bénard Convection	§8.4.1–8.4.2	§9.6
Kármán Vortex Street	§11.2.1 (staircase BB)	§8.8.3
Shan-Chen Multiphase	§9.3	§11.1
Beyond LBM: Conway's Game of Life	— (not LBM)	—
Reduced-Gravity Ocean Circulation	— (see Salmon 1999)	—

Krüger et al. = T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva & E. M. Viggen, *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2017.

Mohamad = A. A. Mohamad, *Lattice Boltzmann Method: Fundamentals and Engineering Applications with Computer Codes*, 2nd ed., Springer, 2019.

A third useful reference for geoscience applications is:

Sukop & Thorne = M. C. Sukop & D. T. Thorne, *Lattice Boltzmann Modeling: An Introduction for Geoscientists and Engineers*, Springer, 2006. The tutorial's *Kármán Vortex Street* and *Shan-Chen Multiphase* chapters are ported from Sukop's worked examples.

18.2 Bibliography

Original papers cited in the tutorial, in chronological order:

Bibliography

[ShanChen1993] X. Shan & H. Chen (1993). "Lattice Boltzmann model for simulating flows with multiple phases and components." *Physical Review E*, **47**, 1815–1819. — The pseudopotential multiphase model (*Shan-Chen Multiphase*).

[Ladd1994] A. J. C. Ladd (1994). "Numerical simulations of particulate suspensions via a discretized Boltzmann equation. Part 1. Theoretical foundation." *Journal of Fluid Mechanics*, **271**, 285–309. — Bounce-back boundary conditions (*Lid-Driven Cavity*, *Poiseuille with Body Force*).

[ZouHe1997] Q. Zou & X. He (1997). "On pressure and velocity boundary conditions for the lattice Boltzmann BGK model." *Physics of Fluids*, **9**, 1591–1598. — Velocity/pressure boundary conditions (*Boundary Conditions Cookbook*).

[Salmon1999] R. Salmon (1999). "The lattice Boltzmann method as a basis for ocean circulation modeling." *Journal of Marine Research*, **57**, 503–535. — Reduced-gravity ocean model (*Reduced-Gravity Ocean Circulation*).

[dHumieres2002] D. d'Humières (2002). "Multiple-relaxation-time lattice Boltzmann models in three dimensions." *Philosophical Transactions of the Royal Society A*, **360**, 437–451. — MRT collision operator (*Poiseuille with Body Force*, *Rayleigh-Bénard Convection*).

[Guo2002] Z. Guo, C. Zheng & B. Shi (2002). "Discrete lattice effects on the forcing term in the lattice Boltzmann method." *Physical Review E*, **65**, 046308. — Guo body force scheme (*Poiseuille with Body Force*, *Reduced-Gravity Ocean Circulation*).

[YoshidaNagaoka2010] H. Yoshida & M. Nagaoka (2010). "Multiple-relaxation-time lattice Boltzmann model for the convection and anisotropic diffusion equation." *Journal of Computational Physics*, **229**, 7774–7795. — Anti-bounce-back temperature BC (*Rayleigh-Bénard Convection*).